408 · 概念复习

易错易混与冷门考点

239 个考点，涵盖数据结构、计算机组成原理、操作系统与计算机网络。

完整内容 · 2026-11-06

章节目录

按章节浏览

数据结构53 条

- 基础与复杂度
- 线性结构
- 栈、队列与串
- 数组与矩阵
- 树与二叉树
- 图
- 查找与排序

计算机组成原理56 条

- 计算机概述
- 数据表示
- 运算与浮点
- 存储系统
- 指令与数据通路
- CPU 与流水线
- 总线
- 中断与 I/O

操作系统57 条

- 操作系统概述
- 进程与线程
- 调度与中断
- 同步与死锁

- 存储管理
- 文件系统
- I/O 与磁盘

计算机网络73 条

- 网络体系结构
- 物理层与链路层
- 数据链路层
- 网络层
- 传输层
- 应用层

408 · DS

数据结构

53 条 · 7 个章节

基础与复杂度

逻辑结构、存储结构与抽象数据类型　线性/树/图是逻辑关系；顺序/链式等是存储实现。
记忆概念　易错　易混　　要点

核心结论

逻辑结构描述元素关系：线性、树、图等；存储结构描述内存实现：顺序、链式、索引、散列。ADT 由数据对象、关系和操作组成。

别混淆线性/树/图是逻辑关系；顺序/链式等是存储实现。

详解

01 解析

同一种逻辑结构可有多种存储方式。算法依赖访问接口和存储代价，不应仅按逻辑关系判断复杂度；ADT 可隐藏具体实现。

02 例子

线性表可用顺序表或链表实现，逻辑上都是线性关系。

03 易错点

错误说法链表是一种逻辑结构，顺序表和线性表互不相关。

自检：二叉树只能用链式存储吗?

不能，也能顺序存储，完全二叉树尤其适合数组编号。

复杂度：求和、独立阶段与递归栈　嵌套不必平方；循环工作量求和，递归栈计空间。　　记忆概念　易错　易混　　要点

核心结论

时间复杂度累计实际操作次数；独立阶段相加取主导项，嵌套次数需按内层边界求和。空间通常计辅助空间并包括递归栈。

别混淆嵌套不必平方；循环工作量求和，递归栈计空间。

详解

01 解析

1. 先写总操作次数的求和式，不能只数循环嵌套有几层。
2. i 每轮翻倍、内层运行 i 次时，工作量为 1+2+4+⋯+n，小于 2n，所以是 $O(n)$。
3. 空间另看同时存活的对象和递归深度：遍历退化树需同时保留 $O(n)$ 层调用记录，不能因每层只用几个变量就判 $O(1)$。

补充原理与条件

i 每轮翻倍而内层循环 i 次时，总量可为 1+2+4+⋯=$O(n)$，不是 $O(n\log n)$。O 是渐近上界，不总代表紧确界；平均、最好、最坏要分开。

02 例子

递归遍历一棵退化树，栈深 h=n，辅助空间为 $O(n)$。

03 易错点

错误说法两层循环必为 $O(n^2)$；所有 $O(1)$ 额外数组的递归都为 $O(1)$ 空间。

自检：循环 for(i=1;i<n;i*=2) 的次数是什么量级?

$O(\log n)$；若内层工作变化，须再累计各轮代价。

线性结构

带头结点与不带头结点：空表条件　先确认是否带头结点，再判断空表。　　易错　易混　　要点

核心结论

带头结点：L 存在时，L->next == NULL 表示空表；不带头结点：L == NULL 表示空表。

别混淆先确认是否带头结点，再判断空表。

详解

01 解析

1. 先看 L 指向头结点还是首个数据结点：两种模型的空表指针形态不同。
2. 有头结点时，L 本身仍存在，L->next 为 NULL 才说明没有数据；无头结点时，L 为 NULL 即空表。
3. 写遍历时对应从 L->next 或 L 出发；若题设允许 L 本身为 NULL，先判 L 再访问 L->next。

补充原理与条件

头结点不存放有效数据。计算表长、遍历和删除时，起点必须与题设一致；若允许 L 本身为空，还要先做空指针保护。

02 例子

带头结点的空表仍有一个头结点，但有效元素个数为 0。

03 易错点

错误说法L != NULL 就代表表中有元素。

自检：带头结点求表长从哪里开始?

从 L->next 开始，只计数据结点。

链表操作：先保存后继，再改指针　先存 next，再改链或释放结点。　　易错　近年考查要点

核心结论

覆盖链接前先保存旧后继；free 后不能再读取该结点。

别混淆先存 next，再改链或释放结点。

详解

01 解析

1. 在修改 p->next 之前，用 next=p->next 记住原链的后继。
2. 把 p 接入新链，或将 p 从旧链摘下并释放；释放后禁止再读 p->next。
3. 使用保存的 next 继续遍历。检查每轮结束后，未处理链的入口是否仍可到达。

补充原理与条件

逆置和删除都会改变链路。先保存 next = p->next，再改链接或释放 p，最后用 next 继续。

02 例子

头插逆置中若先令 p->next = L->next，再用 p = p->next，会回到新链而不是原链。

03 易错点

错误说法改完 p->next 后，还能沿原链继续遍历。

自检：删除 p 后，能否使用 p->next?

不能；应在释放前保存后继。

循环队列：牺牲一格还是计数器　满队列判定式与 rear 的含义必须配套。　易错　易混　要点

核心结论

牺牲一格：空为 front == rear，满为 (rear+1)%M == front；计数器法用 count 区分。

别混淆满队列判定式与 rear 的含义必须配套。

详解

01 解析

$\ell=(r-f+M)\bmod M$

牺牲一格模型：f 为队头，r 为下一插入位置，M 为数组长度；容量上限 M−1。

1. 先确认 rear 指向队尾元素还是下一个空槽；本条使用“下一个空槽”约定。
2. 牺牲一格时，空队列 front=rear；再插一项就追上 front 时已满，所以满条件为 (rear+1)%M=front。
3. 求长度用 (rear-front+M)%M，最大为 M−1；若另设 count，满条件改为 count=M，不再牺牲一格。

补充原理与条件

不同约定不能混用。牺牲一格的数组长度 M 只能存 M−1 个元素；rear 通常指向下一插入位置。

02 例子

M=5 的牺牲一格循环队列，最大有效容量为 4。

03 易错点

错误说法front == rear 一定能同时判断空和满。

自检：牺牲一格时如何求元素数?

(rear-front+M)%M。

栈、括号匹配与后缀表达式　括号看种类与顺序；后缀先弹右操作数。　易错　易混　近年考查　要点

核心结论

必须同时检查嵌套顺序与括号种类；右括号应与栈顶左括号配对。后缀求值要区分弹栈顺序。

别混淆括号看种类与顺序；后缀先弹右操作数。

详解

01 解析

1. 括号匹配遇左括号压栈，遇右括号先验栈非空，再核对栈顶类型，最后弹栈。
2. 全串扫完仍有左括号也失败；“数量一样”不能替代顺序检查。
3. 后缀求值遇二元运算先弹右操作数 b，再弹左操作数 a，计算 a op b；减法和除法最能检验顺序是否写反。

补充原理与条件

扫描左括号入栈，右括号时栈空或类型不符则失败；扫描结束栈须空。后缀二元运算第一次弹出的是右操作数。

02 例子

([)] 数量相等但嵌套不合法；后缀 a b − 的运算是 a−b。

03 易错点

错误说法栈里只记括号数量，左右数量相等就说明匹配。

自检：容量如何确定?

括号匹配栈的最大深度由扫描时同时未匹配的左括号数决定。

顺序表：定位、插入、删除的时间　随机访问 O(1)，不代表任意位置插删也是 O(1)。　　记忆概念　易错　易混　　要点

核心结论

按下标访问 $O(1)$；一般位置插入或删除需移动后续元素，最坏和常见平均为 $O(n)$。按值查找无序表通常 $O(n)$。

别混淆随机访问 $O(1)$，不代表任意位置插删也是 $O(1)$。

详解

01 解析

尾插在容量足够时可 $O(1)$，扩容会产生额外开销。只给下标与只给值的操作成本不同；复杂度必须包含定位。

02 例子

长度 n 的表头插入，需要把原有 n 个元素后移。

03 易错点

错误说法顺序表既然能随机访问，任意位置插入删除也都是 $O(1)$。

自检：已知下标删除末元素是否需移动全部元素?

不需要；但删除一般中间位置需移动后续元素。

双向链表与循环链表：四条链接和终止条件　双向操作维护两端链接；循环链表回到起点才结束。　　记忆概念　易错　易混　　要点

核心结论

双向插入需维护双方的前后链接；循环链表通常回到头结点或起点时停止，不能照搬 NULL 终止。

别混淆双向操作维护两端链接；循环链表回到起点才结束。

详解

01 解析

典型 p 后插 q：q->next=p->next，q->prior=p，再更新旧后继 prior 和 p->next。带/不带头结点及空表不同，单独处理边界。

02 例子

有头结点的双向循环空表可令 head->next=head->prior=head。

03 易错点

错误说法双向链表插入只改前驱的 next；循环链表遇 NULL 结束。

自检：删除 p 时只改 p->prior->next 够吗?

不够，还需维护后继的 prior；并处理题设中的边界结点。

栈、队列与串

栈、共享栈与递归：不同操作约定　top 指栈顶还是空位，决定先加减还是先访问。　　记忆概念　易错　易混　　要点

核心结论

top 可指栈顶元素或下一空位，入栈/出栈先后需统一。两栈共享数组时从两端增长，共享空闲空间。

别混淆top 指栈顶还是空位，决定先加减还是先访问。

详解

01 解析

若 top 指栈顶，空栈 top=−1，入栈通常 a[++top]=x；若 top 指下一个空位，则 a[top++]=x。函数递归用调用栈，不能省略终止条件。

02 例子

两端共享栈若 top1+1==top2，表示两栈之间无空槽。

03 易错点

错误说法top 初始为 −1 和初始为 0 的栈可使用同一入栈代码。

自检：共享栈能否要求两边各固定一半容量?

这种固定分区失去动态共享空闲空间的主要作用。

串、子串和模式匹配：长度与结束标志　子串必须连续；C 结束符不计串长。　　记忆概念　易错　易混　　要点

核心结论

抽象串是有限字符序列，长度计有效字符；C 的结束标志 \0 不计字符串字符数。空串长度 0，空格串含实际空格。

别混淆子串必须连续；C 结束符不计串长。

详解

01 解析

子串是连续字符，子序列可以不连续。朴素匹配失配后需重新尝试位置，最坏 $O(nm)$；KMP 用模式信息减少回退，$O(n+m)$。

02 例子

"a b" 长度为 3；"" 为 0；存入 C 数组还要留结束标志位。

03 易错点

错误说法C 风格字符串的字符数包含末尾 \0；空串和空格串一样。

自检：“ac”是“abc”的子串吗?

不是，但可以是子序列。

数组与矩阵

数组地址：行优先与列优先　行优先跨行乘列数；列优先跨列乘行数。　　记忆概念　易错　易混　　要点

核心结论

下标从 (r_0,c_0) 起，R 行 C 列，每元素 L B：行优先偏移=$[(i-r_0)C+j-c_0]L$；列优先偏移=$[(j-c_0)R+i-r_0]L$。

别混淆行优先跨行乘列数；列优先跨列乘行数。

详解

01 解析

$\mathrm{LOC}(i,j)=A_0+\left[(i-r_0)C+(j-c_0)\right]L$

行优先；C 为列数，L 为元素字节数，A_0 对应下标 (r_0,c_0)。

若行下标起点 r_0、列下标起点 c_0，行数 R、列数 C，行优先偏移=$[(i-r_0)C+(j-c_0)]L$；列优先=$[(j-c_0)R+(i-r_0)]L$。L 为元素字节数。

02 例子

C 的普通二维数组通常行优先，a[i][j+1] 与 a[i][j] 相差一个元素大小。

03 易错点

错误说法二维数组的任意存储顺序都是 base+(i×行数+j)×元素大小。

自检：公式里应乘行数还是列数?

行优先跨一行乘列数；列优先跨一列乘行数。

特殊与稀疏矩阵：压缩存储不等于只有一种公式　压缩公式先定上/下三角、行/列优先及下标起点。　　记忆概念　易错　易混　　要点

核心结论

对称矩阵只存一半；三角矩阵可另存共同常数；带状矩阵只存带内元素；稀疏矩阵常用三元组或十字链表。

别混淆压缩公式先定上/下三角、行/列优先及下标起点。

详解

01 解析

$N=\frac{n(n+1)}{2}$

n 阶对称矩阵只存一个三角区且包含主对角线。

上/下三角、行/列优先、0/1 起始都会改变地址公式。三元组保存行、列、值，顺序不当会影响访问；十字链表提供行链和列链。

02 例子

n 阶对称矩阵存下三角含对角线需 $\frac{n(n+1)}{2}$ 个元素，而非 n^2。

03 易错点

错误说法对称矩阵与稀疏矩阵都按下三角连续压缩，零元素不用任何信息。

自检：下三角公式可否直接用于上三角列优先?

不可直接套，必须先确定存储顺序和下标约定。

树与二叉树

表达式树：括号围住子表达式　括号包住整个子表达式，不能分别包住操作数。　个人错题　易错　要点

核心结论

空结点返回；叶子输出操作数；非根的非叶结点通常在整个子表达式外加一对括号。

别混淆括号包住整个子表达式，不能分别包住操作数。

详解

01 解析

根可省最外层括号；括号必须在左递归前打开、右递归后关闭。若题设含一元运算符，应按其孩子约定处理，不能把“仅有右孩子”误当叶子。

02 例子

a+b 的子树应输出 (a+b)，而不是 (a)+(b)。根是乘号时，可输出 (a+b)*c。

03 易错点

错误说法左孩子为空或右孩子为空，就决定是否输出括号。

代码对照

```
typedef struct Node {
    char data;
    struct Node *left, *right;
} Node;

void printExpr(const Node *p, int isRoot) {
    if (p == NULL) return;
    int nonLeaf = p->left != NULL || p->right != NULL;
    if (!isRoot && nonLeaf) putchar('(');
    printExpr(p->left, 0);
    putchar(p->data);
    printExpr(p->right, 0);
    if (!isRoot && nonLeaf) putchar(')');
}
/* #include <stdio.h>；调用 printExpr(root, 1)。
  本例结点 data 为单字符，二元或右孩子承载的一元表达式。 */
```

自检：非叶结点的判定条件是什么?

p 非空且 (p->left != NULL || p->right != NULL)。

高度与最少结点：先确认树的种类　先确认树种类与高度起点；AVL 递推需给初值。　个人错题　易错　记忆概念　要点

核心结论

规定根为第 1 层：普通二叉树最少 h 个、最多 2^h-1 个；AVL 最少结点满足 $N(h)=N(h-1)+N(h-2)+1$。

别混淆先确认树种类与高度起点；AVL 递推需给初值。

详解

01 解析

$N(h)=1+N(h-1)+N(h-2)$

AVL 最少结点数，根为第 1 层；N(0)=0，N(1)=1。

1. 普通二叉树只限制每个结点最多两个孩子，因此高度 h 的单链已有 h 个结点。
2. AVL 还限制两子树高度差至多 1。要在高度为 h 时尽量少放结点，左右子树取 h−1 与 h−2。
3. 连同根得到 N(h)=$1+N(h-1)+N(h-2)$，以 N(0)=0、N(1)=1 向上递推；不要把普通二叉树的 h 当作 AVL 的最少结点数。

补充原理与条件

AVL 的左右子树高度差至多 1。最少结点时取高度 h−1 与 h−2；N(0)=0、N(1)=1。B 树或红黑树不能直接套此式。

02 例子

h=3 时普通二叉树可只有 3 个结点，AVL 至少 4 个，满二叉树有 7 个。

03 易错点

错误说法所有高度为 h 的二叉树都有 2^h-1 个结点。

自检：看到“最少结点”先检查什么?

树的类型、高度从 0 还是 1 开始，以及是否含外部结点。

二叉排序树与平衡树：有序不等于平衡　中序有序不代表树高是对数级。　易混　近年考查　要点

核心结论

二叉排序树的查找复杂度为 $O(h)$；退化成链时为 $O(n)$，平衡树才把高度控制在 $O(\log n)$。

别混淆中序有序不代表树高是对数级。

详解

01 解析

1. BST 只要求“左小右大”，先用中序有序验证键值关系。
2. 查找每次沿一条根到叶路径下降，比较次数受高度 h 控制，所以是 $O(h)$。
3. 有序数据依次插入可令 h=n；只有额外的平衡约束将 h 控制为 $O(\log n)$，才能保证对数级查找。

补充原理与条件

中序有序是键值约束，不是高度约束。插入有序序列到普通 BST 可以产生单链。

02 例子

依次插入 1、2、3、4 到普通 BST，可能形成全右链。

03 易错点

错误说法二叉排序树查找总是 $O(\log n)$。

自检：普通 BST 中序输出有序，能否保证查找对数时间?

不能，仍取决于高度。

线索二叉树：孩子指针与线索指针　先看 tag，再决定递归还是沿线索走。　　冷门　易错　记忆概念　　要点

核心结论

先看 ltag/rtag：孩子才递归，线索指向指定遍历序列的前驱或后继。

别混淆先看 tag，再决定递归还是沿线索走。

详解

01 解析

常用约定 tag=0 为孩子、tag=1 为线索，但要以题设为准。把线索当孩子可形成重复访问或环。

02 例子

中序线索树中，右线索可以直接指向中序后继；右孩子则要找其子树的最左结点。

03 易错点

错误说法所有 left/right 指针都可以递归访问。

自检：普通二叉树递归能直接套在线索树上吗?

不能，递归前须检查标志位。

B 树与 B+ 树：数据记录在哪里　B+ 树查记录必须到叶层；B 树可在内部成功。　　冷门　易混　近年考查　记忆概念　　要点

核心结论

典型 B+ 树的记录或记录指针位于叶结点，内部结点用于索引；B 树可以在内部结点查找成功。

别混淆B+ 树查记录必须到叶层；B 树可在内部成功。

详解

01 解析

1. 若问“能否在非叶结点查找成功”，先区分 B 树与典型 B+ 树。
2. B 树的内部关键字可直接对应记录，命中即可结束；B+ 树内部键起索引作用，查记录通常仍须走到叶结点。
3. 若问范围查询，利用 B+ 树叶链从起始叶继续扫描；不要把内部命中索引键当作已取得记录。

补充原理与条件

B+ 树叶结点按键值链接，便于范围扫描。m 阶的键数/孩子数约定需看题目，尤其根的下限。

02 例子

查一个很靠近根索引键的记录，B+ 树仍需走到叶结点。

03 易错点

错误说法B+ 树的内部结点也用于终止记录查找。

自检：哪种结构更便于顺序范围查询?

典型 B+ 树，利用叶结点链。

哈夫曼树：最优不等于唯一编码　最优 WPL 不代表具体码字唯一。　　冷门　易混　近年考查　记忆概念　　要点

核心结论

哈夫曼算法构造最小带权路径长度的前缀编码树；左右次序和同权选择可产生不同的最优编码。

别混淆最优 WPL 不代表具体码字唯一。

详解

01 解析

n 个叶结点的二叉哈夫曼树（n>1）有 2n−1 个结点，不含度为 1 的结点。前缀性允许无需分隔符逐字解码。

02 例子

交换某结点左右子树可改变码字，但不改变对应路径长度与 WPL。

03 易错点

错误说法权值相同就只能得到同一个编码；编码长度完全相等。

自检：最优 WPL 唯一是否意味着每个码字唯一?

不意味着，具体码字可不同。

B 树高度公式：关键字数与高度上下界　n 是关键字总数，h 不含失败结点；根从第 1 层计。　记忆概念　易错　易混　图解　要点

核心结论

非空 m 阶 B 树，令 $a=\lceil m/2\rceil$：$2a_{h-1}-1\le n\le m_h-1$。n 是关键字总数，h 从根第 1 层计，不含失败结点，且 m≥3。

别混淆n 是关键字总数，h 不含失败结点；根从第 1 层计。

详解

01 解析

$2a_{h-1}-1\le n\le m_h-1$

非空 m 阶 B 树，m≥3，$a=\lceil m/2\rceil$；n 为关键字数，h 不含失败结点。

$\lceil \log_m(n+1)\rceil \le h$

整数高度的下界。

$h\le \left\lfloor 1+\log_{a}\frac{n+1}{2}\right\rfloor$

整数高度的上界；两端同时满足，并不总能唯一确定 h。

1. 先固定符号：m 是最多孩子数，$a=\lceil m/2\rceil$，n 是关键字数，h 不含失败结点。
2. 高度 h 的最满情形有 m_h-1 个关键字；最稀疏情形根有 1 个关键字、2 个孩子，下层按最小分支扩展，共 $2a_{h-1}-1$ 个关键字。
3. 将给定 n 代入两端不等式，再取整数高度。m=5、n=17 时得到 2≤h≤3，不能仅凭下界结点数就断言 h=3。

补充原理与条件

同一高度下，填满得到最大关键字数，稀疏得到最小关键字数。由两端反解 h，并按整数高度取整。空的失败结点不计入 h；空树另行约定。

02 例子

m=5、h=3：最少 $2\times3^2-1=17$ 个关键字，最多 $5^3-1=124$ 个。n=17 时只能 h=2 或 3；不是唯一高度。

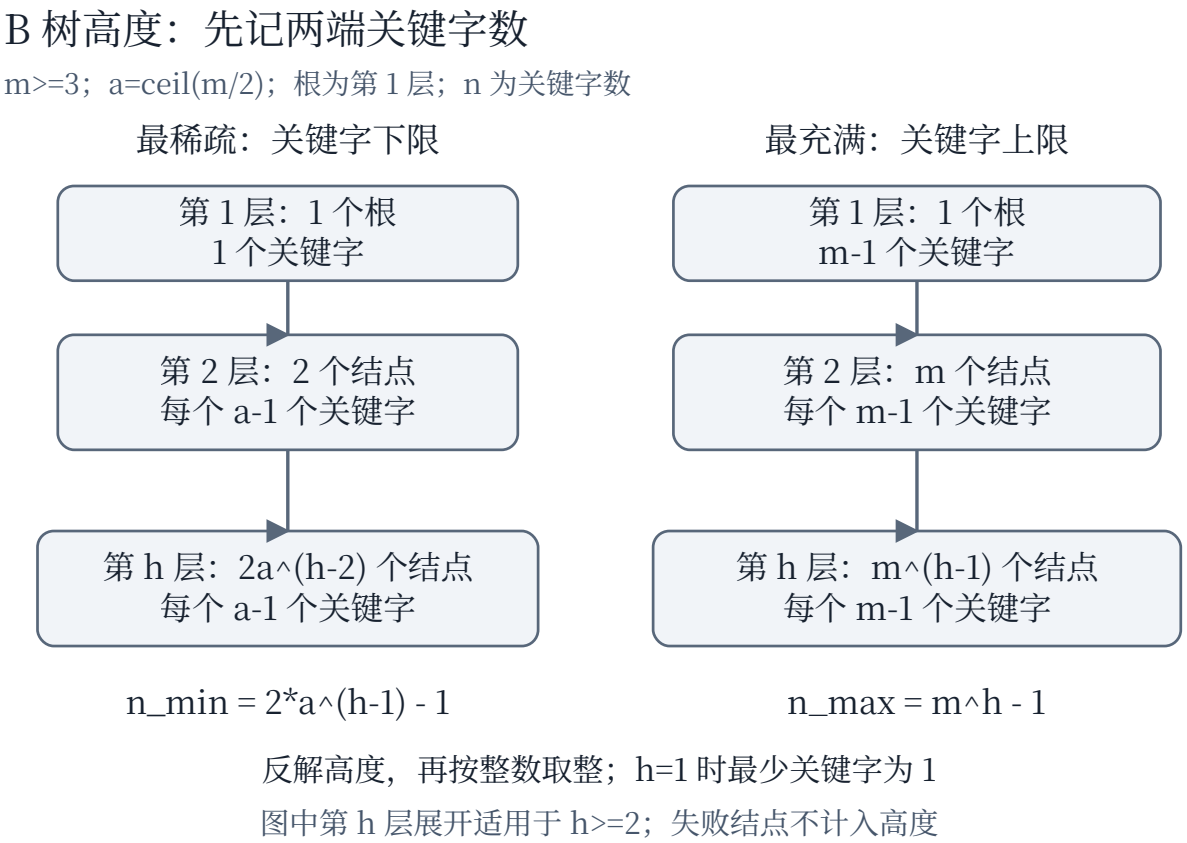
03 易错点

错误说法高度公式中的 n 是结点数；m 是最少孩子数。

04 图表

要点对照

记忆项	公式（上述约定）
最少关键字数	$n_{\min}(h)=2a^{h-1}-1$
最多关键字数	$n_{\max}(h)=m^{h}-1$
整数高度下界	$h\geqslant\lceil\log_{m}(n+1)\rceil$
整数高度上界	$h\leqslant\lfloor 1+\log_{a}\frac{n+1}{2}\rfloor$



单独查看图片

变式

自编模拟变式 · 关键字数量是否唯一确定高度

m=5 的非空 B 树有 17 个关键字，根从第 1 层计。能否直接断定高度为 3?

答案与解析

不能。高度 2 最多 24 个关键字，高度 3 最少 17 个，所以 2、3 都满足数量界，仍需结构信息。

关键区别：给出上下界与求出唯一值是两种任务；取整后仍可能有多个整数高度。

参考资料

参考：高校 B 树讲义：定义与高度推导

自检：用 n=17、m=5 能直接确定 h=3 吗?

不能；下界为 2，上界为 3，必须结合实际结构。

B 树根的例外：孩子数与关键字数 根有特例；终端数据叶没有实际孩子，但仍有关键字下限。 记忆概念 易错 易混 要点

核心结论

非根结点关键字数在 a−1～m−1；非叶根至少 2 个孩子、至少 1 个关键字。叶结点没有实际孩子，但仍受关键字数约束。

别混淆根有特例；终端数据叶没有实际孩子，但仍有关键字下限。

详解

01 解析

内部结点有 k 个关键字就有 k+1 个实际孩子。仅含根的非空树有 1～m−1 个关键字；不能把“非根内部孩子下限”机械套给根或数据叶结点。

02 例子

5 阶 B 树的非根结点至少 2 个关键字；非叶根可以只有 1 个关键字、2 个孩子。

03 易错点

错误说法根与所有非根结点都至少有 $\lceil m/2 \rceil$ 个实际孩子。

参考资料

参考：高校 B 树讲义：定义与高度推导

自检：“叶结点有 3 个孩子”在这个图示约定下正确吗?

不正确；这里叶是含关键字的终端结点，不含失败结点。

二叉树的叶数关系：$n_0=n_2+1$ $n_0=n_2+1$ 适用于非空二叉树，不要求满或完全。 记忆概念 易错 易混 要点

核心结论

任何非空二叉树满足 $n_0=n_2+1$；n_1 不影响这个等式。

别混淆$n_0=n_2+1$ 适用于非空二叉树，不要求满或完全。

详解

01 解析

$n_0 = n_2 + 1$

任意非空二叉树；n_0、n_2 分别为度为 0、2 的结点数。

$n=n_0+n_1+n_2$；边数 $n-1=n_1+2n_2$，联立得到结论。适用于普通二叉树，不要求满或完全；一般多叉树另有关系。

02 例子

只有一条长度 4 的单链：$n_0=1$、$n_2=0$，仍满足。

03 易错点

错误说法所有树都满足叶结点数等于度为 2 的结点数加 1。

自检：二叉树有 7 个度为 2 的结点，叶结点数是多少?

8 个。

完全二叉树：高度与编号公式 1 起始编号：末非叶 ⌊n/2⌋，叶从下一号开始。 记忆概念 易错 易混 要点

核心结论

$n \geqslant 1$、根为第 1 层：高度 $h=\lfloor \log_2 n \rfloor+1$；用 1 起始编号，父结点 $\lfloor i/2 \rfloor$，孩子编号为 2i、2i+1（须≤n）。

别混淆1 起始编号：末非叶 $\lfloor n/2 \rfloor$，叶从下一号开始。

详解

01 解析

$h=\lfloor \log_2 n \rfloor+1$

完全二叉树 $n \geqslant 1$，根为第 1 层；不能不加条件推广到任意二叉树。

完全二叉树最后一层可不满但从左连续排列；最后一个非叶结点编号 $\lfloor n/2 \rfloor$，叶从 $\lfloor n/2 \rfloor$+1 开始。0 起始数组需改为 2i+1、2i+2。

02 例子

n=6：高度 3，叶编号为 4、5、6；结点 3 只有左孩子 6。

03 易错点

错误说法完全二叉树必须每层都满，叶子编号总从 n/2 开始。

自检：n=6 的完全二叉树允许结点 3 仅有右孩子吗?

不允许，应有左孩子 6，不应有右孩子 7。

红黑树：最长路径与黑高的约定　红不相邻、各路黑数相同；黑高先统一 NIL 计数约定。　记忆概念　易错　易混　　要点

核心结论

红黑树根与 NIL 叶为黑；红结点的孩子为黑；任一结点到其后代 NIL 的各路黑结点数相等。到 NIL 的最长根叶路径不超过最短路径的两倍。

别混淆红不相邻、各路黑数相同；黑高先统一 NIL 计数约定。

详解

01 解析

相邻红结点被禁止，所以红结点数量受黑结点间隔限制。黑高是否计入当前根和 NIL 需按题设，不能直接把不同教材的数字相加。

02 例子

同一条路径可穿插红结点，但不能出现红→红。

03 易错点

错误说法红结点的两个孩子可以有一个红结点；高度等于黑高。

自检：黑高公式能否不说明是否计 NIL 就直接套用?

不能，先统一黑高、实际高度和外部叶的计数约定。

哈夫曼 WPL：叶权与内部合并权之和　只按叶权算 WPL；根深度为 0，不是层号 1。　　记忆概念　易错　易混　　要点

核心结论

$\mathrm{WPL}=\sum_{i=1}^{n} w_i l_i$；也等于构造过程中每次合并产生的内部结点权值之和。
别混淆只按叶权算 WPL；根深度为 0，不是层号 1。

详解

01 解析

$\mathrm{WPL}=\sum_{i=1}^{n} w_i l_i$
w_i 为叶权，l_i 为根到该叶的边数，根深度为 0。

每次合并使对应子树中的叶子深度增加 1，所以它们的权之和贡献一次。根深度取 0，不能按根为第 1 层的层号直接相乘。

02 例子

叶权 1、2、3：先合并 1+2=3，再合并 3+3=6，WPL=3+6=9。

03 易错点

错误说法WPL 是所有结点的权值与深度乘积之和。

自检：只有一个叶结点时，按根深度为 0 的树定义 WPL 是多少?

0；单字符编码人为设 1 位是另一种编码约定。

遍历序列恢复：中序划分与唯一性　唯一恢复通常要求标签互异；先序加后序一般不够。　记忆概念　易错　易混　　要点

核心结论

关键字互异时，先序+中序或后序+中序可唯一恢复二叉树；先序+后序一般不能。

别混淆唯一恢复通常要求标签互异；先序加后序一般不够。

详解

01 解析

1. 先确认标签互异；先序首项或后序末项是根，再在中序序列中找根来划分左右子树。
2. 对子区间重复同样划分；某侧中序为空就没有该侧子树。
3. 只有先序和后序时，单孩子在左或右都可能产生相同序列，不能保证唯一恢复；用两个结点即可构造反例。

补充原理与条件

先序首元素/后序末元素确定根，中序根位置分左右子树。重复标签可能破坏唯一性；只有一个孩子时先后序不能分辨它在左还是右。

02 例子

两个结点 a→b，b 为左孩子或右孩子，先序都是 ab，后序都是 ba。

03 易错点

错误说法先序加后序能唯一确定任何二叉树。

自检：关键字互异时，中序加层序能否用于确定结构?

可以逐步选出各子树层序最先出现的根；仅中序本身不够。

树与森林转换：左孩子右兄弟　转换后的右指针是兄弟，不是原树的孩子。　　记忆概念　易错　易混　　要点

核心结论

左指针指原树的第一个孩子，右指针指原树的下一个兄弟；森林各根也通过右兄弟链连接。

别混淆转换后的右指针是兄弟，不是原树的孩子。

详解

01 解析

转换保留的是孩子和兄弟关系，二叉树中的度和原树的度不同。原树的先根遍历对应转换后二叉树的先序；原树后根对应中序（按常见约定）。

02 例子

原树根有 3 个孩子，可表示成首孩子和连续右兄弟链，不要求二叉根有 3 个指针。

03 易错点

错误说法树转成二叉树后，二叉树的右孩子仍是原树的孩子。

自检：原树叶结点在转换后能有右孩子吗?

能，右孩子可能是它的兄弟，因此原树叶和二叉树叶不能等同。

AVL 调整：按失衡结点到插入位置的方向 LL 右旋、RR 左旋；LR/RL 各做两次旋转。

记忆概念 易错 易混 要点

核心结论

LL 用右旋，RR 用左旋，LR 先左旋子树再右旋，RL 先右旋子树再左旋。

别混淆LL 右旋、RR 左旋；LR/RL 各做两次旋转。

详解

01 解析

1. 从插入点往上找最低的失衡祖先，以该祖先到插入点的前两段方向命名 LL/LR/RL/RR。
2. LL 对祖先右旋，RR 左旋；LR 先对子结点左旋再对祖先右旋，RL 反之。
3. 旋转后用中序序列复核 BST 次序没有改变，再更新高度；名称是路径方向，不是让你照名字朝同一方向旋转。

补充原理与条件

名称描述从最低失衡祖先出发经过的两段方向，不是最终旋转方向。旋转只改局部指针，需保持中序顺序；删除可能需要向上继续调整。

02 例子

依次插入 3、2、1，根 3 为 LL 失衡，对 3 右旋使 2 成为局部根。

03 易错点

错误说法LL 型意味着向左旋，RR 型意味着向右旋。

自检：LR 是只做一次右旋吗?

不是，先对左子树左旋，再对失衡根右旋。

并查集：父指针、路径压缩与按秩合并　集合代表不是图中父边；复杂度要区分单次与摊还。　记忆概念　易错　易混　要点

核心结论

并查集维护不相交集合，find 返回代表，union 合并集合；按秩/按大小控制树高，路径压缩缩短 find 路径。

别混淆集合代表不是图中父边；复杂度要区分单次与摊还。

详解

01 解析

根表示集合代表，不一定是某种数值最小顶点。优化后的摊还复杂度近似常数，但不能把任意一次未压缩查找都宣称严格 $O(1)$。

02 例子

Kruskal 中边两端已属同一集合，加入该边会形成环，可跳过。

03 易错点

错误说法并查集的父指针表示图中实际的一条边；按秩等于总按编号。

自检：find 返回哪个对象?

该元素当前所在集合的代表结点。

一般树的结点度与边数：m 叉树不等于 B 树　一般 m 叉树不保证半满或叶同层，不能套 B 树高度界。　记忆概念　易错　易混　要点

核心结论

树 n 个结点有 n−1 条边；结点孩子数之和为 n−1。一般 m 叉树只限制最多 m 个孩子，不保证半满与叶层一致。

别混淆一般 m 叉树不保证半满或叶同层，不能套 B 树高度界。

详解

01 解析

若所有非叶恰为 m 个孩子，则 n=mI+1、叶数 L=(m−1)I+1，I 为非叶数；无此条件不能套。B 树关键字数与树结点数仍是不同量。

02 例子

一棵普通三叉树可以是一条长链，因此高度可为 n，不能套 B 树的对数高度界。

03 易错点

错误说法任意 m 叉树每个非叶结点都恰有 m 个孩子，也满足 B 树高度界。

自检：一般树各结点的“度”和无向图度是否同义?

不是，树结点度常指孩子数，无向图度计关联边。

图

BFS 最短路：仅适用于无权或等权　BFS 求最少边数，不适用于一般非等权最短路。　　易混　易错　近年考查　　要点

核心结论

BFS 保证边数最少；边权相同且非负时，也能给出权值最短路。一般非负带权图用 Dijkstra。

别混淆BFS 求最少边数，不适用于一般非等权最短路。

详解

01 解析

1. BFS 按经过边数从少到多扩展，并不累加或比较边权。
2. 无权图求的就是边数；所有边权相同且非负时，边数少也意味着总权值不大。
3. 只要边权不同，先构造“一条重边与两条轻边”的对照；若两条轻边总权更小，BFS 的最短路结论就不能使用。

补充原理与条件

BFS 按层扩展而不比较累计权值。标记通常应在入队时完成，防止同一顶点重复入队。

02 例子

一条直接边权 10，两条间接边各权 1：BFS 边数最少路径的权值却为 10。

03 易错点

错误说法BFS 能求任意带权图最短路。

自检：无权图 BFS 什么时候置 visited?

首次发现并入队时。

Dijkstra 与 Floyd：负权限制不同　Dijkstra 用非负权；Floyd 可有负边，但须处理负环。　　易混　　要点

核心结论

Dijkstra 要求非负边权；Floyd 可处理负边，但不能直接以存在负权回路的结果作为有限最短路。

别混淆Dijkstra 用非负权；Floyd 可有负边，但须处理负环。

详解

01 解析

1. Dijkstra 将当前最小暂定距离定为最终值，依赖后续边不会把距离再降小。
2. 负边可能从尚未确定的顶点返回并改小已确定距离，破坏上述依据；不能只看起点附近有没有负边。
3. Floyd 按中间顶点集合逐步扩展，可处理负边；若有可达负环，相关点对不存在有限最短路，不能照抄矩阵值作为答案。

补充原理与条件

Dijkstra 的已确定最短距离可能被后来负边推翻。Floyd 的 k 必须作为最外层，表示允许的中间顶点集合。

02 例子

含负边且无负环的图可以使用 Floyd；不能只因负边就判它失效。

03 易错点

错误说法两种算法都不能有负边。

自检：Floyd 的三层循环，哪一层必须最外?

中间顶点 k。

最小生成树与最短路径树　最小总边权，不等于源点到各点距离最短。　　易混　　要点

核心结论

MST 最小化整棵树边权总和；最短路径树保证给定源到各点的路径最短。

别混淆最小总边权，不等于源点到各点距离最短。

详解

01 解析

1. 先看题目最小化什么：全树边权和，还是某个源到各点的路径长度。
2. MST 要连通所有顶点且总边权最小；它不为每个源点单独优化路径。
3. 检验某树是否最短路径树，逐点比较树上源点路径和原图最短距离；仅有总边权较小不足以证明。

补充原理与条件

两种优化目标不同。Prim/Kruskal 用于无向带权图；非连通图得到最小生成森林。负权不会破坏 MST 的适用性。

02 例子

三角形边 AB=2、BC=2、AC=3：MST 可选 AB、BC，但 A 到 C 路径为 4，比直接边 3 长。

03 易错点

错误说法最小生成树保证根到每点距离最短。

自检：MST 能否包含负权边?

可以。

拓扑排序与关键路径：删点数和工期　关键路径可多条；缩短一条未必缩短工期，延长关键活动会延长工期。　　冷门　易混　近年考查　　要点

核心结论

DAG 可有多种拓扑序；删去顶点数不足总数说明有环。AOE 网中 e=l 的活动为关键活动。

别混淆关键路径可多条；缩短一条未必缩短工期，延长关键活动会延长工期。

详解

01 解析

在通常 AOE 工期模型中，关键路径可有多条。其他活动持续时间与依赖关系不变时，延长任一关键活动会延长总工期；缩短一个关键活动则未必缩短工期，因为其他关键路径可能仍决定工期。

02 例子

两个独立顶点可以任意先后，拓扑序不唯一，但图无环。

03 易错点

错误说法拓扑序唯一才说明没有环；关键活动只有一条链。

参考资料

参考：Princeton Algorithms：DAG 最长路径与关键路径法

自检：多条关键路径时，缩短某一条上的一个活动一定缩短工期吗?

不一定，其他关键路径可能仍决定工期。

图的度数与边数：有向和无向分开　无向度数和 2E；有向入度和、出度和各为 E。　　记忆概念　易错　易混　　要点

核心结论

无向图度数和=2|E|；有向图入度和=出度和=|E|。树有 n 个顶点时边数为 n−1。

别混淆无向度数和 2E；有向入度和、出度和各为 E。

详解

01 解析

$\sum_{v} \deg(v)=2|E|$

无向图度数和；有向图入度和、出度和各为边数。

一条无向边贡献两个端点的度；一条有向边贡献起点出度与终点入度各一次。允许自环时无向自环计度 2，简单图通常不含自环。

02 例子

有向图有 6 条边，所有顶点出度之和为 6，不是 12。

03 易错点

错误说法有向图所有顶点的出度和等于 2|E|。

自检：连通无向简单图有 n−1 条边，一定是树吗?

是；只给边数 n−1 而不保证连通则不能这样判断。

图的存储：矩阵、邻接表、十字链表、多重表　十字链表用于有向图，邻接多重表用于无向图。　　记忆概念　易错　易混　　要点

核心结论

邻接矩阵占 $O(V^2)$，邻接表占 $O(V+E)$；无向邻接表常每边存两次。十字链表用于有向图，邻接多重表用于无向图。

别混淆十字链表用于有向图，邻接多重表用于无向图。

详解

01 解析

无向邻接矩阵通常对称；有向邻接表可方便找出边，入边可用逆邻接表；十字链表把入/出链接汇于同一边结点。

02 例子

有向矩阵按 A[i][j] 表示 i→j，则第 i 行关联出边、第 i 列关联入边。

03 易错点

错误说法邻接表每个无向边只出现一次；十字链表用于无向图。

自检：矩阵中 0 总表示没有边吗?

不总是；允许权 0 时需另用无边标记，通常还要处理对角线。

DFS/BFS：遍历所有分量与复杂度　非连通图需外层扫未访问点；BFS 入队即标记。　记忆　概念　易错　易混　要点

核心结论

非连通图需外层循环从未访问顶点继续。邻接表遍历 $O(V+E)$，矩阵通常 $O(V^2)$。

别混淆非连通图需外层扫未访问点；BFS 入队即标记。

详解

01 解析

1. 从一个起点完成一次 DFS/BFS 后，检查是否还有未访问顶点；有就继续作为新起点，覆盖不连通分量。
2. 访问数组在整次遍历开始时初始化，不能每换一个起点就清空，否则已访问分量又会被重复遍历。
3. 邻接表扫描所有顶点和边为 $O(V+E)$；邻接矩阵每个顶点都扫一行，为 $O(V^2)$。

补充原理与条件

DFS 栈/递归与 BFS 队列的最大空间通常 $O(V)$。访问标记不能在每个新起点前清空，否则可能重复；BFS 通常在入队时标记。

02 例子

一个图有两个互不相连的三角形，从第一个三角形出发无法到达另一分量。

03 易错点

错误说法从一个起点 DFS 后，所有图的顶点都已访问。

自检：DFS 是否一定只能递归实现?

不一定，可用显式栈实现，但要维护正确的访问与推进顺序。

连通、强连通与环判定　有向图 visited 不足以判环，还要看是否在递归路径上。　记忆概念　易错　易混　要点

核心结论

无向连通要求任意两点有路径；有向强连通要求互相可达。无向 DFS 应排除父边；有向环常看指向递归栈中顶点的边。

别混淆有向图 visited 不足以判环，还要看是否在递归路径上。

详解

01 解析

有向图已访问的顶点可能已完成，单纯 visited 无法区分回边。无向图需考虑边模型，含重边时不能仅按顶点父亲排除所有同向连接。

02 例子

DAG 的两个分支汇合到同一已访问点，并不意味着存在环。

03 易错点

错误说法DFS 遇到已访问顶点就说明所有图有环；单向可达就是强连通。

自检：有向图中 A 可达 B 而 B 不可达 A，能否认定强连通?

不能。

Prim/Kruskal：选边规则与复杂度　Prim 选树内外跨边；Kruskal 选不成环的全局轻边。　记忆概念　易错　易混　要点

核心结论

Prim 在当前树与外部之间选最轻跨边；Kruskal 按全局边权排序，加入不成环的边。

别混淆Prim 选树内外跨边；Kruskal 选不成环的全局轻边。

详解

01 解析

朴素矩阵 Prim 为 $O(V^2)$，邻接表+堆常为 $O(E\log V)$；Kruskal 主要排序 $O(E\log E)$。边权重复可能产生多个同权 MST，不代表不存在解。

02 例子

Prim 不能选两端都在树外的全局最轻边来扩展当前树。

03 易错点

错误说法Prim 每轮选全图最小未选边，Kruskal 每轮选最近顶点。

自检：Kruskal 的不成环判定常用什么结构?

并查集。

DFS 三色状态：未发现、已发现未完成、已完成　灰色未完成，黑色已完成；有向回边可证有环。　记忆概念　易混　图解　要点

核心结论

常用 DFS 三色标记：白色未发现，灰色已发现但搜索未完成，黑色搜索完成；这是常用算法约定，并非所有实现都必须使用颜色。

别混淆灰色未完成，黑色已完成；有向回边可证有环。

详解

01 解析

有向图 DFS 中，指向灰色祖先的回边可说明有环；指向黑色点不必构成环。无向图要排除树边反向返回父结点的情况，简单图与多重图处理亦有差别。

02 例子

有向边 A→B、A→C、B→C，访问到黑色 C 不代表存在环。

03 易错点

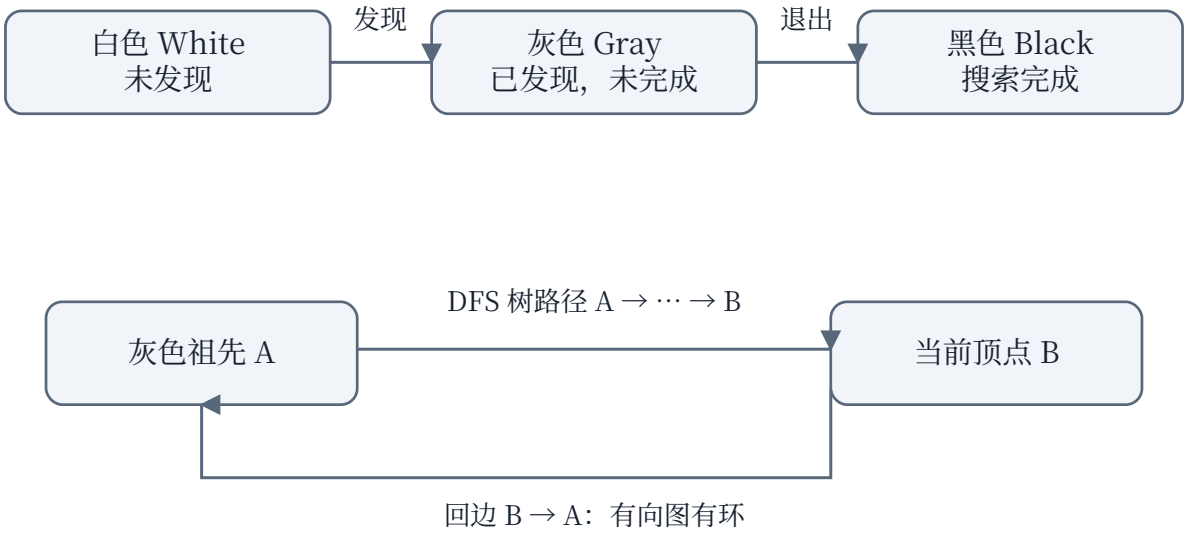
错误说法灰色顶点就是已经完成搜索；有向图见到任意已访问顶点都是环。

04 图表

要点对照

名称	含义	典型时刻
白色 White	未发现	开始前
灰色 Gray	已发现，未完成	进入递归/搜索栈后
黑色 Black	已完成	处理完全部相邻边并退出时

DFS 三色：灰色还在搜索路径上



指向黑色顶点不必构成环；无向图要排除返回父结点的树边。

白色被发现后变灰，完成所有相邻边的搜索后变黑。有向图中，从当前顶点指回灰色祖先形成回边，可证明有环；指向黑色顶点并不必然有环。单独查看图片

自检：DFS 何时把灰色顶点标黑?

完成该顶点所有相邻边/后继的搜索后。

查找与排序

快排：终止、枢轴与递归区间　先确认 p 是枢轴最终位置，还是仅为分界。　　个人错题　易错　近年考查　　要点

核心结论

若划分返回已归位的下标 p，递归 [low,p−1] 与 [p+1,high]，并以 low>=high 终止。

别混淆先确认 p 是枢轴最终位置，还是仅为分界。

详解

01 解析

划分方案有不同返回约定，Hoare 分割点不一定是枢轴最终位置，不能混套区间。必须保存枢轴值并保证扫描前进。

02 例子

长度为 1 或全相等数组是必须检查的边界；不缩小区间可能无限递归。

03 易错点

错误说法划分后还递归处理包含已归位枢轴的同一区间。

代码对照

```
int partition(int a[], int low, int high) {
    int pivot = a[low];
    while (low < high) {
        while (low < high && a[high] >= pivot) --high;
        a[low] = a[high];
        while (low < high && a[low] <= pivot) ++low;
        a[high] = a[low];
    }
    a[low] = pivot;
    return low;
}
void quickSort(int a[], int low, int high) {
    if (low >= high) return;
    int p = partition(a, low, high);
    quickSort(a, low, p - 1);
    quickSort(a, p + 1, high);
}
/* 调用 quickSort(a, 0, n-1)。
 此首元素枢轴实现平均 O(n log n)，最坏 O(n²);
 递归栈平均 O(log n)，最坏 O(n)。 */
```

自检：已归位枢轴下标能再次留在同侧递归区间中吗?

这种约定下不应保留；两侧排除 p。

排序稳定性与算法实现条件　稳定看相等键的相对次序；归并相等先取左。　易错　易混　近年考查　记忆概念　要点

核心结论

稳定表示相等关键字的记录保持原相对次序；归并相等时应先取左段，直接插入不跨越相等记录。

别混淆稳定看相等键的相对次序；归并相等先取左。

详解

01 解析

1. 把相同关键字附上不同身份，例如 5a、5b，而不只写数字 5。
2. 跟踪比较、交换和插入过程：相等记录若可越过彼此，算法就可能不稳定。
3. 归并相等时先取左段才能保留先后；插入排序只移动严格大于待插记录的元素，避免跨过相等项。

补充原理与条件

常见稳定：直接插入、冒泡、归并、LSD 基数；常见不稳定：希尔、简单选择、堆、快排。稳定性受实现影响。

02 例子

两条记录键都为 5，分别附带 a、b；排序后 a 仍在 b 前才保持该对稳定。

03 易错点

错误说法稳定表示运行时间稳定，或所有归并写法都稳定。

自检：归并两段时，相等键先取右段会怎样?

可能破坏稳定性。

建堆与逐个插入：复杂度不同　自底向上建堆 O(n)，不等于逐个插入建堆。　易混　近年考查　要点

核心结论

自底向上建堆总时间 $O(n)$；逐个插入建堆最坏 $O(n\log n)$；堆排序整体 $O(n\log n)$。

别混淆自底向上建堆 $O(n)$，不等于逐个插入建堆。

详解

01 解析

$\sum_{j\geq 1} \frac{j}{2^j}=O(n)$

按高度统计下沉工作量，用于说明自底向上建堆为线性时间。

1. 自底向上建堆从最后一个非叶结点调整；高度越大的结点越少。
2. 总工作量按各层“结点数×可下沉高度”相加，得到线性级，而不是每个结点都乘根高。
3. 逐个插入则每次可能上浮 $O(\log n)$，最坏总计 $O(n\log n)$；题目给的建堆方法决定用哪个结果。

补充原理与条件

自底向上建堆中，多数结点接近叶子，调整高度很小。不能把所有结点都按根的高度估计为紧确界。

02 例子

堆只保证父子大小关系，中序遍历并不有序。

03 易错点

错误说法建堆每个结点都 $O(\log n)$，所以必为 $O(n\log n)$。

自检：大根堆是否保证左孩子大于右孩子?

不保证。

折半查找与分块查找：存储要求 块间有序即可；链表不具备 O(1) 随机定位。 冷门 易混 近年考查 要点

核心结论

数组支持 $O(1)$ 随机访问；链表定位中点需遍历。分块查找只要求块间有序、块内可无序。

别混淆块间有序即可；链表不具备 $O(1)$ 随机定位。

详解

01 解析

折半减少比较次数不等于减少定位时间。分块索引可折半，但块内通常顺序查找。

02 例子

链表不断找中点，不能直接宣称总时间 $O(\log n)$。

03 易错点

错误说法只要元素有序，链表折半查找也有 $O(\log n)$ 时间。

自检：分块查找要求每一块内部有序吗?

不要求，必须保证块间有序。

KMP：next 与 nextval 的下标约定　先固定下标与失配规则，不能混用 next 版本。　冷门　易错　近年考查　要点

核心结论

next/nextval 的值取决于从 0/1 开始及失配时更新约定；KMP 通常保持主串指针前进，只调整模式位置。

别混淆先固定下标与失配规则，不能混用 next 版本。

详解

01 解析

nextval 在回退后字符仍相等时继续跳转，减少重复比较；不能把最长相等前后缀长度直接当所有版本的 next 下标。

02 例子

先写清 i、j 的初值和 j=next[j] 的含义，再推数组；避免差 1。

03 易错点

错误说法不同教材的 next 数组可以不转换直接混用；失配时主串指针回退。

自检：KMP 总时间复杂度是多少?

预处理加匹配为 $O(n+m)$，n 为主串长、m 为模式长。

外部排序：归并段、败者树与 I/O　内部败者位置与额外冠军位置要分清。　冷门　易混　近年考查　要点

核心结论

升序多路归并选当前最小记录；败者树内部保存比较败者，专门的冠军位置通常保存获胜归并段编号。

别混淆内部败者位置与额外冠军位置要分清。

详解

01 解析

结构图的根与额外冠军结点须按题设区分。提高归并路数减少归并趟数和外存 I/O，但比较及缓冲区开销也要考虑。

02 例子

置换选择生成的初始归并段可长于工作区容量，不应按固定块数机械分段。

03 易错点

错误说法败者树根结点总保存当前最大记录；增大归并路数只减少比较次数。

自检：k 路归并一趟是处理 k 个记录还是 k 个有序段?

k 个有序归并段，每个段可以包含很多记录。

开放定址删除：空槽与删除标记　删除标记继续查，从未使用的空槽才可终止。　　记忆概念　易错　易混　　要点

核心结论

开放定址删除通常保留删除标记，查找遇删除标记继续探测；遇从未使用的空槽才可在对应规则下判失败。

别混淆删除标记继续查，从未使用的空槽才可终止。

详解

01 解析

冲突记录可能被探测到后面的槽，清成空槽会截断探测链。插入可利用删除槽，但必须保证查重和停止规则正确；链地址法删除是另一机制。

02 例子

关键字 A、B 冲突，B 存在 A 后面；删 A 后若查找立即停止，可能漏掉 B。

03 易错点

错误说法删除散列表元素后立即把槽设成从未使用的空槽。

自检：删除标记表示查找应停止吗?

不表示，应继续探测。

BST 删除：用前驱/后继替代，再删原位置　前驱是左子树最大，不一定是直接左孩子。　记忆概念　易错　易混　　要点

核心结论

有两子树时，常用中序前驱或后继替代键，再在对应原位置删除该结点；需保留 BST 有序性和链接。

别混淆前驱是左子树最大，不一定是直接左孩子。

详解

01 解析

前驱是左子树最大值，后继是右子树最小值，它们在原位置至多有一个孩子，便于链接修复。键值与附带记录应一起处理。

02 例子

用右子树最小结点替代后，其原父结点应接上该结点可能存在的右孩子。

03 易错点

错误说法删掉有两孩子的结点，直接用左孩子顶上且丢掉右子树。

自检：前驱等同于直接左孩子吗?

不等同，应沿左子树最右路径寻找。

查找 ASL：成功与失败，概率与比较次数　ASL 是按概率加权的比较次数，不是最长路径。　　记忆概念　易错　易混　　要点

核心结论

ASL=Σ(查找概率×比较次数)。成功按实际元素计，失败按间隙或不可达范围计，需看判定树与概率。

别混淆ASL 是按概率加权的比较次数，不是最长路径。

详解

01 解析

$$ASL=\sum_{i} p_i c_i$$

p_i 为第 i 个查找结果的概率，c_i 为该结果的比较次数；成功与失败分别计算。

1. 先区分成功与失败 ASL，列出各结果的发生概率及对应比较次数。
2. 逐项计算 $\sum_{i} p_i c_i$；只有题目说明等概率时，才可把比较次数直接算术平均。
3. 无序顺序查找允许调整排列且仅最小化成功 ASL 时，将高概率记录前移可减少加权比较次数；有序查找须同时保留有序约束。

补充原理与条件

有序顺序查找遇更大的元素可提前失败；折半查找的失败叶/间隙也有不同路径长度。不能未说明均匀概率就直接平均。

02 例子

某元素概率高而位置靠后，会显著提高顺序查找 ASL。

03 易错点

错误说法ASL 是最长查找次数，成功和失败有相同平均值。

变式

自编模拟变式 · 不等概率时调整查找次序

无序表只有 A、B 两项，成功查找概率分别为 0.2、0.8，顺序比较且允许重排。谁应放前面?

答案与解析

B 在前：ASL=0.8×1+0.2×2=1.2；A 在前为 1.8。

关键区别：交换高低概率项，直接比较加权代价；但有序表仍必须遵守其顺序约束。

自检：ASL 可以只取最大比较次数吗?

不能，那是最坏情况指标，平均应加权。

散列冲突：装填因子与探测序列 后续冲突可来自非同义词；二次探测不必覆盖全表。 记忆概念 易错 易混 要点

核心结论

同义词初始散列地址相同；开放定址中的后续冲突也可能发生在非同义词间。α=元素数/散列表槽数反映装填程度。

别混淆后续冲突可来自非同义词；二次探测不必覆盖全表。

详解

01 解析

性能还依赖散列函数与冲突处理。线性探测可在未满时按完整步进找到空位，但可能聚集；二次探测能否覆盖全部槽取决于表长和探测规则。

02 例子

一个被冲突挤到槽 4 的记录，可以与初始散列地址为 4 的另一记录发生冲突。

03 易错点

错误说法散列地址不同就永远不会冲突；二次探测必遍历所有槽。

自检：开放定址法和链地址法的查找过程相同吗?

不同，前者沿探测序列，后者在桶链中查找。

内排序总表：稳定性、时间与辅助空间 稳定性、时间、辅助空间须分别判断。 记忆概念 易错 易混 要点

核心结论

按具体算法和实现判断，时间、辅助空间和稳定性是三类指标，不能互相推出。

别混淆稳定性、时间、辅助空间须分别判断。

详解

01 解析

直接插入/冒泡对已排序输入可为 $O(n)$（冒泡须设置早停）；简单选择通常始终 $\Theta(n^2)$。递归栈算空间；LSD 基数的每趟分配/收集需稳定。

02 例子

比较排序最坏时间下界 $\Omega(n\log n)$ 不适用于利用键位数的基数排序。

03 易错点

错误说法$O(n\log n)$ 的排序都稳定；归并和快排的空间都 $O(1)$。

04 图表

要点对照

算法	平均 / 最坏时间	典型辅助空间	通常稳定
直接插入/冒泡	$O(n^2)$ / $O(n^2)$	$O(1)$	是
简单选择	$O(n^2)$ / $O(n^2)$	$O(1)$	否
希尔	依增量序列而变	$O(1)$	否
快排	$O(n\log n)$ / $O(n^2)$	栈平均 $O(\log n)$，最坏 $O(n)$	否
堆排序	$O(n\log n)$ / $O(n\log n)$	$O(1)$	否
二路归并	$O(n\log n)$ / $O(n\log n)$	$O(n)$	相等先取左时是
LSD 基数	O(d(n+r))	O(n+r)	每趟稳定时是

自检：简单选择排序最坏移动次数与比较次数都是 $O(n^2)$ 吗?

常见交换实现移动次数 $O(n)$，比较次数 $\Theta(n^2)$。

多路归并趟数与置换选择：内存工作区不是段长上限　工作区容量不是置换选择初始段长度的上限。　　记忆概念　易错　易混　　要点

核心结论

置换选择用“当前段/下一段”标记和最小记录选择，初始归并段可长于工作区；k 路归并将若干有序段逐趟合并。

别混淆工作区容量不是置换选择初始段长度的上限。

详解

01 解析

$p=\lceil \log_k r \rceil$

r≥1 为初始归并段数，k>1 为归并路数；初始段生成阶段另计。

若初始 r 段且每趟最多合并 k 段，k≥2 时归并趟数通常为 $\lceil \log_k r \rceil$。I/O 次数还看初始生成与末次输出是否计入，不能用趟数代替总读写量。

02 例子

输入近乎升序可产生很长初始段，逆序时可能较短；段长取决于输入。

03 易错点

错误说法工作区能放 w 个记录，就只能生成长度 w 的初始归并段。

自检：败者树改变归并路数就一定不需要更多输入缓冲吗?

不，应为各路及输出安排相应缓冲。

快排阶段与边界：划分、递归、终止　一趟划分只建立分区关系，两侧内部未必有序。　　记忆概念　易混　图解　　要点

核心结论

经典快排包含划分 Partition、递归处理子区间、规模足够小时终止；划分通常只建立左右分区的大小关系。

别混淆一趟划分只建立分区关系，两侧内部未必有序。

详解

01 解析

返回枢轴最终位置的划分法常递归 [low,p−1] 和 [p+1,high]；Hoare 等返回分界的实现可能不同。划分返回值的语义决定递归边界，不能只背函数名称。

02 例子

一趟后枢轴左侧都不大于它，左侧元素之间仍可能无序。

03 易错点

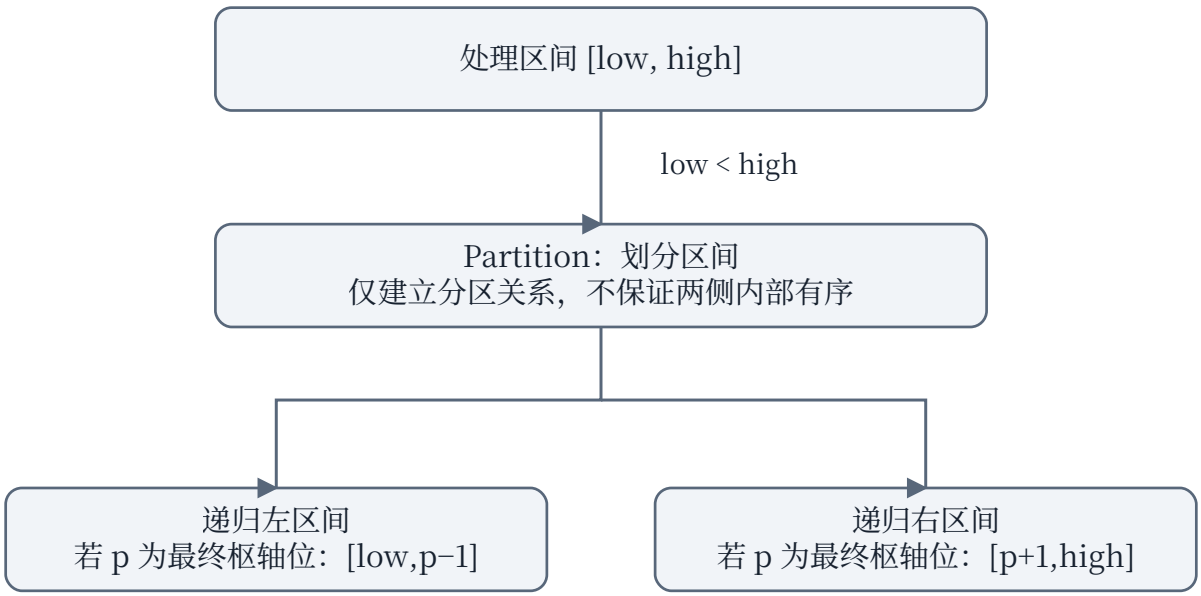
错误说法一趟快排结束后两边子区间内部已经排好序。

04 图表

要点对照

阶段	常见名称	必须检查
划分	Partition	返回枢轴位置还是分界
递归	Recursion	子区间严格缩小
终止	Base Case	空区间/单元素无需排序

快排：划分后递归，区间必须缩小



low ⩾ high：空区间或单元素，直接返回

递归边界由划分返回值的语义决定；Hoare 分界方案不能机械套用。

区间至少两个元素时先划分，再递归处理两侧子区间。图中边界仅用于返回枢轴最终位置 p 的划分方案；空区间和单元素直接返回。单独查看图片

参考资料

参考：Princeton Algorithms：Quicksort

自检：Partition 返回值是否总是枢轴最终下标?

不是，须检查采用的划分方案。

外部排序两阶段：初始归并段生成与多趟归并　先生成初始段，再算归并趟数；两阶段分别计。　　记忆概念　易混　图解　　要点

核心结论

外部归并排序主要分初始归并段生成、对归并段进行多趟多路归并。置换选择用于生成较长初始段；败者树常用于归并时选出最小输出记录。

别混淆先生成初始段，再算归并趟数；两阶段分别计。

详解

01 解析

$p=\lceil \log_k r \rceil$

仅计算归并趟数；初始段生成和实际读写量另按题设统计。

两阶段计数要区分；若初始段数为 r、每次 k 路且 k>1，通常还需 $\lceil \log_k r \rceil$ 趟归并。置换选择的约 2m 平均段长有输入随机性等条件，不是最大上限或保证。败者树是一种选择结构，也可用于置换选择实现。

02 例子

先得到 9 个初始段，再做 3 路归并，需 2 趟归并；初始生成阶段另计。

03 易错点

错误说法生成一个初始归并段就是完成一趟多路归并；败者树生成初始段。

04 图表

要点对照

阶段/结构	名称	任务
初始段生成	Run Generation	内排序或置换选择生成有序段
多趟归并	Multiway Merge	逐步合并有序段
置换选择	Replacement Selection	利用工作区延长初始段
败者树	Loser Tree	维护候选记录的选择过程

外部排序：先生成段，再逐趟归并



初始段生成另计；败者树可用于归并的候选记录选择，也可用于置换选择。

以九个初始有序段做三路归并为例，第一趟合并为三个段，第二趟合并为一个最终有序文件。初始归并段生成与后续两趟归并分开计数。单独查看图片

自检：置换选择与 k 路归并的主要区别?

前者组织输入生成初始有序段，后者把多个已有序段合为更大有序段。

408 · CO

计算机组成原理

56 条 · 8 个章节

冯·诺依曼与哈佛：存储与总线的组织　一级指令/数据 Cache 分离，不等于整机纯哈佛。　记忆概念　易错　易混　　要点

核心结论

冯·诺依曼思想把指令与数据存入存储器，按指令执行；哈佛结构将指令和数据存储/通路分开。

别混淆一级指令/数据 Cache 分离，不等于整机纯哈佛。

详解

01 解析

现代处理器可能主存统一而指令/数据 Cache 分开，不能只凭一级 Cache 分离就断言整个系统纯哈佛。软硬件共同决定执行行为。

02 例子

一条机器指令在存储器中也以二进制编码保存。

03 易错点

错误说法存储程序计算机只允许数据放在内存，指令不属于存储内容。

自检：指令与数据外观都是位串，如何区分用途?

由地址、程序执行阶段与体系结构的解释方式决定。

编译、汇编、链接与装入　编译、汇编、链接、装入分工不同，工具可合并步骤。　　记忆概念　易错　易混　　要点

核心结论

常见流程：预处理→编译→汇编→链接→装入；编译生成低层表示，汇编生成目标代码，链接处理符号与模块组合。

别混淆编译、汇编、链接、装入分工不同，工具可合并步骤。

详解

01 解析

1. 把源代码翻译成汇编属于编译，把汇编翻译为机器目标代码属于汇编。
2. 目标模块调用另一个模块的符号时，由链接解析外部引用并组织可执行文件。
3. 把可执行代码放进运行所需地址空间属于装入；遇到“找不到外部函数定义”，先检查链接而不是归为语法编译错误。

补充原理与条件

具体工具链可合并阶段，但职责仍需区分。目标文件未必已经解析全部外部符号；装入与执行时的动态链接也可另发生。

02 例子

调用库函数的符号引用可在链接阶段与库模块对应。

03 易错点

错误说法汇编器负责把高级语言直接翻译成最终已装入的运行程序。

自检：汇编语言与机器语言是一回事吗?

不是，汇编用助记符表达低层指令，机器语言是其可执行编码。

数据表示

机器字长、指令字长与总线宽度　字长、地址位数、指令长度、总线宽度不能互相代替。　个人错题　易混　记忆概念　　要点

核心结论

机器字长通常是 CPU 一次整数运算的基本位数，与通用寄存器和整数 ALU 宽度密切相关；其他宽度可不同。

别混淆字长、地址位数、指令长度、总线宽度不能互相代替。

详解

01 解析

1. 题目若问机器字长，优先看整数运算通路及通用寄存器一次处理的基本位数。
2. 指令字长表示一条指令编码有多少位；数据总线宽度表示一次可传多少位，均不能直接替代机器字长。
3. 求可编址容量时再看地址位数及编址单位：A 位地址按字节编址为 2^A B，按字编址为 2^A 个字。

补充原理与条件

变长指令可以跨多个字；总线可分多次传送一个字。寻址范围取决于地址位数与按字/按字节编址约定。

02 例子

32 位机器并不意味着每条指令均为 32 位，也不意味着所有总线都是 32 位。

03 易错点

错误说法机器字长一定等于地址位数、指令长度或数据总线宽度。

自检：机器字长能单独决定内存寻址容量吗?

不能，还需要地址位数和编址单位。

MSB、LSB 与大小端　大小端改变字节排列，不反转字节内的各位。　个人错题　易混　近年考查　记忆概念　要点

核心结论

MSB/LSB 表示位权；大小端描述多字节数据的字节地址顺序。

别混淆大小端改变字节排列，不反转字节内的各位。

详解

01 解析

1. 先按位权把 0x12345678 分为 12、34、56、78 四个字节。
2. 按地址从低到高填写：大端写 12 34 56 78，小端写 78 56 34 12。
3. 只改变字节排列，不把每个字节的二进制位反转；MSB/LSB 的位权含义也不改变。

补充原理与条件

大端低地址存最高有效字节，小端低地址存最低有效字节；一个字节内部的位权不会因大小端而反转。

02 例子

0x12345678：大端低地址为 0x12，小端为 0x78。

03 易错点

错误说法低地址一定存放数值的最高位。

自检：小端存储是否反转每个字节的 8 个 bit?

不反转。

有符号与无符号比较：看转换规则　混合比较先看类型转换，不能一律按无符号解释。　个人错题　易错　易混　近年考查　要点

核心结论

硬件位串需按指定类型解释；C 混合比较还要按整数提升和通常算术转换判断。

别混淆混合比较先看类型转换，不能一律按无符号解释。

详解

01 解析

1. 先写出两个操作数提升后的类型，不能一见 unsigned 就立即决定所有值都转无符号。
2. 同等级 int 与 unsigned int 比较时通常转为 unsigned int；较高等级有符号类型若能覆盖对方全部取值，可保留有符号比较。

3. 类型确定后再解释位串。例如同宽 -1 转成无符号最大值，再与 1 比较，结果为大于。

补充原理与条件

同等级 signed int 与 unsigned int 常转为 unsigned；若更高等级有符号类型能表示较低等级无符号类型的全部值，则可转为该有符号类型。

02 例子

同宽 int 与 unsigned int 比较：-1 通常转为最大的 unsigned int。不能把这一例推广到所有类型组合。

03 易错点

错误说法混合比较一律按无符号，或最高位为 1 就必为负数。

自检：比较前先核对哪些条件?

类型、位宽、等级、表示范围及题设语言规则。

符号扩展、零扩展与截断　负补码扩宽补 1；无符号扩宽补 0。　　易错　易混　近年考查　记忆概念　　要点

核心结论

补码有符号数扩宽用符号扩展；无符号数扩宽用零扩展；截断可能改变值。

别混淆负补码扩宽补 1；无符号扩宽补 0。

详解

01 解析

1. 先确定原数据按补码有符号数还是无符号数解释。
2. 有符号扩宽复制原最高位，无符号扩宽补 0；检验扩宽前后的数学值是否一致。
3. 缩窄则取低位并重新按目标类型解释。若原数超出目标范围，不能因“低位没变”就说数值不变。

补充原理与条件

补码符号扩展保持数值不变。低位截取只有原值在目标范围内才保持数值；浮点扩精度有不同格式转换规则。

02 例子

8 位补码 11111111 表示 −1，扩为 16 位应为 1111111111111111。

03 易错点

错误说法提高精度就是左侧一律补 0。

考点联系

截断后再扩展，必须分两步解释位串。

变式

自编模拟变式 · 截断后再扩展

在题设 32 位 int、16 位 short 的补码模型中，整数 32777 转 short，再转回 int，结果是否仍为 32777?

答案与解析

不是。截取低 16 位后为 0x8009，即 −32759；再符号扩展仍为 −32759。

关键区别：扩宽能保留当前 short 的数值，不能找回此前截断丢掉的高位。

自检：负补码扩宽时高位补什么?

补符号位 1。

IEEE 754：正规、非正规、无穷与 NaN　E=0 用隐含 0；非正规数指数是 −126，不是 −127。　　记忆概念　易错　易混　　要点

核心结论

单精度 E 为 8 位、F 为 23 位。正规数 1⩽E⩽254；E=0 用隐含 0，E=255 表示无穷或 NaN。

别混淆E=0 用隐含 0；非正规数指数是 −126，不是 −127。

详解

01 解析

正规数=(−1)^s×(1.F)₂×2^(E−127)；非正规非零数=(−1)^s×(0.F)₂×2^(−126)。不要把 E=0 的指数写成 −127。

02 例子

E=0、F=0 表示 ±0；E=255、F=0 表示 ±∞；E=255、F≠0 表示 NaN。

03 易错点

错误说法阶码全 0 或全 1 时都照正规数的隐含 1 公式解释。

04 图表

要点对照

阶码 E	尾数 F	解释
0	0	正零/负零
0	非 0	非正规数
1～254	任意	正规数（隐含 1）
255	0	正/负无穷

阶码 E	尾数 F	解释
255	非 0	NaN

自检：单精度非正规数是否有隐含的最高位 1?

没有，隐含位为 0。

原码、反码、补码与移码：范围和零　补码零唯一且多一个最小负数；移码关系取决于偏置。　　记忆概念　易错　易混　　要点

核心结论

n 位补码范围为 $-2^{n-1}\sim 2^{n-1}-1$，零唯一；常见原码/反码范围为 $\pm(2^{n-1}-1)$，零有正负两种。
别混淆补码零唯一且多一个最小负数；移码关系取决于偏置。

详解

01 解析

移码是给真值加偏置；只有特定偏置下才可与补码用翻最高位建立简单关系。IEEE 阶码偏置不是任意普通移码例题的偏置。

02 例子

8 位补码能表示 −128，8 位原码通常不能。

03 易错点

错误说法n 位补码与原码的负数范围完全相同；移码总只翻符号位。

自检：将负数补码取真值，只取低 7 位就对吗?

不对，应按补码权值或取反加一等正确方式求值。

奇偶校验与海明码：检错和纠错能力　最小码距 d：检 d−1 位，纠 ⌊(d−1)/2⌋ 位。　　记忆概念　易错　易混　　要点

核心结论

奇偶校验可检出奇数位翻转，不能定位；海明码通过多个校验关系形成综合征，典型最小距离 3 的码可纠一位。

别混淆最小码距 d：检 d−1 位，纠 $\lfloor (d-1)/2 \rfloor$ 位。

详解

01 解析

$d_{\min}\geq t+1, d_{\min}\geq 2t+1$

分别为保证检出至多 t 位差错、保证纠正至多 t 位差错的码距要求。

若要同时实现单错纠正和双错检测，常加整体奇偶位成为扩展海明码。理论能力由最小码距决定：检 d−1 位、纠 $\lfloor (d-1)/2 \rfloor$ 位。

02 例子

翻转两位可能让普通奇偶校验仍满足，而不是一定被检出。

03 易错点

错误说法单个奇偶位可以检测所有多位错误并定位错误位置。

自检：m 个信息位、r 个海明校验位常需满足什么基本关系?

2^r⩾m+r+1，为无错及每个单比特错误预留可区分状态。

运算与浮点

CF 与 OF：进位不等于有符号溢出　CF 看无符号，OF 看有符号；减法借位规则看题设。　易错　易混　近年考查　记忆概念　　要点

核心结论

CF 反映无符号进位/借位相关情况，OF 反映有符号溢出；具体减法借位标志定义看机器约定。

别混淆CF 看无符号，OF 看有符号；减法借位规则看题设。

详解

01 解析

1. 先固定运算位宽，再完成位串加法，记录最高位之外的进位。
2. 无符号加法用该进位判断是否超范围；有符号加法看“同号相加是否得到异号结果”。
3. 8 位 01111111+1 得 10000000：无第 9 位进位，却从两个非负数算出负补码，故 OF=1、加法 CF=0。

补充原理与条件

同一位串运算可按有符号或无符号解释。补码加法同号相加结果异号发生有符号溢出；不能仅看最高位进位。

02 例子

8 位 127+1 得位串 10000000：有符号溢出，未产生第 9 位进位。

03 易错点

错误说法最高位有进位就说明补码溢出。

自检：8 位 255+1 按无符号解释的结果是什么?

模 256 结果为 0，并有进位；不能由此直接推出 OF=1。

补码算术移位：右移与左移的边界　算术右移补符号；负奇数右移不等于向 0 截断。　个人错题　易错　要点

核心结论

固定字长的补码算术右移补符号位，右移一位相当于除以 2 后向负无穷取整；左移可能溢出。C 语言表达式还须另查类型、版本和移位规则。

别混淆算术右移补符号；负奇数右移不等于向 0 截断。

详解

01 解析

1. 把 -3 写成 8 位补码 11111101，右移时保留最高符号位并丢弃最低位。
2. 结果为 11111110，即 -2；因为丢弃的低位为 1，负奇数的算术右移对应向负无穷取整。
3. 若换成逻辑右移，高位补 0 后结果变成正数；若题目问 C 源程序，还要单独考虑其类型和语言规则。

补充原理与条件

逻辑右移补 0。题目若指定 C 语言而非机器移位，应注意版本和实现规则，不把硬件题的结论无条件套到源语言。

02 例子

8 位补码 −3 算术右移一位常得到 −2，而不是 −1。

03 易错点

错误说法负数算术右移总是按向 0 取整除以 2；左移永远等价于乘 2。

自检：逻辑右移负补码与算术右移是否相同?

不同，补入的高位不同。

浮点对阶、规格化与舍入　左规阶码减，右规阶码加；舍入后可能再右规。　个人错题　易混　近年考查　记忆概念　要点

核心结论

对阶通常小阶向大阶、尾数右移；左规尾数左移、阶码减小；右规尾数右移、阶码增大。

别混淆左规阶码减，右规阶码加；舍入后可能再右规。

详解

01 解析

保持数值不变需要尾数和阶码反向变化。舍入可能再次引起右规，不能只在开始检查阶码范围。IEEE 与题设补码尾数格式的规格化判据不同。

02 例子

尾数左移一位放大 2 倍，阶码减 1 才抵消该变化。

03 易错点

错误说法左规时阶码加 1；对阶把较大阶码向较小阶码调整。

自检：IEEE 单精度正规数的隐含位是否用于零和非正规数?

不用于；正规数隐含 1，非正规数隐含 0。

浮点精度与表示范围：大范围不保证整数精确　范围大不代表范围内所有整数都能精确表示。　　易错　易混　近年考查　记忆概念　　要点

核心结论

表示范围由阶码等决定，有效精度由尾数决定；IEEE 单精度正规数通常有 24 位有效二进制精度。

别混淆范围大不代表范围内所有整数都能精确表示。

详解

01 解析

单精度可连续精确表示的整数范围通常到 ±2^24；超过后并非所有整数可精确表示。双精度有 53 位有效精度。

02 例子

2^24 可精确表示，但 $2^{24}+1$ 在单精度中通常不能精确表示。

03 易错点

错误说法单精度浮点能表示很大的数，因此范围内所有整数都能精确表示。

考点联系

选择数值类型时，精度要求与范围要求须同时满足。

变式

自编模拟变式 · 浮点范围足够就能精确吗

单精度浮点数能够精确表示所有不超过 2^{30} 的整数吗?

答案与解析

不能。单精度正规数通常只有 24 位有效二进制精度；$2^{24}+1$ 就不能被精确表示。

关键区别：“范围覆盖”解决能否达到数量级，“有效位数”解决相邻整数能否区分。

自检：选存储格式只看是否溢出够吗?

不够，还要检查所需精度。

串行乘法器：被乘数、乘数与部分积 组合右移模型要保留加法进位 C。 记忆概念 易错易混 图解 要点

核心结论

乘法基本操作是“按乘数位决定加法，再移位”；硬件含被乘数寄存器、部分积/乘数寄存器、加法器和控制计数。

别混淆组合右移模型要保留加法进位 C。

详解

01 解析

图采用无符号组合右移模型：M 保存被乘数，Q 初始保存乘数，A 初始为 0；Q_0=1 则 A+M，经 C:A:Q 联合逻辑右移。保留加法进位 C，进行 n 轮，最终 A:Q 是 2n 位乘积。

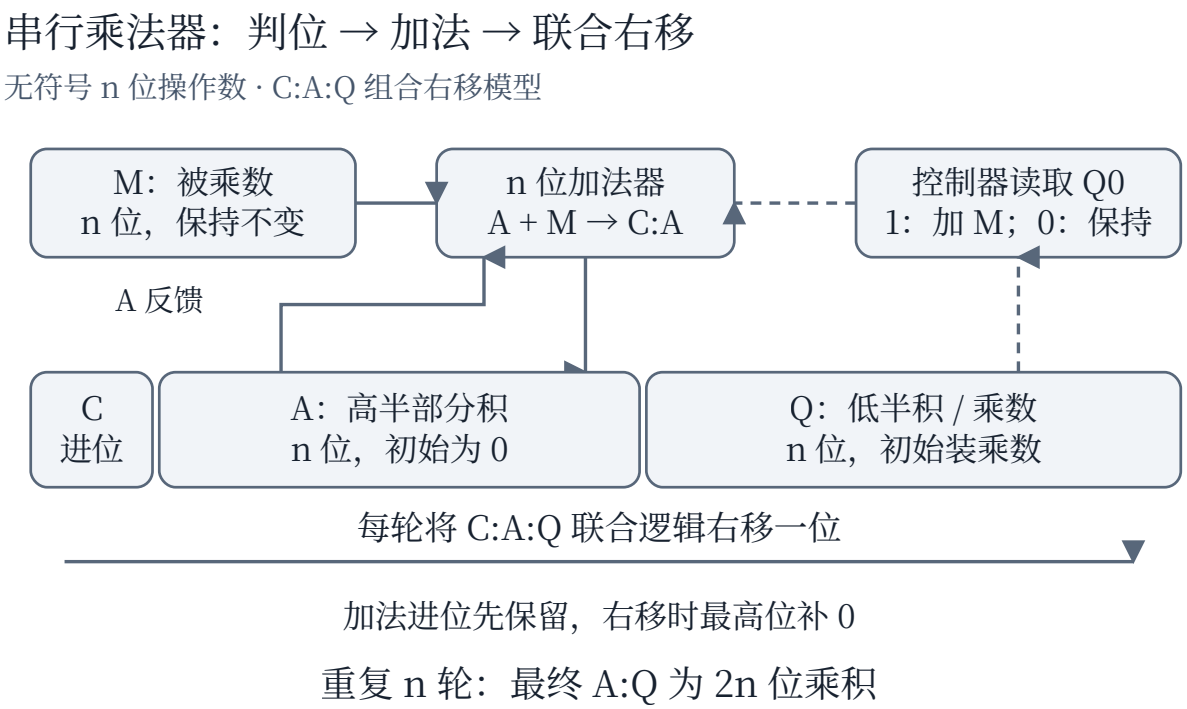
02 例子

4 位无符号 15×15 需要 8 位保存 225；若丢掉加法进位可能算错。

03 易错点

错误说法乘法器只需要一个结果寄存器，移位一定只移动被乘数。

04 图表



单独查看图片

参考资料

参考：高校计组讲义：移位加法与除法硬件

自检：乘积最大需要多少位?

两个 n 位无符号数的完整乘积需要至多 2n 位。

阵列乘法器与迭代乘法器　迭代轮数不等于任意乘法器的时钟周期数。　　记忆概念　易错　易混　　要点

核心结论

迭代乘法器复用加法器，分多轮加移；阵列乘法器用硬件并行生成并累加部分积。

别混淆迭代轮数不等于任意乘法器的时钟周期数。

详解

01 解析

阵列实现可减少迭代轮次但组合延迟更长、硬件更多；是否一拍完成取决于电路、时钟周期和流水化。不能把算法步骤数直接当任意机器的周期数。

02 例子

同一乘法指令可由组合阵列、多周期迭代或流水化单元实现。

03 易错点

错误说法所有乘法器都必须花 n 个时钟周期；阵列乘法完全不使用加法。

考点联系

乘法器硬件实现与指令执行时间不能混同。

自检：乘法指令是否必然只用一个 CPU 周期?

不必然，需看具体实现与题设。

Booth 补码乘法：看 Q_0Q_{-1} 的顺序　顺序固定 Q_0Q_{-1}：01 加、10 减，再算术右移。　　记忆概念　易错　易混　图解　　要点

核心结论

规定观察顺序为 Q_0Q_{-1}：01 加 M，10 减 M，00/11 不加减，然后 A:Q:Q_{-1} 算术右移。

别混淆顺序固定 Q_0Q_{-1}：01 加、10 减，再算术右移。

详解

01 解析

Q_{-1} 初始为 0，用相邻位变化编码连续 1。必须保留符号，并给 −M 留出可表示的位宽，尤其 M 为最小负数时；不同教材若写 $Q_{-1}Q_0$，表的方向须转换。 若中间 A 扩成 n+1 位，最终去掉冗余符号扩展位，按题设取 2n 位乘积。

02 例子

Q_0=1、Q_{-1}=0 对应 10，所以 A←A−M，不是 A+M。

03 易错点

错误说法01 做减法，10 做加法；每轮使用逻辑右移。

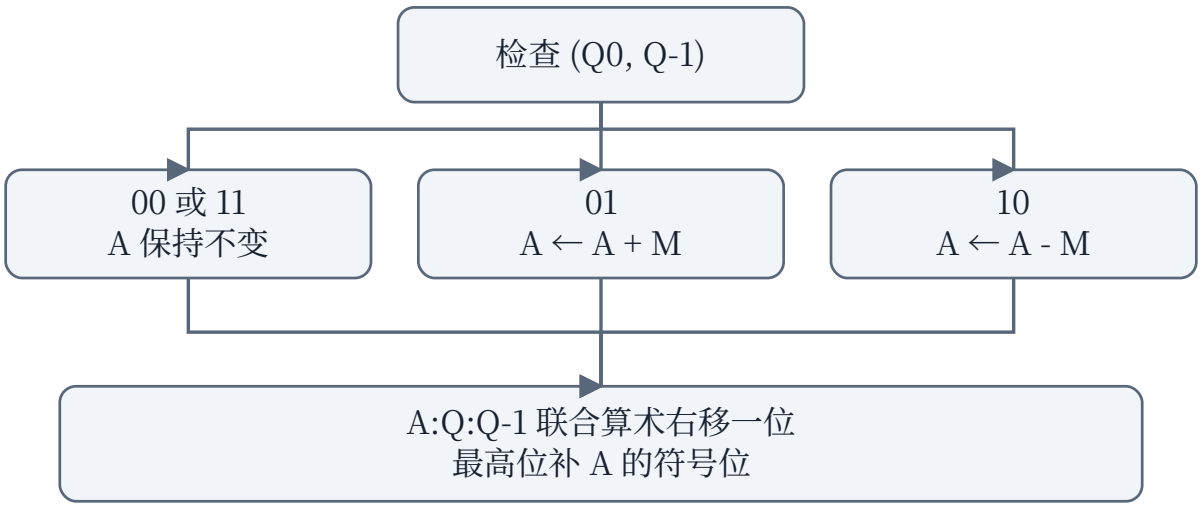
04 图表

要点对照

Q_0Q_{-1}	移位前操作
00 或 11	A 不变
01	A←A+M
10	A←A−M

Booth 判位规则：01 加、10 减

补码乘法 · 观察顺序固定为 (Q0, Q-1)



Q-1 初始为 0；重复 n 轮后读取 A:Q

M 为最小负数时，A 与 ±M 须用足够位宽

单独查看图片

参考资料

参考：IIT Kharagpur：Booth 乘法虚拟实验

自检：Booth 右移最高位补什么?

补 A 的符号位；不是一律补 0。

除法器：除数、商和余数存放位置　Q 从被除数变为商；A 最后保存余数。　　记忆概念　易错　易混　图解　　要点

核心结论

概念模型中 M 保存除数，A 保存部分余数，Q 初始装被除数并逐位形成商；位置是模型约定，不是所有 ISA 的固定寄存器号。

别混淆Q 从被除数变为商；A 最后保存余数。

详解

01 解析

图采用恢复余数的无符号迭代模型：A:Q 左移→A−M→若负则恢复并写商位 0，非负写商位 1。A 需留足试减的额外位，迭代 n 轮，最终 Q 为商、A 为余数。

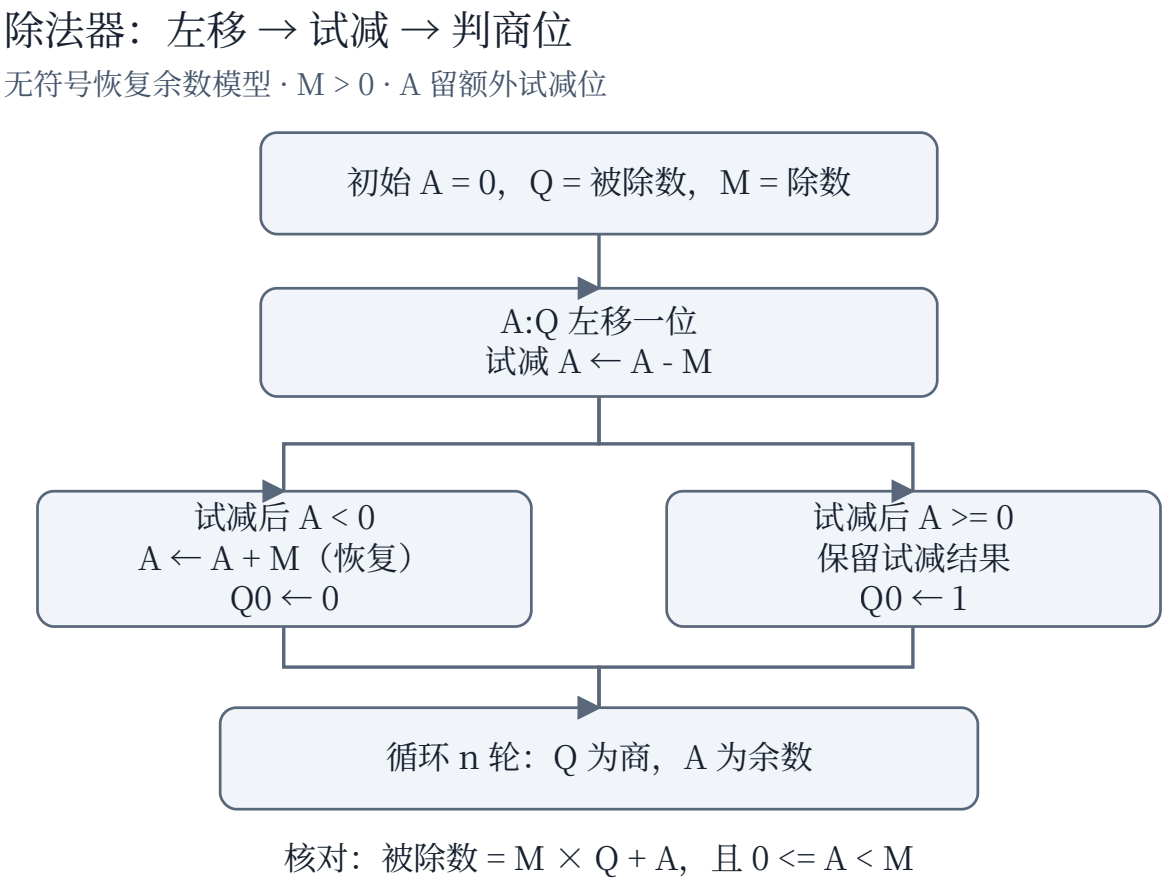
02 例子

13÷3 的最终商为 4、余数为 1，满足 13=3×4+1。

03 易错点

错误说法商和余数存放位置永远是某两个固定编号的 CPU 寄存器。

04 图表



单独查看图片

参考资料

参考：UC San Diego：乘除法与恢复/不恢复余数

自检：循环开始时 Q 保存什么，结束时保存什么?

开始保存被除数，结束保存商。

恢复余数与不恢复余数：恢复的是余数　不恢复法不是永不修正，末尾负余数仍可能需加 M。　　记忆概念　易错　易混　　要点

核心结论

恢复法试减为负时加回除数；不恢复法根据前轮余数符号选择下一步加或减，结束时可能仍需修正余数。

别混淆不恢复法不是永不修正，末尾负余数仍可能需加 M。

详解

01 解析

无符号标准移位版本：旧 A≥0 时下一轮左移后减 M；旧 A<0 时左移后加 M，再据结果设置商位。若最终 A<0，补加 M 修正。补码有符号算法另看题设。

02 例子

“下一轮用加法替代立即恢复”不等于永远没有任何恢复或末尾校正。

03 易错点

错误说法“不恢复余数”意味着最终负余数可以原样输出。

参考资料

参考：UC San Diego：乘除法与恢复/不恢复余数

自检：不恢复法的最终余数是否仍需满足范围?

需要，无符号且 M>0 时应满足 0≤R<M。

除法结果边界：零除与商溢出　除数非零仍可能商溢出：最小负数除以 −1。　　记忆概念　易错　易混　　要点

核心结论

还要检查目标商是否可表示；n 位补码最小值 -2^{n-1} 除以 −1，商为 2^{n-1}，超出 n 位有符号范围。
别混淆除数非零仍可能商溢出：最小负数除以 −1。

详解

01 解析

对双字长被除数/单字长商，也需专门检查商位宽。机器整数除法和浮点除法的异常处理不同，不能混用。

02 例子

8 位补码 −128÷(−1) 的数学商为 128，无法装入 8 位有符号结果。

03 易错点

错误说法只检查除数不为 0 就能保证有符号除法合法。

自检：8 位 −128÷1 是否也会商溢出?

不会，−128 在 8 位补码范围内。

补码加减：减法变加法与双符号位　双符号位 01 正溢出、10 负溢出；减法是加负数补码。　记忆概念　易错　易混　要点

核心结论

A−B 以 A+（−B 的补码）实现；常见双符号位运算中 01 为正溢出、10 为负溢出，00/11 未发生该类溢出。

别混淆双符号位 01 正溢出、10 负溢出；减法是加负数补码。

详解

01 解析

1. 先保持同一 n 位位宽，用 A−B=A+(-B)补完成减法，末尾只保留 n 位结果。
2. 另用输入输出符号或双符号位检查有符号溢出；截去进位并不能证明结果正确。
3. 对最小负数尤其小心：n 位补码的 -2^{n-1} 取负后超出正数范围，不能按普通可表示正数处理。

补充原理与条件

位宽固定时结果截取低位，但有符号溢出仍要检查。形成 −B 时注意最小负数边界；借位、进位、OF、符号位不能混称。

02 例子

正数减负数可能正溢出，而不同号相加通常不会有有符号加法溢出。

03 易错点

错误说法补码减法等于把被减数取补；双符号位相同表示溢出。

自检：补码减法的 OF 能只看最高位进位吗?

不能，还需按减法符号关系或最高位进/出位关系判断。

串行进位与先行进位：进位链决定延迟　先行进位改变进位实现与延迟，不改变正确结果。　记忆概念　易错　易混　要点

核心结论

串行进位逐级等待低位进位；先行进位根据产生/传递函数并行推导进位，缩短关键路径。

别混淆先行进位改变进位实现与延迟，不改变正确结果。

详解

01 解析

两者都执行相同二进制加法，但延迟和硬件代价不同。多级先行进位还按组组织，不能只按全宽线性级数估算。

02 例子

8 位加法的最高位不必在先行结构中等 7 次逐位传播才得到进位。

03 易错点

错误说法先行进位是把整数的各位顺序相加，只提高时钟频率。

自检：两种加法器是否会对同一输入给不同正确和?

不会，差别是实现与速度等代价。

浮点四步：对阶、运算、规格化、舍入　舍入可能再次右规，最终还要检查阶码范围。　记忆概念　易错　易混　要点

核心结论

加减先对阶，再对尾数运算，规格化并舍入；乘除则按规则处理阶码与尾数，最终仍检查规格化和范围。

别混淆舍入可能再次右规，最终还要检查阶码范围。

详解

01 解析

对阶尾数右移可能丢位，需保护/舍入/粘滞信息。舍入进位可能需要再次右规；阶码上溢与尾数暂时超范围不同，下溢可能转为非正规数或按题设处理。

02 例子

两个尾数相加超过正规范围，可右移尾数并增加阶码。

03 易错点

错误说法浮点相加可直接加阶码；舍入发生一次后无需再检查溢出。

自检：舍入后尾数变为 10.000… 时应怎么处理?

再次右规并更新阶码，检查最终可表示性。

存储系统

Cache：块内偏移、组号、标记　先算组数；比较标记和值有效位后才判命中。　个人错题　易错　近年考查　记忆概念　要点

核心结论

主存块号=地址/块大小向下取整；组号=块号 mod 组数；标记=块号/组数向下取整。

别混淆先算组数；比较标记和值有效位后才判命中。

详解

01 解析

$S=$

C 为 Cache 数据容量，B 为块大小，w 为路数，S 为组数；地址与块大小单位一致。

$s=\lfloor _ \rfloor \bmod S, t=\lfloor _ \rfloor$

A 为地址，s 为组号，t 为标记数值；标记位宽另按地址长度计算。

1. 先统一单位：组数 S=Cache 数据容量/(块大小 B×路数)，地址与 B 必须使用同一编址单位。
2. 计算块号 $\lfloor _ \rfloor$、组号 $\lfloor _ \rfloor \bmod S$、标记 $\lfloor _ \rfloor$，余数 $A \bmod B$ 为块内偏移。
3. 定位组后仍须检查各路有效位及标记；组号相同只说明竞争同一组，不能据此判断命中。

补充原理与条件

按字节编址且参数为 2 的幂时：低位为块内偏移，中间为组索引，高位为标记。组数=数据容量/(块大小×路数)，不能把路数当组数。

02 例子

两块映射到同一组，还需要比较标记；组相同不代表命中。

03 易错点

错误说法主存块号就是 Cache 组号；标记包含组号。

考点联系

TLB 与 Cache 都有映射字段，但处理的地址对象不同。

变式

自编模拟变式 · 把 Cache 拆分迁移到 TLB

虚拟地址 32 位、页大小 1 KiB；TLB 有 32 项、4 路组相联，不计 ASID 等附加字段。标记需多少位?

答案与解析

页内偏移 10 位，虚拟页号 22 位；TLB 为 8 组，组索引 3 位，标记为 22−3=19 位。

关键区别：TLB 按虚拟页号分组，不按主存页框号拆标记；它缓存的是页映射。

自检：增加路数且总数据容量固定，会怎样影响组数?

组数减少，索引位减少；地址位数和块大小不变时标记位增加。

写直达/写回与写分配：两套决策　写命中策略与写缺失策略是两个维度。　　易混　近年考查　　要点

核心结论

写直达/写回规定写命中后如何更新下层；写分配/非写分配规定写不命中时是否把块调入。

别混淆写命中策略与写缺失策略是两个维度。

详解

01 解析

1. 先判命中与否：写命中才用写直达/写回决定怎样更新下一级。
2. 写缺失先看分配策略：写分配调入块后修改；非写分配把这次写交给下级而不调入该块。
3. 写回替换时再检查脏位：仅被修改的块需要写回。把“脏块写回”和“写缺失是否调入”分开计数。

补充原理与条件

两套策略分别回答不同问题。写回通常需要脏位；替换干净块无需写回。题目给出组合时必须按组合执行。

02 例子

写回且写分配：写缺失先调块，修改后置脏，替换脏块再写回。

03 易错点

错误说法写回一定等于写分配；写直达必然没有缓存。

自检：有效位和脏位能互相替代吗?

不能；有效位表示内容是否有效，脏位表示是否相对下层被修改。

MAR、MDR 与按字节编址　容量看地址位数和编址单位，不看 MDR 位宽。　　易错　易混　　要点

核心结论

MAR 保存地址，其位数关联寻址范围；MDR 保存一次数据传送的内容，位数关联数据通路。

别混淆容量看地址位数和编址单位，不看 MDR 位宽。

详解

01 解析

按字节编址时 n 位地址可表示 2^n 个字节地址。按字编址则表示 2^n 个字，容量还需乘每字字节数。

02 例子

MAR 为 24 位而 MDR 为 32 位，不矛盾；不能用 MDR 位数直接计算地址容量。

03 易错点

错误说法MAR 位数必须等于 MDR 位数。

自检：24 位字节地址能表示多少地址空间?

2^24 B，即 16 MiB。

SRAM、DRAM、ROM 与刷新　SRAM 不刷新但仍易失；DRAM 刷新与读后恢复不同。　　记忆概念　易错　易混　　要点

核心结论

SRAM 依靠双稳态电路保存，通常不需刷新；DRAM 用电容保存，需刷新且读出常需恢复。ROM 类用于非易失保存。

别混淆SRAM 不刷新但仍易失；DRAM 刷新与读后恢复不同。

详解

01 解析

DRAM 容量密度高、常作主存，SRAM 较快常作 Cache。刷新并非 CPU 逐条执行读写指令完成；读写周期与刷新开销要区分。

02 例子

断电后普通 SRAM/DRAM 数据通常丢失，不能因 SRAM 不刷新就认为它非易失。

03 易错点

错误说法DRAM 读出不需恢复，SRAM 每隔一段时间必须刷新。

自检：DRAM 是否因为断电前会刷新就能长期断电保存?

不能，仍属易失存储器。

存储芯片扩展：位扩展与字扩展　位扩展加位宽，字扩展加地址数；片选别漏。　　记忆概念　易错　易混　　要点

核心结论

位扩展并联增加一个地址的数据位数；字扩展增加可寻址字数，需要片选区分组。

别混淆位扩展加位宽，字扩展加地址数；片选别漏。

详解

01 解析

1. 先做位扩展：目标位宽/单片位宽，决定同一组并联多少片，共享地址和片选，分别接不同数据位。

2. 再做字扩展：目标字数/单片字数，决定有多少组，由高位地址译码选组。
3. 总片数为两者乘积；8K×8 组成 16K×16 时，每组 2 片、共 2 组，总计 4 片。

补充原理与条件

总片数可按目标总位数/单片总位数求，但仍需按每组位宽、组数和地址线划分连线。读写控制、地址输入可共用，数据线和片选要按方案组织。

02 例子

8K×8 的芯片构成 16K×16：每组 2 片位扩展，2 组字扩展，共 4 片。

03 易错点

错误说法芯片数量只等于总容量比，连线方式不必区分。

自检：位扩展时，同组芯片的地址输入是否通常相同?

相同，分别提供该地址不同的数据位。

低位交叉与高位交叉：连续地址落在哪里　低位交叉对连续访问有利，固定步长仍可冲突。　　记忆概念　易错　易混　　要点

核心结论

低位交叉用低地址位选模块，连续地址分散到不同模块，适合重叠访问；高位交叉用高位选模块，连续块可在同一模块。

别混淆低位交叉对连续访问有利，固定步长仍可冲突。

详解

01 解析

1. 低位交叉用地址低位选模块，模块号通常为地址 mod 模块数。
2. 连续地址会轮流落到不同模块；步长若等于模块数，则可能反复访问同一模块而发生冲突。
3. 计算并行带宽前再比较模块存储周期与发起间隔；“低位交叉”本身不保证任何访问序列都无等待。

补充原理与条件

并行访存受模块忙闲、周期和发起间隔限制。对步长与模块数有规律的访问，低位交叉也可能持续冲突，不能保证任意访问都满带宽。

02 例子

4 模块低位交叉时，地址 0、1、2、3 分别去模块 0、1、2、3。

03 易错点

错误说法低位交叉让连续地址全落在同一模块。

自检：访问步长恰为 4，在 4 模块低位交叉中一定分散吗?

不一定，可能总落在同一模块。

Cache 三种映射：比较器、冲突与地址字段　全相联无行索引字段，仍有标记和块内偏移。　　记忆概念　易错　易混　　要点

核心结论

直接映射每块只有一个行位置；全相联可放任意行；组相联先定位组，再在组内任意路寻找。

别混淆全相联无行索引字段，仍有标记和块内偏移。

详解

01 解析

直接映射比较少但冲突限制强；全相联查找代价大，索引字段可无；组相联折中。每一路保存自身标记与状态；命中需有效位和标记匹配。

02 例子

两主存块映同一直接行会互相替换；组内有多个路时可同时驻留。

03 易错点

错误说法全相联仍必须有 Cache 行号字段；直接映射能把冲突块放到任意行。

自检：全相联的块内偏移是否也消失?

不会，仍需在块内选择字节/字。

虚拟存储硬件：页表基址、MMU 与保护　存在位有效不代表写权限通过。　　记忆概念　易错　易混　　要点

核心结论

MMU 按页表/TLB 完成地址转换和保护检查；页表项记录页框、存在与权限等，页表基址帮助找到页表。

别混淆存在位有效不代表写权限通过。

详解

01 解析

TLB 可能包含地址空间标识以区分进程；切换时如何处理看实现。页框号与页内偏移拼出物理地址，缺页由硬件检测并转交 OS 处理。

02 例子

同一虚拟页号在两个进程中可映射到不同物理页框。

03 易错点

错误说法MMU 保存整个进程的所有数据，页表只记录是否在 Cache 中。

自检：页表中存在位有效但权限不允许写，写访问能否直接通过?

不能，仍需保护检查，可能触发保护异常。

存取时间、存储周期与突发传输　延迟看首个结果，吞吐还要看后续访问可重叠的间隔。　记忆概念　易错　易混　　要点

核心结论

存取时间是请求到数据有效的延迟；存储周期还包括恢复等使下次同类操作可开始的间隔；CPU 时钟周期是处理器节拍。

别混淆延迟看首个结果，吞吐还要看后续访问可重叠的间隔。

详解

01 解析

突发传输常首次延迟较长，随后连续传多字，摊薄地址/访问开销。刷新、行命中/未命中等可改变 DRAM 时序，题目必须明确计时模型。

02 例子

总线有数据阶段不代表主存每个周期都能接受独立访问，仍受存储周期约束。

03 易错点

错误说法存取时间、存储周期与 CPU 时钟周期永远相等。

自检：求吞吐率能否只用首次数据访问延迟而忽略连续传送间隔?

不能，需按完整事务和重叠规则计算。

指令与数据通路

PC：顺序后继、跳转目标与返回地址　返回地址来自先前保存的断点，不是 RET 的顺序后继。　　个人错题　易混　　要点

核心结论

顺序执行按指令长度推进；CALL 设置调用目标并保存返回地址；RET 从保存位置取得返回地址。

别混淆返回地址来自先前保存的断点，不是 RET 的顺序后继。

详解

01 解析

“下一条执行的指令”可以是转移目标，而“内存中紧邻的下一条”指顺序后继。即使两个数值恰好相同，也不改变地址来源和指令语义。

02 例子

CALL 的目标可恰好紧邻，但它仍按调用语义执行；RET 通常返回对应调用后的指令。

03 易错点

错误说法CALL 或 RET 的目标偶然等于下一条地址，就算顺序执行。

自检：RET 的返回位置是否总是 RET 自己的顺序后继?

不是，通常是先前保存的调用后续地址。

PC、IR 与指令译码器：控制部件　PC 管地址，IR 存指令，译码器分析操作码。　　个人错题　易混　近年考查　　要点

核心结论

PC 保存指令地址，IR 保存当前指令，译码器分析操作码；通常都归入控制部件。

别混淆PC 管地址，IR 存指令，译码器分析操作码。

详解

01 解析

IR 不一定在每个执行微操作中变化。ALU 完成运算，控制器生成相应微操作控制信号；具体组成以题设模型为准。

02 例子

指令取出后送入 IR，随后译码；执行期间 PC 可能已指向后继。

03 易错点

错误说法IR 存的是运算数据，译码器完成算术运算。

自检：指令译码器的主要输入是什么?

通常是 IR 中的操作码等指令字段。

有效地址与间接寻址：地址不等于数据 间址得到的可能仍是地址，别提前当成操作数。 个人错题 易错 要点

核心结论

先按寻址方式求有效地址 EA，再依指令访问 M[EA]；立即数是指令内的数据。

别混淆间址得到的可能仍是地址，别提前当成操作数。

详解

01 解析

1. 对照寻址方式，先算有效地址 EA，暂不把地址当作操作数。
2. 直接寻址 EA=A；一次间接寻址先取 EA=M[A]；最后再取 M[EA] 才得到存储器操作数。
3. 题目若问访存次数，先分清是否包含取指，再逐次记录读取指针和读取操作数的访问。

补充原理与条件

直接寻址 EA=A；一次间接寻址 EA=M[A]；基址/变址/相对寻址还需叠加相应寄存器。计算时须区分地址、地址处的指针和最终操作数。

02 例子

一次间接寻址：A=100，M[100]=200，M[200]=7，操作数是 7。

03 易错点

错误说法M[address] 总能直接作为操作数。

自检：间接寻址中 M[A] 一定是最终数据吗?

不是，通常是有效地址。

ISA 与微体系结构：可见接口和实现细节 ISA 是软件可见接口，不等于具体流水线和电路。 冷门 易混 近年考查 要点

核心结论

ISA 规定软件可见的指令、寄存器、数据类型及寻址等接口；流水线级数和具体电路通常属于实现。

别混淆ISA 是软件可见接口，不等于具体流水线和电路。

详解

01 解析

相同 ISA 可由不同处理器实现，性能和内部结构可以不同。某些状态寄存器或扩展特性是否可见仍要看具体 ISA。

02 例子

支持相同指令集的两个 CPU 可以有不同 Cache 和不同主频。

03 易错点

错误说法时钟周期、Cache 容量和加法器结构都属于 ISA 的必然规定。

自检：指令操作码和寻址方式属于哪一层?

通常属于 ISA。

指令格式：操作码、地址码与扩展操作码 短操作码占用的整个前缀空间不能再分配长操作码。 记忆概念 易错 易混 要点

核心结论

固定总长下，操作码与地址字段共享位数预算；扩展操作码通过前缀区分不同格式，需要避免编码冲突。

别混淆短操作码占用的整个前缀空间不能再分配长操作码。

详解

01 解析

多地址指令耗较多地址字段，留给短操作码的位数较少。已使用的短操作码前缀会占用对应的所有更长编码，不可重复计数。

02 例子

一个已分配的 k 位操作码，可排除具有该前缀的 2^{l-k} 个 l 位长编码。

03 易错点

错误说法操作码增加一位不影响地址空间；扩展操作码按最短码长度直接相加。

自检：指令字长是否必与机器字长相同?

不必，可固定或可变，也可跨多个机器字。

基址、变址与相对寻址：谁提供变化 相对寻址的 PC 时刻与位移符号扩展都须按题设。 记忆概念 易错 易混 要点

核心结论

基址：EA=基址寄存器+位移；变址：EA=变址寄存器+地址/位移；相对：EA=题设时刻的 PC+有符号位移。

别混淆相对寻址的 PC 时刻与位移符号扩展都须按题设。

详解

01 解析

位移可能有符号，需要扩展。PC 究竟是当前指令地址还是已更新后的顺序后继，要看题设；基址与变址在硬件求和相似，但用途和可变部分不同。

02 例子

数组遍历常改变变址寄存器；程序重定位常调整基址。

03 易错点

错误说法基址、变址、相对寻址都以 IR 地址字段直接作最终地址。

自检：相对位移为负时应一律零扩展吗?

不应，按题设有符号位移进行符号扩展。

RISC/CISC 与 Load/Store　RISC 利于流水，不代表所有指令都固定一拍。　　记忆概念　易错　易混　　要点

核心结论

典型 RISC 强调较规则的指令、寄存器操作和 Load/Store 访问存储器，利于流水；CISC 通常有较复杂格式与操作。

别混淆RISC 利于流水，不代表所有指令都固定一拍。

详解

01 解析

这些是风格特征，不是对所有实现的绝对保证。现代处理器可把复杂指令分解成内部操作；访存、乘除或异常可能使执行时间不同。

02 例子

典型 Load/Store ISA 中算术指令主要在寄存器间运算，访存由专门指令完成。

03 易错点

错误说法RISC 的所有指令都固定一拍完成；CISC 不可能流水化。

自检：RISC 是否必然不用任何多周期功能单元?

不必然，乘除等可以多周期。

单总线数据通路：暂存寄存器和微操作　单总线通常只能有一个驱动源，ALU 常需暂存操作数。　记忆概念　易错　易混　要点

核心结论

单总线一次通常只有一个源驱动；ALU 常需 Y 等暂存第一个操作数，Z 等暂存结果，再按微操作转送。

别混淆单总线通常只能有一个驱动源，ALU 常需暂存操作数。

详解

01 解析

1. 每个微步骤先找总线驱动源：单总线同一时刻不能由两个不同寄存器同时输出。
2. 两个操作数相加时，把第一个先存入 Y，再让另一个经总线进入 ALU，结果暂存 Z。
3. 最后令 Z 输出并由目标寄存器写入。每步核对源输出、ALU 功能、目标装入三个条件，不能只写寄存器传送愿望。

补充原理与条件

控制信号需同时满足总线不冲突、源有效、目标写入、ALU 功能及存储器时序。MAR 送地址、MDR 传数据不能互换；具体寄存器名看题图。

02 例子

实现 R1←R1+R2 可分为 R1→Y、R2与Y运算→Z、Z→R1 等微步骤。

03 易错点

错误说法单内部总线能同拍把任意两个寄存器同时驱动到同一 ALU 输入。

自检：能否随意让两个不同寄存器同时把数值输出到单总线?

不能，通常会造成驱动冲突，除非题图明确有特殊机制。

CPU 与流水线

流水线：相关、冒险与转发　相关不必停顿；能否转发、何时可读须看时序。　个人错题　易错　近年考查　要点

核心结论

数据相关是依赖关系；是否形成冒险及停顿取决于流水线阶段、写回/读寄存器时序与转发路径。

别混淆相关不必停顿；能否转发、何时可读须看时序。

详解

01 解析

1. 标出前一条指令什么时候产生结果，以及后一条什么时候真正需要该操作数。
2. 有转发时画出题设允许的路径；没有对应路径就必须等写回，不能因存在数据相关就直接断言固定停几拍。
3. 无转发且寄存器同拍先写后读时，可在 WB 当拍完成依赖指令的 ID；若不允许同拍读写，还须再等一拍。

补充原理与条件

ALU 结果可提前转发；典型五段流水线的紧邻 load-use 仍可能停一拍。无转发题必须等结果可读，不能擅自套转发时序。

02 例子

同一周期能否先写后读寄存器文件，必须看题设。

03 易错点

错误说法相关指令必停顿；有转发就不会停顿。

考点联系

转发不是消除所有数据冒险的万能条件。

变式

自编模拟变式 · 有转发仍要等的情况

经典五级流水中，load 在 MEM 末才产生数据，下一条 ADD 在 EX 初需要它；允许 MEM/WB→EX 转发。紧邻执行能零停顿吗?

答案与解析

不能。原排程中 ADD 的 EX 初早于 load 的 MEM 末，通常须停 1 拍后再转发。

关键区别：有无通路与数据是否来得及到达是两个条件；无转发时还须等寄存器可读。

自检：有转发能否一律判定 CPI=1?

不能，还可能有 load-use、控制冒险和结构冒险。

硬布线与微程序：微指令不等于机器指令　机器指令在主存；微指令通常在控制存储器。冷门　易混　近年考查　　要点

核心结论

微程序控制器用控制存储器中的微指令生成控制信号；一条机器指令通常对应一个微程序。

别混淆机器指令在主存；微指令通常在控制存储器。

详解

01 解析

机器指令描述软件层操作，微指令描述一个微周期的控制操作。水平型允许较多并行控制，垂直型编码更紧凑、译码更多。

02 例子

取指微程序可由多条微指令组成，并由多种机器指令共享。

03 易错点

错误说法微程序存放在普通主存，微指令由程序员直接编写执行。

自检：微指令地址与 PC 保存的地址是一回事吗?

不是，通常由微地址寄存器管理控制存储器地址。

CPU 时间：指令数、CPI 与周期　同时比较指令数、CPI、主频，不能只看其中一个。　记忆概念　易错　易混　要点

核心结论

CPU 时间=指令数×平均 CPI×时钟周期=指令数×平均 CPI/主频。

别混淆同时比较指令数、CPI、主频，不能只看其中一个。

详解

01 解析

$T_{\text{CPU}} = \frac{I \cdot \text{CPI}}{f}$

I 为执行指令数，平均 CPI 按指令比例加权，f 为主频。

比较时要同时考虑三个因子。总线时钟频率乘每拍传输次数才是传输频率，带宽再乘每次传送字节数；MT/s 与 MHz 不能无条件互换。

02 例子

同一程序若机器 A 的 CPI 降低但时钟周期变大，CPU 时间不一定缩短。

03 易错点

错误说法CPI 小的机器运行所有程序都更快；频率等于数据传输率。

自检：平均 CPI 应按什么加权?

按各类指令实际执行数量或比例加权，而不是按指令类别简单平均。

指令周期、机器周期与中断周期　时钟周期、机器周期、指令周期不能混为一谈。　记忆概念　易错　易混　要点

核心结论

时钟周期是基本时间单位；一条指令可历经取指、间址、执行等阶段，必要时响应中断。机器周期的定义取决于题设模型。

别混淆时钟周期、机器周期、指令周期不能混为一谈。

详解

01 解析

间接寻址可能增加访存阶段；中断响应含保存断点及转向服务入口等操作。不要把存储周期、机器周期、指令周期都视为固定相等。

02 例子

多周期 CPU 一条访存指令可以由多个时钟周期组成。

03 易错点

错误说法一个时钟周期必执行一条指令；中断周期就是执行一次普通取指。

自检：中断服务程序的每条指令是否也要正常取指执行?

需要，中断响应阶段与服务程序执行仍应分别计入。

流水线性能：延迟、吞吐率与加速比　流水主要提高吞吐，不保证单条指令延迟缩短。　记忆概念　易错　易混　要点

核心结论

流水主要提高多条指令的吞吐；单条指令仍要经过各级。理想等长 k 段、每段 τ，n 条连续执行为 $(k+n-1)\tau$。

别混淆流水主要提高吞吐，不保证单条指令延迟缩短。

详解

01 解析

$T=(k+n-1)\tau$

理想等长 k 段流水，n 条指令连续执行，无停顿。

$S=$

与相同 k 段逐段非流水执行比较；n 很大时才趋近 k。

1. 先求一次流水节拍：若各段不等长，通常由最长段加必要锁存开销决定，不能直接用各段平均值。
2. 理想等长 k 段时，首条需 kτ，后续每条再用 τ，n 条合计 $(k+n-1)\tau$。

3. 加速比要拿同一任务的非流水 $nk\tau$ 来除；n=1 时为 1，n 很大才趋近 k。停顿周期另加到分母。

补充原理与条件

非流水同模型为 $nk\tau$，理想加速比 $\frac{nk}{k+n-1}$，n 大时趋 k。锁存延迟、最长阶段、停顿和填充/排空降低实际收益。

02 例子

k=5、n=1 时理想加速比为 1，不是 5。

03 易错点

错误说法k 级流水线任何情况下都使一条指令延迟变为原来的 1/k。

自检：吞吐率提高是否等于单条指令响应时间一定缩短?

不等于，两种指标不同。

指令阶段名称：取指、间址、执行、中断　取指取的是指令；间址与中断阶段按条件出现。　　记忆概念　易混　图解　　要点

核心结论

教材指令周期模型常分取指周期、间址周期、执行周期、中断周期；是否出现间址与中断由寻址方式和中断条件决定。

别混淆取指取的是指令；间址与中断阶段按条件出现。

详解

01 解析

取指获得指令；间址阶段用于获得有效地址（按题设间接寻址模型）；执行完成操作；中断周期完成中断响应的硬件动作。中断服务程序还包含多条正常执行的指令。

02 例子

直接寻址加法指令通常无须独立间址周期；无可响应中断时不进入中断周期。

03 易错点

错误说法每条指令都依次且必定经历四个周期。

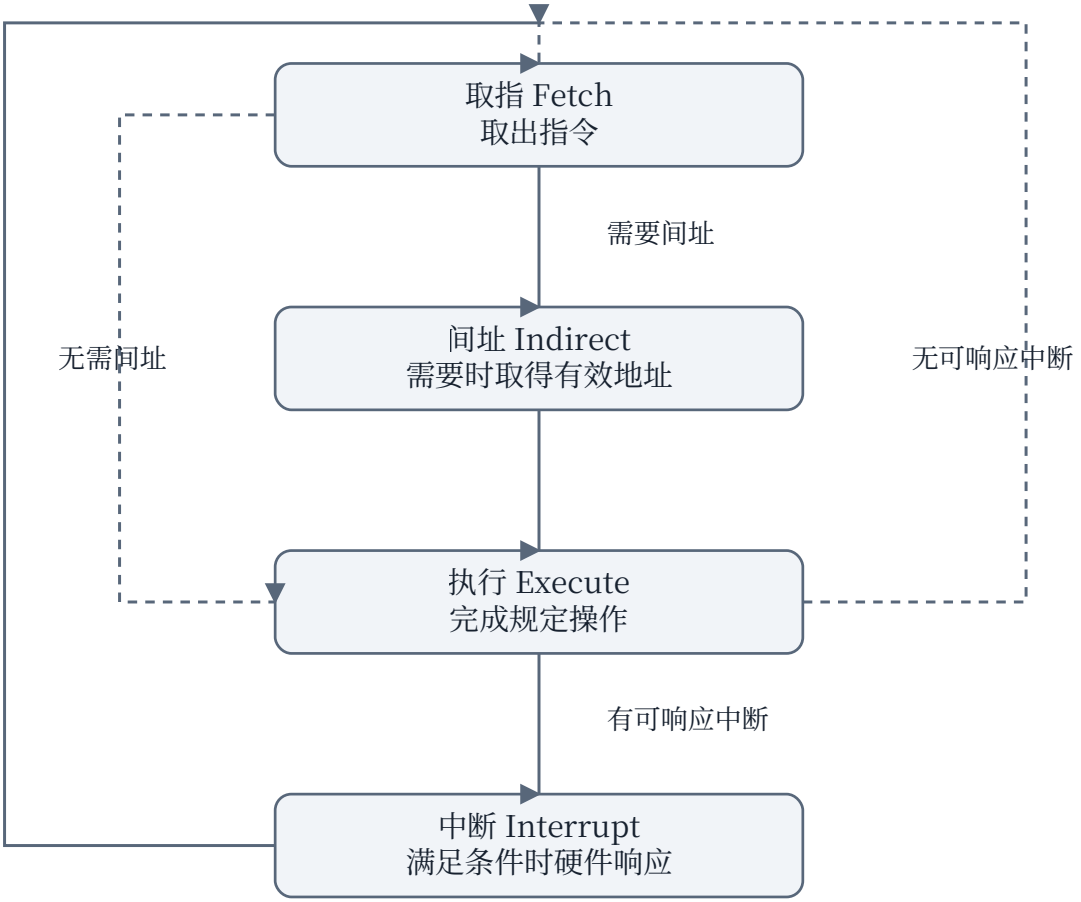
04 图表

要点对照

阶段	英文常见写法	任务
取指周期	Fetch	取指令并更新 PC 等
间址周期	Indirect	按题设取得有效地址

阶段	英文常见写法	任务
执行周期	Execute	完成指令规定操作
中断周期	Interrupt	保存断点、转向服务入口等

指令周期：有些阶段按条件出现



响应后转入服务程序，仍逐条取指执行

图示为教材指令周期模型；并非每条指令都经历间址和中断阶段。

取指后可选间址，再执行。执行完成后若有满足响应条件的中断进入中断周期，否则取下一条指令。中断响应转入服务程序，服务程序也正常逐条执行。单独查看图片

自检：取指周期和取操作数是否一回事?

不是，取指取指令，取操作数通常属于执行所需的数据访问。

五级流水阶段：IF、ID、EX、MEM、WB load 在 MEM 读数据；store 通常无需 WB 写寄存器。 记忆概念 易混 图解 要点

核心结论

经典五级模型：IF 取指、ID 译码/读寄存器、EX 运算/地址计算、MEM 数据访存、WB 写回。

别混淆load 在 MEM 读数据；store 通常无需 WB 写寄存器。

详解

01 解析

这是教学模型，具体阶段划分以题图为准。算术指令可能仅经过 MEM 而不实际访存；store 在 MEM 写存储器且通常不写回寄存器。load-use 即使有常见转发也可能需要停顿，取决于结果可用时刻。

02 例子

load 在 EX 算地址、MEM 读数据、WB 写目的寄存器。

03 易错点

错误说法MEM 阶段一定读数据存储器；每种流水线都固定五级。

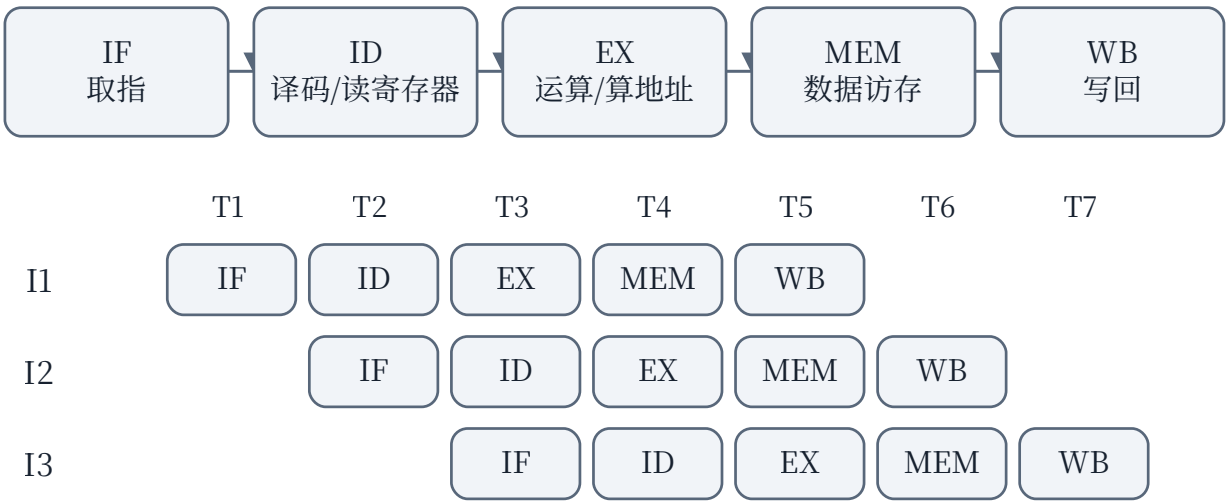
04 图表

要点对照

缩写	英文	中文任务
IF	Instruction Fetch	取指
ID	Instruction Decode	译码、读寄存器
EX	Execute	运算、计算地址
MEM	Memory Access	按指令需要访问数据存储器
WB	Write Back	按需要写回寄存器

五级流水：阶段名称与时间重叠

经典五级教学模型：下表按无停顿、每级 1 拍示意



同一拍各指令可处于不同阶段；并非各级都实际访存或写寄存器。

load：EX 算地址 → MEM 读数据 → WB 写回；store 通常不写回寄存器。

上方列出 IF 取指、ID 译码读寄存器、EX 运算算地址、MEM 数据访存、WB 写回。下方以三条无停顿指令展示各阶段逐拍重叠；实际数据相关和资源冲突可能引入停顿。单独查看图片

自检：store 是否通常在 WB 写目的寄存器?

不需要；其主要结果是 MEM 阶段写存储器。

总线

总线仲裁：链式、计数查询与独立请求　固定优先级可能饥饿；仲裁规则决定谁先使用总线。　　记忆概念　易错　易混　　要点

核心结论

集中仲裁常见链式查询、计数器定时查询、独立请求；在控制线数量、优先级灵活性和响应速度间折中。

别混淆固定优先级可能饥饿；仲裁规则决定谁先使用总线。

详解

01 解析

链式次序常赋予固定优先级且对链路故障敏感；计数器起点可影响公平；独立请求可更灵活但控制线更多。不能仅凭名称判断所有具体策略。

02 例子

固定高优先级设备不断请求可能使低优先级长期等待。

03 易错点

错误说法优先级高的设备一定不会让其他设备饥饿，链式查询对故障完全不敏感。

自检：DMA 和 CPU 同时需总线时，只靠谁先到就总能决定吗?

不一定，应按总线仲裁规则。

同步、异步与分离事务：时钟不是带宽　带宽需扣协议等待；64 位一次仅为 8 B。　　记忆概念　易错　易混　　要点

核心结论

同步总线依赖共同节拍；异步常用握手；分离事务将请求和响应分开，释放等待期间的总线。

别混淆带宽需扣协议等待；64 位一次仅为 8 B。

详解

01 解析

峰值带宽通常按每秒有效传输次数×每次字节数。地址阶段、等待周期、仲裁和协议开销会降低实际吞吐；时钟频率与双倍/多倍传输率要分开。

02 例子

首拍发地址、后几拍等存储器时，不能把每拍都算成数据传输。

03 易错点

错误说法总线频率×位宽在任何事务模型下都等于有效数据吞吐率。

考点联系

算有效带宽时，突发中的地址和等待周期也占时间。

变式

自编模拟变式 · 峰值传输率与突发有效带宽

自编参数：总线 100 MHz、64 位，每拍最多传两次；一次事务共 8 拍，只有后 4 拍传数据。该事务有效带宽是多少?

答案与解析

后 4 拍共传 8×8=64 B；总时间 8/100 MHz=80 ns，有效带宽 0.8 GB/s。

关键区别：按每拍有效数据算出的峰值为 1.6 GB/s；将地址及等待占用纳入事务时间后才是本题结果。

自检：64 位总线的一次传送等于 64 字节吗?

不是，64 位是 8 字节。

中断与 I/O

中断总耗时：响应与服务分别计入 先画时间区间，响应与恢复既不漏算也不重复算。 个人错题 易错 近年考查 要点

核心结论

若题目问一次中断总开销，应按题设计入响应、现场保存、服务、恢复和返回，避免遗漏或重复。

别混淆先画时间区间，响应与恢复既不漏算也不重复算。

详解

01 解析

1. 把题目的一次中断过程列成不重叠区间：响应、保存、服务、恢复、返回。
2. 逐个核对哪些已含在给定“服务时间”中，只把未包含的响应等区间另加一次。
3. 每秒中断次数乘一次总周期，再除每秒 CPU 周期数；若给主频 MHz，先换成每秒周期。

补充原理与条件

硬件完成哪些保存操作、软件完成哪些操作由体系结构决定。中断屏蔽与中断优先级也不同，不能只按服务程序条数估算。

02 例子

题目另给“响应需要若干周期”，若不包含在服务时间中，就要单独计入。

03 易错点

错误说法只计算中断服务程序的执行时间。

自检：响应时间已包含在总服务时间中，是否还应再次相加?

不应，先确认各时间区间的定义。

DMA 与中断方式：谁搬数据 DMA 减轻搬运负担，但仍需 CPU 初始化和结束处理。 冷门 易混 近年考查 要点

核心结论

中断方式通常由 CPU 指令搬运数据；DMA 控制器承担成块数据传送，CPU 负责初始化及结束处理。

别混淆DMA 减轻搬运负担，但仍需 CPU 初始化和结束处理。

详解

01 解析

1. 先问每个数据字由谁搬：中断方式由 CPU 执行读写指令；DMA 方式由控制器完成设备与主存之间传送。
2. CPU 仍须设置方向、主存地址和长度，结束或异常时也通常要处理通知。
3. 再问 CPU 是否受影响：DMA 会竞争访存或总线周期，因此“CPU 不逐字搬运”不等于“CPU 全程无开销”。

补充原理与条件

DMA 可争用总线或访存周期，并在完成/异常时中断 CPU。降低 CPU 搬运负担不等于传送过程对 CPU 没有影响。

02 例子

CPU 设置缓冲区地址、长度和方向后，DMA 按配置传送。

03 易错点

错误说法DMA 完全不需要 CPU，也不使用中断。

自检：DMA 完成后能否请求中断?

可以。

中断响应优先级与处理优先级　硬件响应次序与允许嵌套次序可不同。　　冷门　易混　近年考查　　要点

核心结论

响应优先级决定同时请求时先响应谁；处理优先级决定服务程序执行时允许谁嵌套，由屏蔽设置等控制。

别混淆硬件响应次序与允许嵌套次序可不同。

详解

01 解析

两种次序可以不同。现场保存、断点保存和开/关中断的时机必须按题设区分，不能只比较中断号大小。

02 例子

某源硬件响应优先级高，但服务时可被另一源打断，说明处理次序另有设置。

03 易错点

错误说法中断屏蔽字决定硬件同时请求时的全部响应次序。

自检：改中断屏蔽字是否一定改变硬件排队器顺序?

不一定，屏蔽与排队选择是不同机制。

I/O 接口、端口与统一/独立编址　统一编址占存储地址范围；状态、控制、数据端口分清。　记忆概念　易错　易混　要点

核心结论

接口常有数据、状态、控制寄存器；统一编址将 I/O 端口置入存储地址空间，独立编址用单独端口空间和访问指令。

别混淆统一编址占存储地址范围；状态、控制、数据端口分清。

详解

01 解析

查询方式检查状态，控制命令配置设备，数据寄存器承担实际传送。设备物理执行由控制器协作，CPU 不能把所有端口都当普通可缓存内存访问。

02 例子

统一编址下，普通访存指令可按规定访问 I/O 区域，但会占地址范围。

03 易错点

错误说法设备接口只有数据寄存器；统一编址访问设备不消耗任何地址空间。

自检：设备状态与要传输的数据是否通常属于同一个逻辑字段?

不是，应区分状态/控制与数据。

DMA 传送模式：周期挪用、成组与透明　周期挪用是总线使用方式，不是 CPU 指令代搬数据。　记忆概念　易错　易混　要点

核心结论

DMA 直接控制设备与主存传送；成组方式连续占用总线，周期挪用夺取部分访存周期，透明方式尽量利用 CPU 不需总线的时段。

别混淆周期挪用是总线使用方式，不是 CPU 指令代搬数据。

详解

01 解析

各模式在 CPU 停顿、控制开销与传输速度间折中。CPU 初始化 DMA 和处理中断仍有开销；DMA 请求/总线授权不等于中断响应时机。

02 例子

周期挪用让 CPU 暂失一个或一些访存周期，而不是必然中止整个程序到整块全部传完。

03 易错点

错误说法DMA 的每次传输都必须由 CPU 指令读取并再写入数据。

自检：透明 DMA 是否意味着绝对无控制开销?

不意味着，只是尽量避免与 CPU 总线需求冲突。

中断向量、向量地址与服务入口 向量地址是存向量的位置，不必等于服务入口地址。 记忆概念 易错 易混 要点

核心结论

向量通常含服务入口等信息；向量地址是存放该向量的表项位置；取表项后才得到入口。

别混淆向量地址是存向量的位置，不必等于服务入口地址。

详解

01 解析

1. 先看给的是向量本身、向量表地址，还是中断服务入口，给三者分别命名。
2. 若地址 100 处存放入口 200，CPU 先通过 100 取出 200，再跳到 200 执行。
3. 因此不能把“向量地址=100”直接写成“服务程序从 100 开始”；题设若向量表项还含其他字段，再按表项格式读取。

补充原理与条件

不同架构可能直接编码入口或有不同表结构，以题设为准。硬件识别请求、形成向量地址、保存断点与跳转是不同动作；软件继续保存通用现场。

02 例子

向量地址为 100，M[100]=200，则服务入口可为 200，不是 100。

03 易错点

错误说法中断向量地址就是中断服务程序的入口地址，二者不能有区别。

自检：中断响应是否等同于把 PC 加上固定指令长度?

不等同，要转向相应服务入口。

中断过程名称：请求、判优、响应、服务、返回　请求到达不等于已响应，硬件响应也不等于整段服务程序。　　记忆概念　易混　图解　　要点

核心结论

中断请求→判优及满足响应条件→硬件响应→中断服务程序→中断返回；各步骤的硬件/软件分工以体系结构和题设为准。

别混淆请求到达不等于已响应，硬件响应也不等于整段服务程序。

详解

01 解析

硬件响应通常含保存断点及转向入口等；服务程序常保存/恢复更多现场并处理设备。中断返回恢复被打断执行所需状态。屏蔽中断与关中断的粒度不同；响应优先级与允许嵌套的处理优先级也不同。

02 例子

设备完成 I/O 提出请求，不代表 CPU 已在同一瞬间开始执行服务程序。

03 易错点

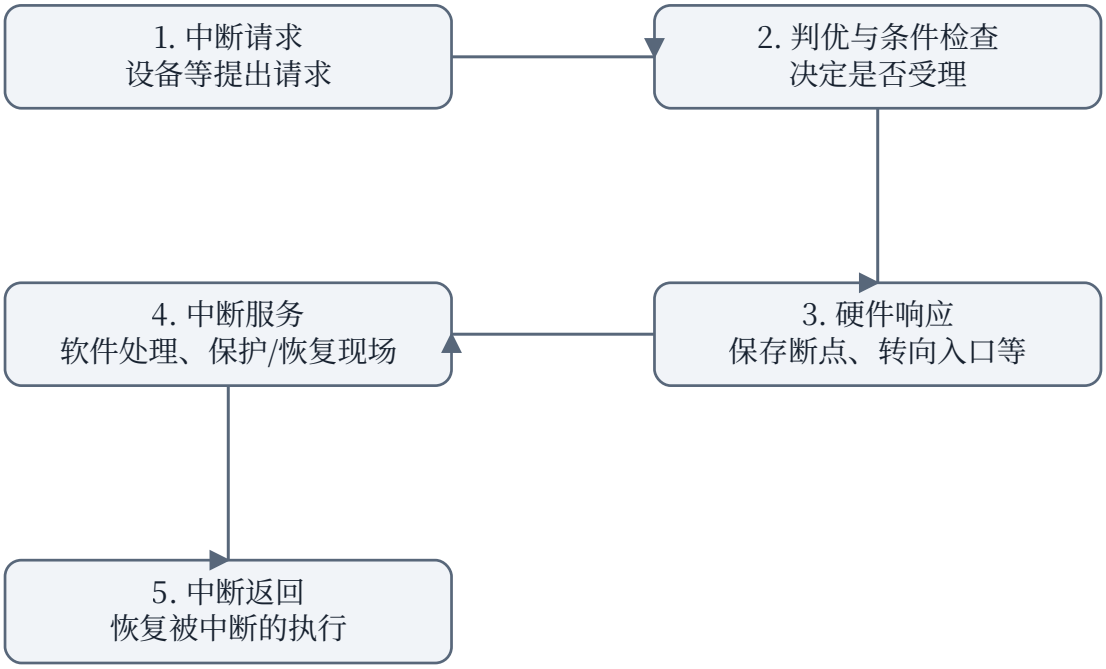
错误说法中断响应就是运行完整中断服务程序；关中断等于 CPU 停止运行。

04 图表

要点对照

名称	主要动作	区别
中断请求	设备等提出请求	尚未必被受理
中断判优	多个请求中确定候选	还要满足响应条件
中断响应	硬件完成转向所需动作	不同于整段服务程序
中断服务	执行处理程序	含软件现场保护等
中断返回	恢复执行状态与断点	常用专用返回指令

中断处理：请求 → 响应 → 服务 → 返回



请求到达 ≠ 已响应；响应 ≠ 完整服务程序；硬件/软件分工以体系结构为准。

设备请求后经判优并满足响应条件才进入硬件响应。硬件响应转向入口；服务程序处理事务及现场，最后中断返回。单独查看图片

自检：中断向量与中断服务程序是否相同?

不同；向量提供入口定位信息，服务程序是执行处理的指令序列。

DMA 阶段名称：预处理、数据传送、后处理　DMA 三阶段：CPU 配置→控制器搬运→CPU 后处理。　　记忆概念　易混　图解　　要点

核心结论

典型 DMA 过程分预处理、数据传送、后处理；CPU 负责启动配置及完成后的处理，DMA 控制器承担数据块传送。

别混淆DMA 三阶段：CPU 配置→控制器搬运→CPU 后处理。

详解

01 解析

预处理设置方向、地址、计数等；数据传送阶段按总线授权进行；完成时通常中断 CPU。周期挪用、成组传送、透明 DMA 是传送方式，不是另外三个先后阶段。

02 例子

CPU 配好一块数据的 DMA 后可执行其他工作，结束时再处理完成通知。

03 易错点

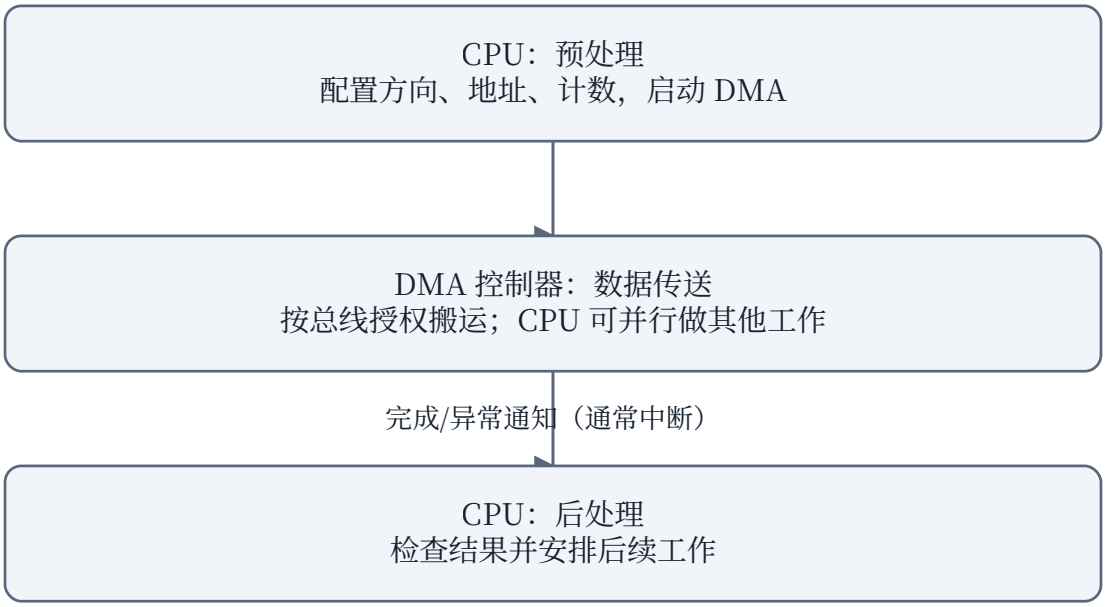
错误说法DMA 全过程不需要 CPU；每搬一个字都要 CPU 执行访存指令。

04 图表

要点对照

阶段	负责者	主要任务
预处理	CPU	配置 DMA、启动 I/O
数据传送	DMA 控制器等硬件	按总线授权搬运数据
后处理	CPU	处理完成/异常通知并安排后续

DMA：CPU 负责前后，控制器负责搬运



周期挪用、成组传送、透明 DMA 是传送方式，不是这三个先后阶段。

CPU 先配置并启动 DMA，DMA 控制器按总线授权搬运数据，完成或异常时通常中断 CPU 进行后处理。单独查看图片

自检：周期挪用是 DMA 的哪类名称?

传送方式，不是预处理/后处理这样的阶段。

408 · OS

操作系统

57 条 · 7 个章节

操作系统概述

OS 特征：并发、共享、虚拟与异步　互斥共享不是永久独占；异步不等于没有响应。　　记忆概念　易错　易混　　要点

核心结论

并发为多任务一段时间内交替推进；共享为多个活动使用资源；虚拟把物理资源抽象成逻辑资源；异步体现推进时机不可预知。

别混淆互斥共享不是永久独占；异步不等于没有响应。

详解

01 解析

互斥共享指同一时刻仅允许一个使用者，不是永久独占。虚拟 CPU 靠分时，虚拟内存靠地址映射等；这些术语要结合具体机制解释。

02 例子

打印机可通过互斥或 SPOOLing 供多个进程使用，并不变成多台物理打印机。

03 易错点

错误说法异步表示 OS 不响应进程，互斥共享等于资源不允许多人使用。

自检：并发程序的推进速度和顺序是否一定固定?

不固定，受调度、I/O 和其他活动影响。

批处理、分时与实时：目标不同　实时重在满足期限，不只是平均速度快。　　记忆概念　易错　易混　　要点

核心结论

批处理重视吞吐与资源利用；分时重视交互响应；实时系统重视规定时间内完成任务，区分硬/软实时。

别混淆实时重在满足期限，不只是平均速度快。

详解

01 解析

多道程序能在某任务等待 I/O 时运行其他任务。时间片越短不一定越好，切换开销会上升；硬实时截止期违反可能不可接受。

02 例子

交互终端轮流获得 CPU 是分时；工控急迫任务须在期限内完成是实时要求。

03 易错点

错误说法分时系统的目标是严格满足所有事件的最迟截止时刻。

自检：实时是否等于单纯执行速度最快?

不等于，重点是可保证的截止时间。

宏内核与微内核：服务放在哪里　微内核强调服务隔离，并非天然性能更高。　　记忆概念　易错　易混　　要点

核心结论

宏内核将较多核心服务放内核空间；微内核保留较小核心，部分服务移到用户空间通过通信协作。

别混淆微内核强调服务隔离，并非天然性能更高。

详解

01 解析

微内核有助隔离和扩展，但跨服务通信/切换带来代价；宏内核可以使用模块化设计，结构风格不等于有没有模块。虚拟机监控器也不等于普通微内核。

02 例子

文件服务放用户进程不代表它不需要内核提供通信与保护机制。

03 易错点

错误说法微内核完全没有内核态服务；宏内核没有模块化可能。

自检：微内核一定比宏内核运行更快吗?

不能无条件推出，取决于通信、实现及工作负载。

引导、内核初始化与应用启动 引导加载现有系统，不等于重新格式化文件系统。 记忆概念 易错 易混 要点

核心结论

启动由固件和引导程序等逐步加载/定位内核，再初始化必要硬件、内存与进程，随后启动服务和应用。

别混淆引导加载现有系统，不等于重新格式化文件系统。

详解

01 解析

具体 BIOS/UEFI 流程不同；引导程序属于启动链的一环，不应混同整个 OS。内核建立中断/内存等基础后才能支持用户程序执行。

02 例子

内核从持久存储加载到 RAM，不是 RAM 自己断电保存了整个系统。

03 易错点

错误说法OS 的全部代码在断电后仍留在 RAM，开机直接从普通应用入口执行。

自检：OS 启动时一定重新创建磁盘文件系统和所有 inode 吗?

不一定，现有文件系统元数据通常读取使用，格式化是另一操作。

进程与线程

并发、并行与多任务 并发看一段时间，并行看同一时刻；单处理机也能并发。 个人错题 易混 记忆概念 要点

核心结论

并发表示一段时间内多个任务都在推进；并行表示同一时刻同时执行。单核可以靠切换实现多任务并发。

别混淆并发看一段时间，并行看同一时刻；单处理机也能并发。

详解

01 解析

在单处理机、无同时多线程等教学模型中，同一时刻只能执行一个任务，CPU 与 I/O 仍可重叠。物理核心数、逻辑处理器数与物理芯片数不能直接等同。

02 例子

单核轮流运行 A、B，两者都有进展但不必在同一时刻执行。

03 易错点

错误说法多任务操作系统必须有多个 CPU。

自检：多任务是否能推出多 CPU?

不能。

进程与线程：共享资源和独立上下文 共享地址空间，不等于共享每个线程的栈与寄存器。 易混 近年考查 要点

核心结论

同进程线程通常共享地址空间和文件等资源，各有独立寄存器上下文和栈。

别混淆共享地址空间，不等于共享每个线程的栈与寄存器。

详解

01 解析

用户线程/内核线程的管理方式另需区分。一个进程可含多个线程，线程局部变量与共享变量不能仅靠变量名判断。

02 例子

两个线程各自栈上的 x 不一定是同一个对象；访问同一全局 x 才可能竞争。

03 易错点

错误说法同一进程的线程共享所有寄存器和栈。

考点联系

区分进程共享资源和线程各自的执行上下文。

变式

自编模拟变式 · 共享 fd 是否共享线程栈

同一进程的两个线程使用同一个已打开的 fd。它们是否因此共用同一份调用栈?

答案与解析

不共用调用栈。fd 属于进程可共享资源；两个线程仍有各自的栈和寄存器上下文。

关键区别：共享同一地址空间意味着可访问某些相同对象，不等于执行现场合并成一份。

自检：同进程线程是否通常共享堆?

通常共享地址空间中的堆，但各有独立栈。

就绪、阻塞与运行：唤醒不是立即运行　唤醒先就绪；只有获得 CPU 才运行。　易错　易混　要点

核心结论

典型状态转换：运行→阻塞为等待事件；阻塞→就绪为事件完成；就绪→运行由调度分派。

别混淆唤醒先就绪；只有获得 CPU 才运行。

详解

01 解析

1. 判断进程缺什么：仅缺 CPU 是就绪；还在等 I/O、信号量等事件是阻塞。
2. I/O 完成只是补足等待条件，所以阻塞→就绪；只有调度器分派 CPU 后才就绪→运行。
3. 时间片用尽但执行条件仍满足，走运行→就绪；不要把所有离开 CPU 的进程都判为阻塞。

补充原理与条件

运行→就绪可由时间片用尽或被抢占造成。阻塞进程缺少执行条件，不能靠给它 CPU 就解决等待。

02 例子

I/O 完成后先进入就绪队列，可能仍需等待 CPU。

03 易错点

错误说法I/O 完成使进程直接从阻塞变为运行。

自检：时间片用尽后进程通常进入什么状态?

就绪态。

fork：独立地址空间与共享文件描述　普通内存写入独立；继承的打开文件描述可共享。　冷门　易混　要点

核心结论

父子进程有各自逻辑地址空间；写时复制可暂时共享物理页，写入时再复制。继承的文件描述符可引用同一打开文件描述。

别混淆普通内存写入独立；继承的打开文件描述可共享。

详解

01 解析

1. fork 后父子从同一调用点继续，但返回值不同，普通变量位于各自的逻辑地址空间。
2. 对子进程局部变量赋值通常不改父进程的变量；写时复制共享的是暂未修改的物理页，不是让变量永久共享。

3. 继承的 fd 可指向同一打开文件描述，所以读写偏移可能联动；内存变量独立与文件偏移共享可以同时成立。

补充原理与条件

父子都从 fork 返回处继续，返回值不同。普通内存修改不会自动修改对方的变量；共享映射是另一个机制。

02 例子

同一继承文件的偏移可能共享，但局部变量 x 的写入通常各自独立。

03 易错点

错误说法fork 后父子进程的所有变量永久共享。

自检：父进程写普通变量 x，子进程能否必然看到更新?

不能。

PCB 与上下文：控制信息不等于整个内存副本　PCB 存控制信息与上下文，不存整个进程的代码数据副本。　　记忆概念　易错　易混　　要点

核心结论

PCB 记录进程标识、状态、调度、资源和 CPU 上下文等控制信息，是系统管理进程的基础；代码与数据另占地址空间。

别混淆PCB 存控制信息与上下文，不存整个进程的代码数据副本。

详解

01 解析

1. 上下文切换保存的是继续执行所需的寄存器和管理状态，先列 PC、寄存器、状态字等。
2. 地址空间一般由页表等结构保存并切换映射，不是把整段进程内存复制到 PCB。
3. 因此“进程切换复制全部内存”应判错；换入换出页面是另一种内存管理行为。

补充原理与条件

进程切换需保存/恢复寄存器及必要管理状态，不是复制全部内存。程序为静态代码，进程为运行实例，同一程序可对应多个进程。

02 例子

多个进程可执行同一可执行文件，但有各自状态与逻辑地址空间。

03 易错点

错误说法PCB 保存进程全部代码、堆和文件内容。

自检：进程被换下 CPU 后，其 PC 值是否应丢弃?

不能，须保存在上下文中以恢复执行。

创建、exec 与终止：程序替换不等于新建进程 fork 建新进程，exec 换程序映像，PID 通常不变。 记忆概念 易错 易混 要点

核心结论

创建产生新进程控制结构；典型 exec 在现有进程中替换程序映像；终止回收资源并处理关联关系。

别混淆fork 建新进程，exec 换程序映像，PID 通常不变。

详解

01 解析

1. fork 创建新的进程执行实例；exec 是在已有进程内替换程序映像，两步不能混为一谈。
2. 典型 Unix 中 exec 成功后仍用原 PID，但执行新的代码和地址空间内容。
3. 看到“运行新程序”先看调用过程：只有 exec，不能据此额外数出一个新进程。

补充原理与条件

不同系统有不同父子终止策略，不能把级联终止作为所有系统必然行为。子进程继承与共享资源也要按规则区分。

02 例子

fork 后子进程可 exec 新程序，但典型 Unix 中其 PID 不因 exec 本身变化。

03 易错点

错误说法exec 必然新建一个 PID；父进程终止必然终止每个子进程。

自检：exec 是否与 fork 承担同样职责?

不是，fork 创建运行实例，exec 替换程序映像。

用户级线程、内核级线程与映射模型 阻塞与多核并行能力取决于用户/内核线程映射。 记忆概念 易错 易混 要点

核心结论

多对一用户线程依赖一个内核执行实体，阻塞和并行受其限制；一对一由内核分别调度；多对多结合两层映射。

别混淆阻塞与多核并行能力取决于用户/内核线程映射。

详解

01 解析

1. 先确定映射模型：多个用户线程映到一个内核线程，还是映到多个内核线程。
2. 多对一时内核只看见一个执行实体；其中一个线程发起阻塞系统调用，可能连同该内核实体一起阻塞。
3. 只有多个可调度内核执行实体，才有机会在多核上同时运行；用户线程数量多不等于可并行核数多。

补充原理与条件

用户级切换可低开销，但不是所有阻塞都等价于整个进程被阻塞，须看是否陷入内核及映射方式。多核并行需要多个可并行内核执行实体。

02 例子

多对一模型若一个线程进行阻塞系统调用，可影响同进程其他用户线程运行。

03 易错点

错误说法所有用户线程阻塞都使整个进程阻塞；所有模型都能多核并行。

自检：纯用户态切换一定要求 OS 为每个用户线程维护独立调度项吗?

不一定，多对一模型可由用户线程库管理。

进程间通信：共享内存、消息与管道　相同虚拟地址不保证共享；共享内存也仍需同步。记忆概念　易错　易混　　要点

核心结论

IPC 可用共享内存、消息传递、管道等。共享内存让进程映射同一对象，但仍需同步；管道通常按字节流或给定消息规则顺序通信。

别混淆相同虚拟地址不保证共享；共享内存也仍需同步。

详解

01 解析

相同虚拟地址可映不同物理页。普通匿名管道与命名管道在使用范围上不同，读写端关闭、空/满等待影响行为。通信通道不自动提供所有并发互斥。

02 例子

两个进程共享一页存数据，生产消费双方仍需约定有效数据与访问顺序。

03 易错点

错误说法独立进程只要虚拟地址相同就自动共享变量；匿名管道可任意随机定位。

自检：共享内存高速是否意味着不用锁或信号量?

不是，仍可能有数据竞争和顺序要求。

进程五状态：新建、就绪、运行、阻塞、终止 就绪只缺 CPU；阻塞等事件，唤醒后先回就绪。 记忆概念 易混 图解 要点

核心结论

五状态教学模型为 New、Ready、Running、Blocked/Waiting、Terminated；就绪仅缺处理机，阻塞还在等待事件。

别混淆就绪只缺 CPU；阻塞等事件，唤醒后先回就绪。

详解

01 解析

1. 先看事件是否已经完成：等待事件未到，即使 CPU 空闲，该进程也不具备运行条件。
2. I/O 完成后，阻塞→就绪，说明已具备运行条件；调度分派后才就绪→运行。
3. 时间片耗尽是运行→就绪；主动等待 I/O 才是运行→阻塞。用“是否只缺 CPU”逐条验证状态图。

补充原理与条件

新建完成必要准备后进入就绪；就绪经分派进入运行；运行可因抢占回就绪、因等待事件入阻塞、因结束入终止。阻塞被唤醒先到就绪。具体系统还有僵尸等实现状态，不与教学五状态直接等同。

02 例子

I/O 完成：阻塞→就绪；时间片耗尽：运行→就绪。

03 易错点

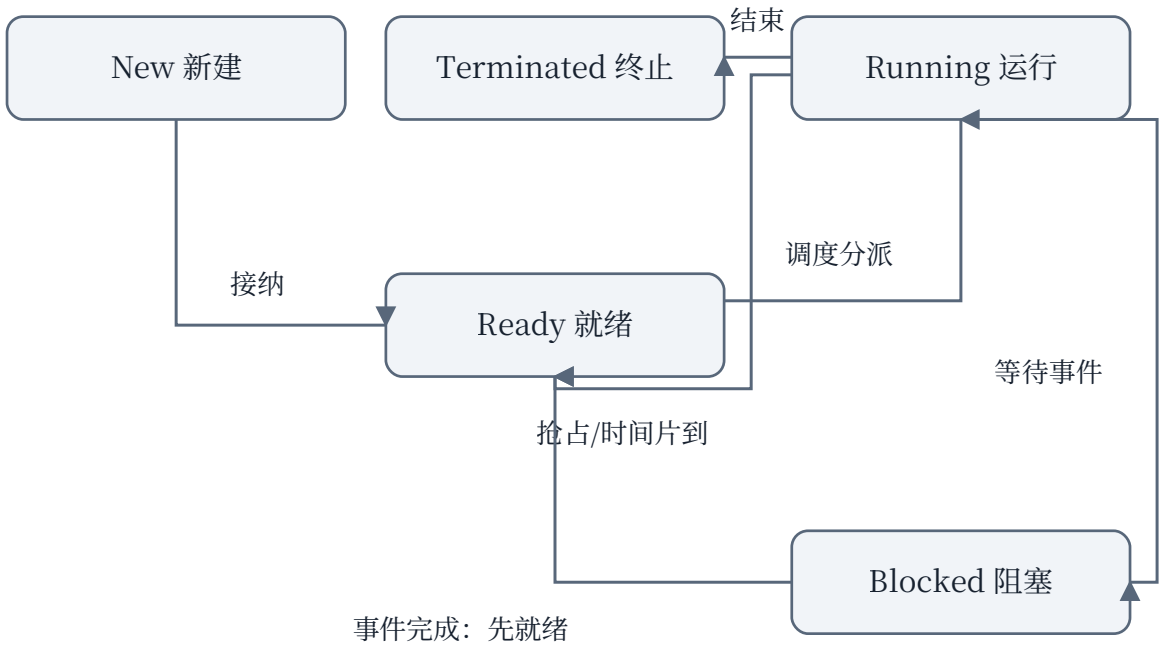
错误说法创建了 PCB 就必然已运行；阻塞进程仅缺 CPU。

04 图表

要点对照

状态	英文	含义
新建	New	正创建并准备接纳
就绪	Ready	具备执行条件，等待 CPU
运行	Running	正在处理机上执行
阻塞/等待	Blocked / Waiting	等待 I/O、资源或事件
终止	Terminated / Exit	执行结束，进入清理过程

进程五状态：箭头由事件决定



基本五状态模型：就绪只缺 CPU；阻塞还缺事件，不存在“唤醒即运行”。

新建被接纳后就绪，就绪由调度分派进入运行。运行被抢占回就绪，等待事件进入阻塞，结束进入终止。阻塞事件完成后先回就绪。单独查看图片

参考资料

参考：OSTEP：进程

自检：基本模型中为什么没有直接就绪→阻塞?

就绪进程尚未执行，不能主动发起阻塞操作；需先获 CPU。

挂起模型：静止就绪、静止阻塞与激活　静止阻塞事件完成→静止就绪，不自动调入或运行。　　记忆概念　易混　图解　　要点

核心结论

常见七状态模型增加就绪挂起与阻塞挂起，教材也称静止就绪、静止阻塞；挂起描述调出内存等管理因素，阻塞描述等待事件。

别混淆静止阻塞事件完成→静止就绪，不自动调入或运行。

详解

01 解析

活动就绪/活动阻塞通常在内存；静止就绪/静止阻塞通常已调出。静止阻塞等待的事件完成后可变静止就绪，并不自动换回内存。激活后仍要根据事件是否完成判断到就绪还是阻塞。

02 例子

某阻塞挂起进程 I/O 完成：进入就绪挂起，仍等待调入。

03 易错点

错误说法挂起就是阻塞；换回内存后一定立即运行。

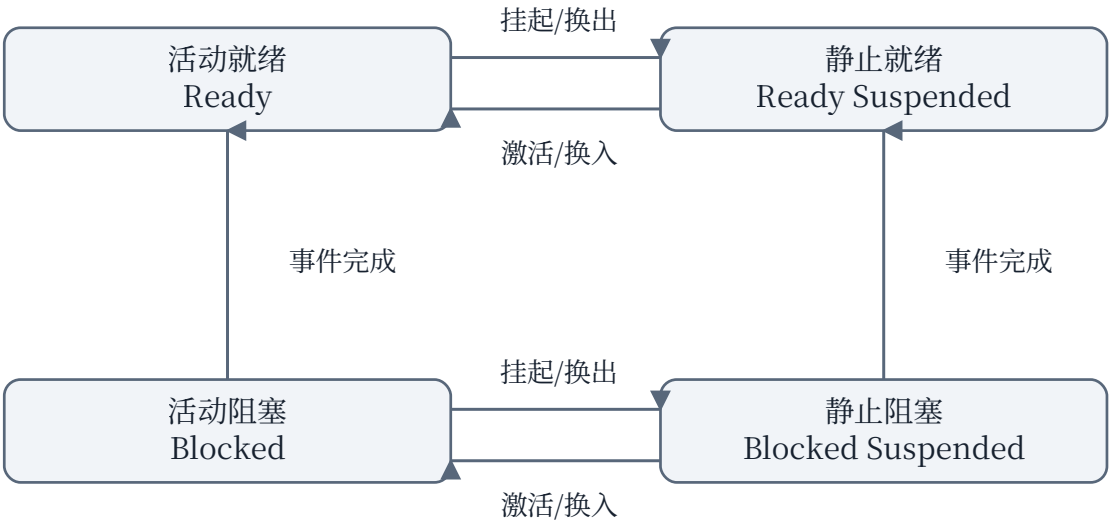
04 图表

要点对照

转换	触发	结果
活动就绪→静止就绪	挂起/换出	仍具备执行条件但暂不在内存
活动阻塞→静止阻塞	挂起/换出	仍等待事件
静止阻塞→静止就绪	等待事件发生	不等于已调入
静止就绪→活动就绪	激活/换入	等待 CPU
静止阻塞→活动阻塞	激活且事件未完成	继续等事件

挂起与阻塞：内存位置、等待事件两条轴

活动：通常在内存　　静止：通常已调出



静止阻塞的事件完成：变静止就绪；并不自动换入，更不直接运行。

活动与静止区分是否在内存等管理因素；就绪与阻塞区分是否等待事件。挂起及激活横向切换，事件完成从阻塞向就绪切换。静止阻塞事件完成后是静止就绪。单独查看图片

自检：就绪挂起能否直接获得 CPU 执行?

该教学模型下一般须先激活/调入，进入就绪再由调度分派。

调度与中断

时钟中断与时间片：中断不一定切换　发生时钟中断，不代表一定进行进程切换。　个人错题　易混　要点

核心结论

定时器产生时钟中断，OS 可更新时间、统计 CPU 使用和检查调度条件；是否切换由条件决定。

别混淆发生时钟中断，不代表一定进行进程切换。

详解

01 解析

时钟中断是外部异步中断。时间片可能由多个时钟滴答组成；内核处理完中断后也可继续原任务。

02 例子

时间片尚未用完且没有更高优先级任务，就可能继续运行当前进程。

03 易错点

错误说法每次时钟中断都会更换当前运行进程。

自检：时钟中断能否仅更新计时而不切换进程?

可以。

动态优先级：读清更新公式和比较方向 先写比较方向、更新公式、更新时刻。 个人错题 易错 要点

核心结论

按题设同时更新 cpuTime、等待量等变量，确认“数值大优先”还是“数值小优先”，以及何时重新计算。

别混淆先写比较方向、更新公式、更新时刻。

详解

01 解析

1. 每次调度前先列各进程的累计运行、等待、基本优先数，不跳过中间时刻。
2. 把当前量代入同一更新公式，再按题设的“大优先/小优先”选择候选。
3. 推进一段时间后，只给实际运行者累加 CPU 时间，给就绪等待者累加等待；是否抢占及等待量何时清零另按题设。

补充原理与条件

不同系统的优先级编号方向不同。到达、抢占、运行、阻塞等事件可能改变状态，静态排序不能替代过程推演。

02 例子

若优先数含 CPU 使用项，运行中的进程会随 CPU 时间积累而变化。

03 易错点

错误说法等待时间越长就一定优先级越高，CPU 时间可以忽略。

自检：调度题最先写清哪三个约定?

优先数比较方向、更新公式、更新时刻。

HRRN、SJF 与饥饿：非抢占也可能饥饿 非抢占也可能饥饿；HRRN 同时考虑等待和服务时间。 冷门 易混 要点

核心结论

SJF 可让长作业长期等待；HRRN 比较 (等待时间+服务时间)/服务时间，通常按非抢占方式选择。

别混淆非抢占也可能饥饿；HRRN 同时考虑等待和服务时间。

详解

01 解析

$R=\frac{W+T}{T}=1+\frac{W}{T}$

高响应比优先：W 为等待时间，T 为所需服务时间，选响应比最大者。

等待越久，响应比越大，能缓解长期等待。绝对“不会饥饿”仍需结合有限作业或到达假设，不能对任何负载无条件保证。

02 例子

一个长作业等待期间不断到来短作业，SJF 可能一直选短作业。

03 易错点

错误说法只有抢占式算法才会饥饿；HRRN 只看作业长度。

自检：HRRN 中等待时间增长，响应比如何变化?

在服务时间固定时增大。

中断、异常与系统调用：发生时刻和处理时刻 区分异常发生前的模式与进入处理程序后的模式。 易错 易混 近年考查 要点

核心结论

中断/异常可以在用户态运行时发生；进入处理程序后由硬件切换到规定特权级。系统调用通过陷入机制请求内核服务。

别混淆区分异常发生前的模式与进入处理程序后的模式。

详解

01 解析

外部中断通常异步；异常与当前指令同步。普通库函数可完全在用户态执行，也可能封装系统调用。发生前模式与处理时模式不能混淆。

02 例子

用户程序访问不存在的页时可以触发缺页异常，而缺页处理程序在内核态执行。

03 易错点

错误说法异常发生时 CPU 一定已在内核态；函数调用自动切换内核态。

自检：普通函数调用是否必然导致用户态转内核态?

不必然，系统调用入口才需要特权切换。

调度层次与上下文切换：三个层次 模式切换不一定换进程；调度是选择，分派是实施。 记忆概念 易错 易混 要点

核心结论

高级调度选择作业进入系统/内存；中级调度管理换入换出；低级调度选择就绪任务使用 CPU。

别混淆模式切换不一定换进程；调度是选择，分派是实施。

详解

01 解析

并非所有 OS 都实现这三层。调度是选择，分派/切换是实施；用户态到内核态转换可仍属于同一进程，不必发生进程切换。

02 例子

系统调用完成后返回原任务，发生模式切换但不一定换进程。

03 易错点

错误说法作业调度、交换调度与 CPU 调度都每执行一条指令进行一次。

自检：CPU 从用户态进入内核态能否直接推出换了进程?

不能。

周转、等待、响应与带权周转 响应看首次响应，周转看全部完成；等待不含 I/O 服务本身。 记忆概念 易错 易混 要点

核心结论

周转时间=完成时刻−到达时刻；带权周转=周转/服务时间；响应时间看首次响应；CPU 调度中的等待时间是就绪队列内等待 CPU 的累计时间。

别混淆响应看首次响应，周转看全部完成；等待不含 I/O 服务本身。

详解

01 解析

$T_i = C_i - A_i, W_i =$

C_i 为完成时刻，A_i 为到达时刻，S_i 为服务时间；W_i 此处表示带权周转。

1. 先逐个作业标注到达、首次响应和完成时刻，周转=完成−到达。
2. 带权周转逐个用周转/服务时间计算，再对比值求平均；不要拿平均周转除平均服务。
3. 含 I/O 时从时间线上单独累计就绪队列等待；周转−CPU 时间还包含阻塞时间，不能一概称为等待。

补充原理与条件

有 I/O 时，就绪等待与 I/O 阻塞必须区分，不能一律用周转−CPU 服务时间求等待。纯 CPU 作业模型才可直接用该差值。平均带权周转要先求每个作业的比值，再平均。

02 例子

任务很快显示第一条输出但仍运行很久，响应短不代表周转短。

03 易错点

错误说法响应时间等于程序从提交到全部完成的时间。

自检：求平均带权周转是否等于平均周转除以平均服务时间?

一般不等，应先分别求各任务比值再按规定平均。

FCFS、SJF、RR 与多级反馈　RR 片大近似 FCFS；SRTF 抢占，常见 SJF/HRRN 非抢占。　记忆概念　易错　易混　要点

核心结论

FCFS 按到达，SJF 常非抢占，SRTF 按剩余时间抢占；RR 轮流给时间片，多级反馈可按运行表现调整队列。

别混淆RR 片大近似 FCFS；SRTF 抢占，常见 SJF/HRRN 非抢占。

详解

01 解析

RR 片很大时可近似 FCFS，片太小切换开销大。到达、抢占、平局排序和新到任务插入位置改变结果，要明确约定。

02 例子

一个长任务前面只有一个很短任务，SJF 可能先完成短任务以降低平均等待。

03 易错点

错误说法时间片轮转越大响应越好；SJF 都按抢占实现。

自检：HRRN 与 SRTF 都必需在新任务到来时抢占吗?

不，常见 HRRN 是非抢占，SRTF 是抢占。

多处理机调度：亲和性与负载均衡　任务数相同不等于负载相同；迁移会损失局部性。　　记忆概念　易错　易混　　要点

核心结论

多处理机调度既考虑资源利用，也考虑缓存/NUMA 等局部性；处理机亲和性倾向保留任务在熟悉的处理机上。

别混淆任务数相同不等于负载相同；迁移会损失局部性。

详解

01 解析

共享或每核就绪队列均可采用；迁移可能损失 Cache 热数据并引入协调开销。任务数相等不代表工作量相等，软/硬亲和的限制也不同。

02 例子

一个 CPU 密集任务与多个短等待任务，数量相同不代表计算负载相同。

03 易错点

错误说法负载均衡只追求各核任务数量相同，迁移没有代价。

自检：强制频繁迁移一定提高总性能吗?

不一定，可能损失局部性与增加管理开销。

同步与死锁

生产者消费者：先申请资源，再进入互斥区　先 P(empty/full)，再 P(mutex)，避免拿锁等资源。　　个人错题　易错　　要点

核心结论

典型有界缓冲区：生产者 P(empty)→P(mutex)；消费者 P(full)→P(mutex)，操作后释放互斥并通知对方。

别混淆先 P(empty/full)，再 P(mutex)，避免拿锁等资源。

详解

01 解析

1. 生产者先 P(empty)，确保有空位；消费者先 P(full)，确保有产品。二者取得相应资源后才 P(mutex)。
2. 若生产者在缓冲区满时先锁 mutex 再等 empty，消费者会被 mutex 挡住，无法取出产品来释放空位。
3. 操作完成先 V(mutex)，再通知另一方的计数信号量。检查所有可能阻塞的 P 是否会抱着对方必需的锁等待。

补充原理与条件

如果持有互斥锁时等待 empty/full，对方可能无法进入缓冲区释放所需资源，形成死锁。每次成功申请都要有对应释放。

02 例子

缓冲区已满，生产者先持有 mutex 再等 empty；消费者无法取得 mutex 去消费。

03 易错点

错误说法先 P(mutex)，再 P(empty/full) 总是安全。

自检：能否持有缓冲区 mutex 再等待 full?

典型方案不应如此，应先 P(full)。

共享变量与原子性：加一不是一个原子步骤　一行代码不等于原子操作；volatile 不提供互斥。　　个人错题　易错　　要点

核心结论

源代码的一行可能包含读、改、写多个步骤；共享变量并发访问需要适当同步。

别混淆一行代码不等于原子操作；volatile 不提供互斥。

详解

01 解析

1. 把 x++ 展成“读 x→加 1→写回 x”，不要把一行源代码当成不可分割动作。
2. 安排两线程都先读到 0，再各自写回 1，就得到一次更新丢失的反例。
3. 锁或原子操作必须覆盖完整读改写；若两个 x 属于不同线程的独立局部对象，则先排除它们共享同一存储的前提。

补充原理与条件

局部变量名相同不一定共享；删掉某个线程的变量也不一定修复同步逻辑。应先明确各变量作用域、原子操作和访问顺序。

02 例子

两线程同时读到 x=0，各自写回 1，最终值可能为 1 而不是 2。

03 易错点

错误说法x++ 在所有机器上都是不可分的操作。

自检：给变量加 volatile 就能保证 x++ 原子吗?

不能，volatile 不是互斥或通用原子同步机制。

死锁、饥饿与安全状态 死锁一定不安全，不安全未必已死锁。 易混 要点

核心结论

死锁：相互等待而无法推进；饥饿：长期得不到服务；不安全：不存在安全序列，但不等于当前已死锁。

别混淆死锁一定不安全，不安全未必已死锁。

详解

01 解析

1. 先问能否找出安全序列：可令某进程完成并归还资源，反复进行，所有进程都能完成则安全。
2. 找不到安全序列说明无法保证未来最大需求都满足，即不安全；它不等于此刻所有相关进程都在等资源。
3. 只有已出现无法自行解除的循环等待，才判实际死锁。不要把银行家算法拒绝一次试分配误读为系统已经死锁。

补充原理与条件

经典可重用资源模型中，互斥、占有且等待、不可剥夺、循环等待是死锁的必要条件。破坏其中任一可预防该模型中的死锁。

02 例子

银行家算法拒绝会使试分配状态不安全的请求，是避免风险，并非证明原状态已经死锁。

03 易错点

错误说法不安全状态就是已经死锁；打破任一必要条件仍可能形成同类资源死锁。

自检：死锁状态是否安全?

不安全；但不安全状态不一定已经死锁。

银行家算法：Need、Available 与 Work Work 加回 Allocation，不是 Max 或 Need。 易错 易混 要点

核心结论

$Need = Max - Allocation$；安全检查尝试寻找 $Need \leq Work$ 的未完成进程，假想完成后 Work 加回其 Allocation。

别混淆Work 加回 Allocation，不是 Max 或 Need。

详解

01 解析

$Need = Max - Allocation$

逐进程、逐类资源计算；假想进程完成后 Work 加回 Allocation。

选择是“存在某个可以完成的进程”，不是固定逐号顺序。申请首先检查 Request≤Need、Request≤Available，再试分配并检查安全性。

02 例子

P1 暂不能完成，但 P2 能完成并释放资源，随后 P1 可能满足需要。

03 易错点

错误说法发现一个进程当前资源不够，就判定整个状态不安全。

自检：安全检查中假想进程完成，Work 加什么?

加回该进程当前已分配的 Allocation，不是 Max 或 Need。

临界区四原则：让权等待与互斥　互斥不自动保证有限等待；自旋不满足让权等待。　　记忆　概念　易错　易混　　要点

核心结论

互斥只解决同时进入；还需考虑空闲让进、忙则等待、有限等待以及题设要求的让权等待。

别混淆互斥不自动保证有限等待；自旋不满足让权等待。

详解

01 解析

1. 先验互斥：构造两个进程都尝试进入的交错执行，看能否同时进入。
2. 再验进展和有界等待：无人占用时能否进入、申请者是否可能无限推迟。
3. 最后单独看等待是否让出 CPU。自旋即使实现互斥，也在忙等，不能因此判它满足让权等待。

补充原理与条件

自旋锁等待时忙等，不能无条件说满足让权等待；阻塞型等待可让出 CPU。应区分安全性（不同时进入）与进展/公平性。

02 例子

一把永远无人释放的锁仍可互斥，却不能使等待任务正常推进。

03 易错点

错误说法保证一次只有一人进入就一定满足所有同步正确性要求。

自检：临界资源与临界区是什么关系?

资源是共享对象，临界区是访问该对象的受保护代码段。

TSL、Swap 与原子指令：不保证公平 硬件原子性不保证公平，也不消除申请顺序死锁。 记忆概念 易错 易混 要点

核心结论

原子读改写将测试/修改作为不可分操作，可构造互斥；普通读写序列可能被交错，不等价。

别混淆硬件原子性不保证公平，也不消除申请顺序死锁。

详解

01 解析

硬件保证该操作原子不自动保证锁公平或让权等待。关闭中断在单核内核特定场景可防本核打断，但不能仅靠它阻止其他核并发访问。

02 例子

两线程均读到 lock=0 再各写 1，会同时误以为获得锁。

03 易错点

错误说法普通函数实现的 swap 与硬件原子 Swap 等价。

自检：原子交换是否自动消除所有死锁?

不，锁的申请顺序等仍可能形成死锁。

Peterson：意愿与 turn，先说明执行模型 Peterson 经典证明有原子读写和顺序一致前提。 记忆概念 易错 易混 要点

核心结论

双进程 Peterson 用 flag 表示进入意愿、turn 打破同时竞争；经典证明基于读写原子和顺序一致等模型。

别混淆Peterson 经典证明有原子读写和顺序一致前提。

详解

01 解析

仅轮流进入可能在另一方不想进入时也阻挡自己。现代硬件/编译器可重排，真实代码还需适当内存语义；408 理论题按给定原子与顺序约定推演。

02 例子

两者同时 flag=true 时，turn 为其中一个提供让步方向。

03 易错点

错误说法只用 turn 就同时满足任何情况下的互斥和空闲让进。

自检：Peterson 的 flag 与 turn 是可互相完全替代的吗?

不能，二者解决的竞争条件不同。

信号量、管程与条件变量：条件需再检查　Mesa 条件变量被唤醒后用 while 重查条件。　记忆概念　易错　易混　　要点

核心结论

信号量维护计数并以 P/V 协调；管程封装共享状态和互斥；条件变量用于在条件不满足时等待并由通知唤醒。

别混淆Mesa 条件变量被唤醒后用 while 重查条件。

详解

01 解析

1. 在 Mesa 语义下，条件变量通知只使等待者有机会重新竞争锁，不会把条件永久保留给它。
2. 等待者醒来拿到锁之前，其他线程可能已经消费了资源，使条件再次为假。
3. 因此用 while 检查条件，不满足就继续 wait；用 if 只检查一次会在被唤醒后误入不满足条件的操作。

补充原理与条件

Mesa 风格条件等待应在 while 中重查条件，因为唤醒后可能被他人先改变。条件通知不等于保留一个永久许可；不同管程语义需按题设。

02 例子

消费者被唤醒但尚未拿锁前，另一个消费者可能取走唯一数据。

03 易错点

错误说法条件变量 signal 等于给资源数永久加一；唤醒后条件一定仍成立。

自检：条件等待返回后通常为何还要检查条件?

唤醒不保证自己实际获得运行时条件仍满足。

读者写者：并发读与独占写　读者可并发，readCount 更新仍须互斥。　　记忆概念　易错　易混　　要点

核心结论

读者可并发读取；写者需排他访问。读者计数本身要保护，访问策略区分读者优先、写者优先与公平方案。

别混淆读者可并发，readCount 更新仍须互斥。

详解

01 解析

首读者锁定写资源、末读者释放是一种常见读者优先方案；源源不断读者可能延迟写者。写者优先也可能使读者长期等待，不能只看互斥。

02 例子

读者共享数据只读，但更新 readCount 属于写共享状态，需单独同步。

03 易错点

错误说法读者之间必须全部互斥，写者优先总能避免所有任务饥饿。

自检：允许并发读能否因此省掉读者计数的保护?

不能。

哲学家进餐：防环路与限制并发 防死锁不自动防饥饿；公平性要另看。 记忆概念 易错 易混 要点

核心结论

各自持一支等待另一支可形成循环等待；可限制同时取叉人数、统一资源顺序或设计仲裁方案。

别混淆防死锁不自动防饥饿；公平性要另看。

详解

01 解析

限制至多 n−1 人尝试是经典思路，但仍需完整释放路径；资源编号顺序消除循环等待。具体公平性另需验证。

02 例子

5 人同时拿左叉后全等右叉，即使每支叉严格互斥，也仍可能死锁。

03 易错点

错误说法每人依次拿左再拿右，只要叉子互斥就不会死锁。

自检：防死锁方案是否自动保证任何人都不会饥饿?

不自动保证，还需看公平与调度规则。

死锁预防、避免、检测与解除 预防破条件，避免保安全，检测与解除允许先发生。 记忆概念 易错 易混 要点

核心结论

预防破坏必要条件；避免动态保持安全；检测允许发生后识别；解除采取撤销/剥夺等策略。

别混淆预防破条件，避免保安全，检测与解除允许先发生。

详解

01 解析

银行家基于最大需求等信息进行避免。预防可能降低利用率；解除要选牺牲者并防重复牺牲造成饥饿。四类策略目的相近但时机不同。

02 例子

统一资源申请顺序是预防；试分配后安全检查属于避免。

03 易错点

错误说法银行家算法是死锁已经发生后的检测与解除算法。

自检：不安全就立即撤销进程是否是银行家必然动作?

不是，典型避免会拒绝/延迟可能使状态不安全的请求。

资源分配图：环的充分性取决于实例数　单实例有环可判死锁，多实例有环未必死锁。　　记忆概念　易错　易混　　要点

核心结论

单实例资源模型中环可构成死锁的充要条件；多实例资源中有环不一定死锁，需结合剩余实例和可完成进程检查。

别混淆单实例有环可判死锁，多实例有环未必死锁。

详解

01 解析

1. 先看每类资源实例数，再画请求边“进程→资源”和分配边“资源→进程”。
2. 每类只有一个实例时，环会使链上进程互等，可据环判死锁。
3. 有多个实例时，环外进程可能完成并释放实例打破等待；必须继续做可满足请求与归还资源的化简，不能见环即判死锁。

补充原理与条件

请求边通常进程→资源，分配边资源→进程。多实例还有外部可完成进程释放资源的可能，单纯画环不能给出完整判定。

02 例子

某类型还有一空闲实例可满足等待者，环内进程仍可能继续。

03 易错点

错误说法有向资源分配图只要有环就必然死锁。

自检：请求边和分配边方向能否随便反过来?

不能，方向代表等待与占用关系。

安全、不安全、死锁：三者不是进程状态 安全序列是可行完成顺序，不是强制执行的调度顺序。 记忆概念 易混 图解 要点

核心结论

安全状态存在安全序列；不安全状态不存在这样的安全完成保证；死锁状态已有相互等待且无法自行推进的进程集合。

别混淆安全序列是可行完成顺序，不是强制执行的调度顺序。

详解

01 解析

1. 按银行家假设，用当前可用资源作为 Work，找 Need≤Work 的未完成进程。
2. 假想该进程完成，Work 加回它已分配的资源，继续寻找；能消去全部进程则有安全序列。
3. 中途无法继续仅证明不安全；要判当前死锁还须看实际等待请求及资源能否释放，不能把潜在最大需求当作当前请求。

补充原理与条件

在银行家算法适用的资源模型中，死锁状态属于不安全状态，但不安全未必已死锁。它们描述系统资源分配情形，不是 Ready/Running/Blocked 的另一组名称。安全序列是可行完成次序，并非必须照此调度。

02 例子

某次分配后找不到安全序列，应拒绝该次试分配；这不说明试分配前已死锁。

03 易错点

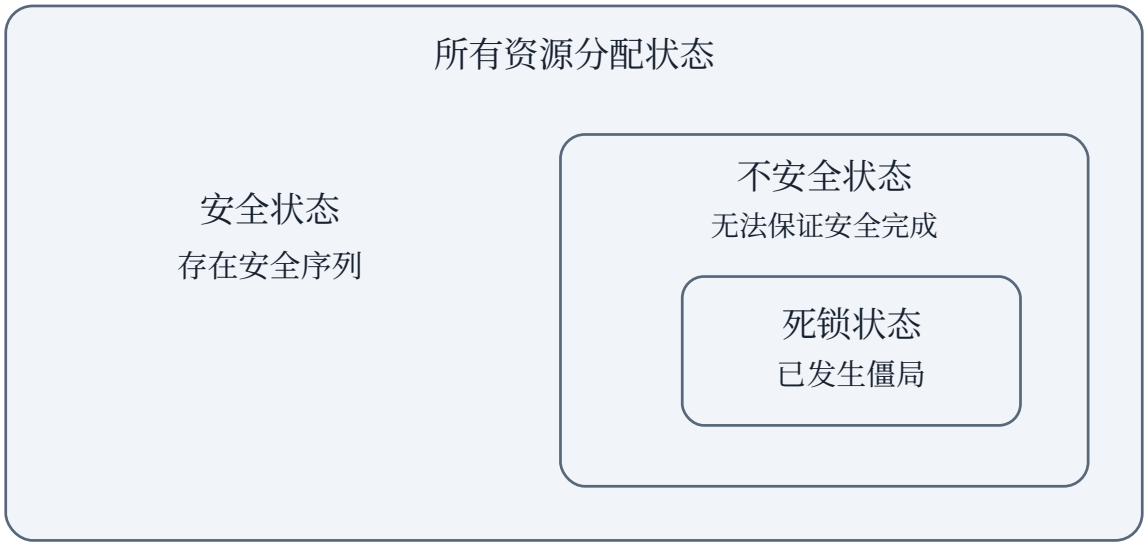
错误说法不安全状态就是已经死锁；没有死锁就必定安全。

04 图表

要点对照

名称	判据	易混点
安全	存在安全序列	可保证各进程按某序完成
不安全	无安全序列	可能走向死锁，未必已发生
死锁	一组进程互等且不能自行推进	是已发生的僵局

资源安全：死锁是不安全状态的子集



银行家算法适用模型：死锁 ⇒ 不安全；不安全 ⇏ 已死锁。

所有状态划分为安全与不安全。安全存在安全序列；死锁属于不安全，但不安全并不一定已经死锁。此处是系统资源分配状态，不是进程五状态。单独查看图片

自检：系统不安全，是否一定已经有环形等待?

不能据此直接断言，需分析当前请求与资源关系。

临界区四段：进入区、临界区、退出区、剩余区　退出区仅释放本次临界区权限，不表示进程终止。　　记忆概念　易混　图解　　要点

核心结论

互斥程序四段：进入区申请权限→临界区访问资源→退出区释放权限→剩余区做其他工作。

别混淆退出区仅释放本次临界区权限，不表示进程终止。

详解

01 解析

进入区申请进入权限；临界区操作临界资源；退出区释放权限；剩余区执行其他工作。临界区是代码段，临界资源是被互斥访问的资源。P/V 位置取决于实现和资源模型，不能把所有 P 都机械等同进入区。

02 例子

互斥锁 lock→修改共享计数器→unlock→其他计算，对应四段。

03 易错点

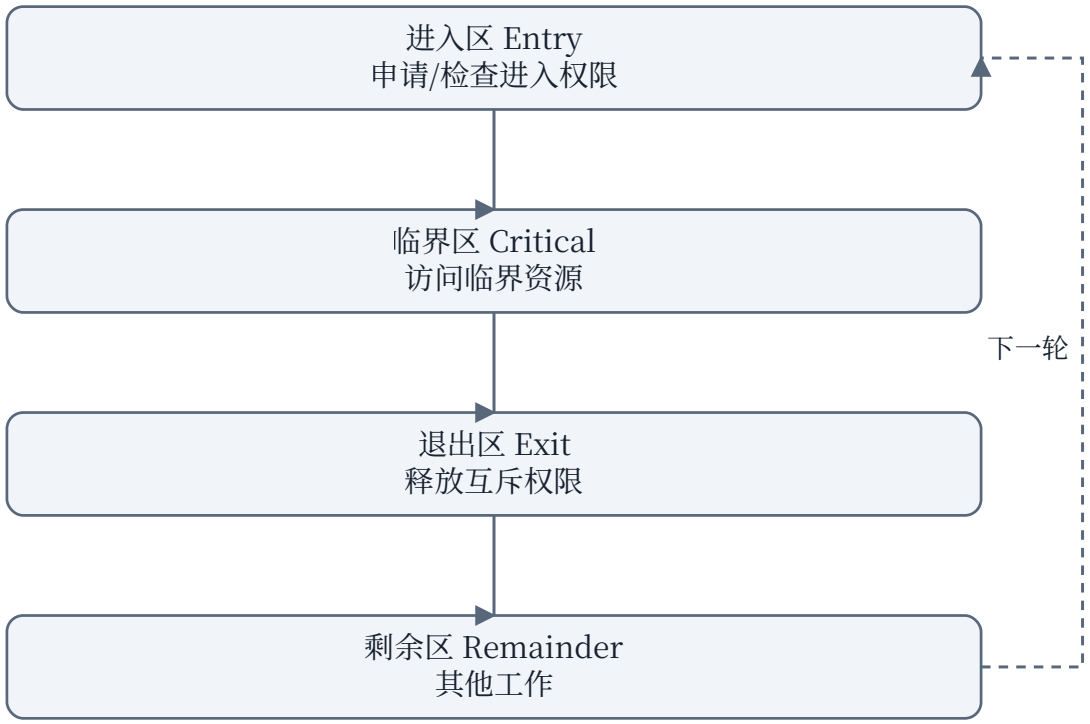
错误说法进入区就是访问共享数据的临界区；退出区就是进程退出。

04 图表

要点对照

名称	英文	作用
进入区	Entry Section	检查/申请进入权限
临界区	Critical Section	访问临界资源
退出区	Exit Section	释放互斥权限
剩余区	Remainder Section	其余代码

互斥程序四段：申请 → 访问 → 释放 → 其他



临界区是代码段，临界资源是资源；退出区不表示进程终止。

程序每次访问临界资源先进入进入区申请权限，在临界区访问，退出区释放权限，再执行剩余区。可再次循环访问，退出区不是进程终止。单独查看图片

自检：退出区是否表示进程生命周期结束?

不是，只是离开本次临界区时释放权限。

存储管理

TLB 缺失、缺页与 Cache 缺失　TLB 未命中不等于缺页；先区分“映射没缓存”与“页没驻留”。　易错　易混　近年考查　记忆概念　要点

核心结论

TLB 缺失表示地址转换缓存未命中，需查页表；请求分页模型中，缺页表示所需页未驻留内存；Cache 缺失表示访问的块不在该级缓存。

别混淆TLB 未命中不等于缺页；先区分“映射没缓存”与“页没驻留”。

详解

01 解析

1. TLB 未命中后先查页表，不直接访问磁盘。若表项显示目标页已驻留且权限满足，取得页框号即可继续。
2. 只有请求的页未驻留时，才触发本条请求分页模型中的缺页处理；处理后再重试原访存。
3. 例如 TLB 查询 10 ns、访存 100 ns、单级页表且无缺页：命中为 110 ns，未命中为 210 ns，不能给未命中分支擅自加磁盘时间。

补充原理与条件

先查 TLB，未命中再按页表完成地址转换。页表表项表明目标页已驻留且权限满足时，无需调页；目标页未驻留才进入缺页处理。数据 Cache 缓存块内容，TLB 缓存映射，二者命中与否不能相互推出。

02 例子

页在内存但其映射未进入 TLB，只发生 TLB miss，不发生 page fault。

03 易错点

错误说法TLB 不命中就必须从磁盘调页。

考点联系

区分地址转换检测与数据缓存命中判断。

变式

自编模拟变式 · 把 TLB 未命中拆成两种结果

两次访存都 TLB 未命中：第一次页表显示页已驻留，第二次显示页未驻留。是否都要从外存调页?

答案与解析

只有第二次需要本条请求分页模型中的调页处理；第一次取页框号并补充 TLB 后即可继续。

关键区别：TLB 命中率影响地址转换开销；缺页率才决定额外的调页分支，不能用同一个概率代替。

参考资料

参考：OSTEP：TLB 与分页机制

自检：TLB 命中能否直接说明数据 Cache 命中?

不能，两者缓存对象不同。

分页与分段：碎片位置和保护单位　分页有内部碎片；分段须先检查段限。　　易混　记忆概念　　要点

核心结论

分页通常存在页内碎片；分段按逻辑段分配，可能有段间外部碎片。段式地址由段号和段内偏移组成。

别混淆分页有内部碎片；分段须先检查段限。

详解

01 解析

固定页大小便于物理分配，逻辑段长度可变。段内偏移必须检查段限，不能只加基址。

02 例子

最后一页用不满是内部碎片；空闲段间空洞是外部碎片。

03 易错点

错误说法分页没有任何碎片；分段地址只需段号。

自检：段内偏移超出段长时，直接用基址+偏移可行吗?

不可行，会触发越界保护。

CLOCK 与 LRU：近似不等于完全一致　CLOCK 是近似 LRU；访问位不能完整记录先后次序。　　冷门　易混　　要点

核心结论

CLOCK 利用访问位给页面第二次机会，是低开销近似；精确 LRU 需要更细的访问顺序信息。

别混淆CLOCK 是近似 LRU；访问位不能完整记录先后次序。

详解

01 解析

1. 按题设确定指针初始位置及新装入页的访问位，再逐次记录每次命中和缺页。
2. 基本 CLOCK 缺页时从指针开始：遇 1 清 0 并继续，遇 0 才替换；命中通常把访问位置 1。

3. LRU 需要比较最近访问先后；CLOCK 只有粗略访问标记，因此不能用一份 LRU 淘汰表代替 CLOCK 的扫描过程。

补充原理与条件

基本 CLOCK 遇访问位 1 清零并继续，遇 0 选择。改进型还考虑修改位，其扫描规则须看题设。

02 例子

两个页面都访问过，CLOCK 可能不保留它们的精确先后次序。

03 易错点

错误说法CLOCK 就是精确的 LRU。

自检：改进型 CLOCK 为什么考虑脏位?

替换脏页可能需要写回，开销更大。

工作集与抖动：更多进程未必更高利用率 抖动时加进程可能更糟，应减轻内存竞争。 冷门 易混 近年考查 要点

核心结论

工作集是最近一段访问中涉及的页面集合；内存不足以容纳活跃工作集可能引起频繁换页和抖动。

别混淆抖动时加进程可能更糟，应减轻内存竞争。

详解

01 解析

抖动时大量时间花在调页，CPU 利用率可能下降；应考虑减少多道程序度或增加可用页框。工作集大小与窗口和局部性相关。

02 例子

进程刚切换阶段，工作集可能改变，不是永久不变的常数。

03 易错点

错误说法缺页越多说明 CPU 越忙，继续增加多道程序度就能改善。

自检：抖动时盲目增加并发进程通常能解决吗?

不能，可能进一步加重内存竞争。

二级页表：页内偏移与每级索引位数　先算每页可放多少表项，再划各级索引位。　记忆概念　易错　易混　要点

核心结论

若每个下级页表恰好占一页，单表表项数=页大小/页表项大小；该级索引位数是表项数的 $\log_2$（参数为 2 的幂）。

别混淆先算每页可放多少表项，再划各级索引位。

详解

01 解析

$b_{\text{offset}}=\log_2 P, b_{\text{index}}=\log_2 \frac{P}{e}$

每个下级页表恰占一页；P 为页字节数，e 为页表项字节数，参数取 2 的幂。

1. 先算页内偏移位数 $\log_2 P$，它不属于任何一级页号。
2. 每个下级页表占一页时，表项数=页大小/表项大小，据此求下级索引位数。
3. 虚拟地址剩余页号位交给上级索引；若题目没有“每个下级表恰占一页”的约定，不能自动这样拆分。

补充原理与条件

先从地址宽度扣除页内偏移，再按能容纳的表项数分各级索引。题设可以让页目录大小与下级页表大小不同，不能擅自平分。

02 例子

32 位地址、4 KiB 页、4 B 页表项：页内偏移 12 位，下级索引 10 位，常见两级结构剩余目录索引 10 位。

03 易错点

错误说法两级页表就把剩余地址位无条件平分；页表项越大索引位越多。

自检：页内偏移经过地址转换会改变吗?

普通分页中页大小相同，页内偏移不变。

连续分配：首次、最佳、最坏与伙伴　最佳适应不保证长期最佳；伙伴只合并指定同级伙伴。　记忆概念　易错　易混　要点

核心结论

首次选首个够大空洞，最佳选最小够大，最坏选最大；伙伴按 2 的幂分配与指定伙伴合并。

别混淆最佳适应不保证长期最佳；伙伴只合并指定同级伙伴。

详解

01 解析

最佳适应可能留下很多小孔洞；连续变长分配主要外部碎片，固定/对齐分配可有内部浪费。伙伴只合并同级且互为伙伴的空闲块，不是任意相邻块。

02 例子

请求 18 单位，伙伴分配可能给 32 单位，剩余部分体现内部浪费。

03 易错点

错误说法最佳适应留下的空闲区一定更容易使用，外部碎片越多内部碎片越多。

自检：最佳适应是否保证全局最优长期利用率?

不保证。

OPT、FIFO 与 LRU：Belady 异常　同一序列、标准初始条件下，LRU/OPT 无 Belady 异常；FIFO 可能有。　　记忆概念　易错　易混　　要点

核心结论

OPT 替换未来最久不用页作理论最优；FIFO 替换最早进入页；LRU 替换过去最久不用页。FIFO 可发生 Belady 异常。

别混淆同一序列、标准初始条件下，LRU/OPT 无 Belady 异常；FIFO 可能有。

详解

01 解析

1. 命中页时也要更新 LRU 的最近使用次序；FIFO 只维护进入内存的先后，命中不改队首。
2. 缺页再按各自规则选牺牲页，每次换出与装入都记录，避免把两种队列混用。
3. 比较不同页框数时，FIFO 可能更多页框反而更多缺页；LRU/OPT 在标准条件下有栈性质，不会出现这种 Belady 异常。

补充原理与条件

对相同访问序列与标准初始条件，LRU 和 OPT 具有栈性质，增加页框不会增加缺页次数；FIFO 无此保证。命中时 LRU 更新最近使用次序，FIFO 不更新入驻先后。

02 例子

FIFO 某序列用更多页框反而更多缺页，是异常，不是计算一定错。

03 易错点

错误说法增加页框后任何页面替换算法的缺页次数都不会增加。

变式

自编模拟变式 · 命中之后要不要移动队列

页框已有 A、B、C，入驻顺序和最近访问顺序均为 A 最早、C 最新。再次访问 A 后，下一次缺页先淘汰谁?

答案与解析

FIFO 仍淘汰 A；LRU 将 A 更新为最近使用，因此淘汰 B。

关键区别：一条命中访问就能把两种算法的后续选择分开，推表时不要只在缺页时更新 LRU。

参考资料

参考：OSTEP：页面替换策略

自检：FIFO 命中是否一定把页面移到队尾?

经典 FIFO 不这样更新入驻顺序；LRU 才更新最近访问信息。

驻留集分配、局部替换与全局替换　固定分配通常配局部替换；局部/全局看页面归属，不看地址远近。　　记忆概念　易错　易混　　要点

核心结论

局部替换只选本进程驻留页；全局替换可选其他进程页。教材常见组合是固定分配+局部替换、可变分配+局部替换、可变分配+全局替换。

别混淆固定分配通常配局部替换；局部/全局看页面归属，不看地址远近。

详解

01 解析

1. 先判固定或可变分配：该进程允许使用的页框数能否随运行改变。
2. 再判替换范围：局部只从本进程驻留页中选，全局可以拿走其他进程的页框。
3. 普通全局替换会改变各进程页框份额，因此与固定分配冲突；“局部”说的是页面归属，不是地址靠近。

补充原理与条件

固定驻留集要求替换后页框数不变，普通全局替换会改变进程间页框分配，因此“固定分配+全局替换”不是教材常见的可行组合。局部/全局是页面归属范围，不是物理地址远近。

02 例子

全局替换可能使另一进程失去页框，影响其未来访问。

03 易错点

错误说法局部置换表示只能替换物理地址靠近缺页地址的页。

参考资料

参考：Universidade da Coruña：Memory Management and Virtual Memory

自检：固定分配能否直接与普通全局替换任意组合?

不能。普通全局替换从别的进程夺取页框，会改变各进程驻留集大小；教材通常用固定局部、可变局部、可变全局三种组合。

内存映射文件：虚拟页关联文件内容　映射不等于立即装入；私有映射修改不必写回原文件。　　记忆概念　易错　易混　　要点

核心结论

内存映射文件把虚拟地址范围与文件内容关联，程序可通过访存访问，页面可按需装入。

别混淆映射不等于立即装入；私有映射修改不必写回原文件。

详解

01 解析

共享/私有映射影响修改可见性和写回；私有映射可写时复制，不保证修改原文件。建立映射、读页、写回同步等是不同动作，不能只凭“映射”认定已物理加载全部文件。

02 例子

首次读映射区域可能触发缺页，由系统把对应文件页装入内存。

03 易错点

错误说法mmap 把整个文件立刻复制进进程每一个内存页，不涉及缺页。

自检：私有映射的写入是否必然立即修改磁盘原文件?

不必然，通常不作为对原文件的共享写入。

文件系统

FAT：下标是当前块，表项值是下一块　FAT 下标是当前物理块号，表项给下一块号。　　个人错题　易错　　要点

核心结论

FAT[当前磁盘块号] = 下一磁盘块号；目录记录起始块，链末使用结束标记。

别混淆FAT 下标是当前物理块号，表项给下一块号。

详解

01 解析

1. 先用“文件字节偏移÷块大小”得到从 0 开始的逻辑块号 k，以及块内余数。
2. 从目录给的起始物理块出发，连续查 FAT 恰好 k 次；每次用当前物理块号作下标。
3. 到达的物理块配合余数才是最终位置。不要直接把 k 当 FAT 下标，也不要把文件偏移加到起始块号上。

补充原理与条件

FAT 集中保存链接信息。文件的逻辑块序号用于决定要走几次链，不等于 FAT 下标。不能把数据块里的内容当链接指针。

02 例子

链 100→106→108：FAT[100]=106，FAT[106]=108，FAT[108]=结束标记。

03 易错点

错误说法FAT 的表项按文件中的第 1、2、3 块编号。

自检：要查文件第 3 个块，应以 3 直接查 FAT[3] 吗?

不应，从起始块按链走两次。

打开、删除与硬链接：目录项不等于打开表　Unix 中 unlink 去掉名字，不必立即销毁仍被引用的文件。　　冷门　易混　近年考查　　要点

核心结论

在典型 Unix 语义下，unlink 删除一个目录链接；仍被打开或仍有硬链接的文件对象可以继续存在。

别混淆Unix 中 unlink 去掉名字，不必立即销毁仍被引用的文件。

详解

01 解析

文件描述符表、系统打开文件表、inode 与目录项各有作用。硬链接共享同一文件对象；符号链接保存路径。

02 例子

删掉一个硬链接名，另一个硬链接仍可访问同一文件。

03 易错点

错误说法文件一被删除，所有已打开的描述符都立即失效。

自检：硬链接与符号链接哪个通常有独立的路径内容?

符号链接。

位示图与空闲空间：先定义 0/1 与编号　先定 0/1 含义和编号起点，再做字号、位号换算。　个人错题　易错　近年考查　记忆概念　要点

核心结论

按题设解释占用位；块号=字号×每字位数+字内位号，前提是两者均从 0 开始。

别混淆先定 0/1 含义和编号起点，再做字号、位号换算。

详解

01 解析

若编号从 1 开始需调整。位示图通常为每个磁盘块保留一位，其大小与总块数有关，不随当前空闲块数线性变化。

02 例子

每字 32 位，0 起始编号的第 2 字第 3 位对应块号 67。

03 易错点

错误说法位示图中的 1 永远代表空闲，磁盘块号直接等于字号。

自检：先确认哪些约定?

位值含义、字长、字和位编号起点，以及位从高位还是低位排列。

索引文件容量：直接、一级与二级指针　容量公式算文件数据；若问总占用量，再加实际使用的各级索引块。　记忆概念　易错　易混　要点

核心结论

b B 索引块、每地址 p B（b 可被 p 整除）：一级覆盖 b/p 块，二级覆盖 $\left(\frac{b}{p}\right)_2$ 块。若有 d 个直接、s 个一级、t 个二级指针，最大数据容量=$\left[d+s\frac{b}{p}+t\left(\frac{b}{p}\right)_2\right]b$。

别混淆容量公式算文件数据；若问总占用量，再加实际使用的各级索引块。

详解

01 解析

$q=\frac{b}{p},L_{\max}=(d+sq+tq^2)b$

b 为块字节数，p 为地址项字节数；d、s、t 为直接、一级、二级指针数量。

1. 先算每个索引块可放多少地址 q=b/p，而不是把块字节数直接当指针个数。
2. 直接指针覆盖 1 个数据块，一级间接覆盖 q 个，二级覆盖 q^2 个；按各类指针数量相加。
3. 最后乘数据块大小 b 才得文件数据容量。中间索引块占磁盘空间，但不算文件有效数据字节。

补充原理与条件

若 inode 有 d 个直接、s 个一级、t 个二级指针，最大数据容量为 $[d+s(b/p)+t(b/p)^2]\times b$；这是数据字节，不包含索引块自身占用。

02 例子

b=1024 B、p=4 B，则每索引块有 256 项，二级间接可覆盖 256^2 个数据块。

03 易错点

错误说法二级间接指针只多指向一块数据；索引块不占存储。

变式

自编模拟变式 · 文件容量与总磁盘占用

块大小 1 KiB、每地址项 4 B，只有 1 个一级间接指针。最大文件数据量和存满时总占用分别是多少?

答案与解析

每索引块 256 项，数据最多 256 KiB；另有 1 个索引块，总占用 257 KiB，忽略其他元数据。

关键区别：容量只算文件有效数据；总占用还要计索引结构，二者相差的不是一个地址项而是一整个索引块。

自检：求“总占用磁盘块数”时是否只算数据块?

不能，还应按实际使用情况计索引块。

目录、inode 与路径查找 目录记名字到对象；inode 记属性与定位信息。 记忆概念 易错 易混 要点

核心结论

目录项建立名字到文件对象/标识的对应；inode 等元数据记录属性与块定位信息；路径按各级目录解析。

别混淆目录记名字到对象；inode 记属性与定位信息。

详解

01 解析

名字与对象分离有助硬链接。打开后用描述符定位打开对象不等于每次读都重走完整路径；缓存命中会改变实际磁盘访问次数。

02 例子

/a/b/c 的打开可能依次解析 a、b、c 的目录项，而非直接按字符串计算数据块号。

03 易错点

错误说法inode 内必定保存文件名，打开文件无需路径解析。

考点联系

区分按路径打开与按描述符读写。

变式

自编模拟变式 · 打开后是否再次按名查找

程序已成功 open 得到 fd，之后连续调用 read(fd)。是否每次都必须从根目录重新查找完整路径?

答案与解析

不需要。read 通过 fd 查已打开对象及当前偏移；路径解析属于 open 等按名操作的职责。

关键区别：实际读写仍可能涉及数据块与缓存访问，但不能把它们统统计作重复的路径查找。

自检：目录文件保存的内容等同于所有文件的实际正文吗?

不同，主要是命名与组织信息。

连续、链接与索引分配：随机访问和增长 链接分配增长方便，但随机访问常需沿链。 记忆概念 易错 易混 要点

核心结论

连续分配随机定位简单但增长与外部碎片受限；链接分配易增长但常需沿链；索引分配通过索引定位块。

别混淆链接分配增长方便，但随机访问常需沿链。

详解

01 解析

1. 连续分配可用“起始块+逻辑块号”定位；普通链接分配必须从起点逐个跟链。
2. 索引分配则按逻辑块号找到对应索引项，多级索引再分解为各级索引位置。
3. 求磁盘访问次数时列出实际要读的索引块和数据块，并扣除已在缓存中的部分；不要将所有分配方式套同一公式。

补充原理与条件

FAT 是链接信息集中化的形式；多级索引的随机定位仍有索引访问开销。最大容量、目录大小和磁盘读次数须按实际结构与缓存条件计算。

02 例子

随机读链末块可能需追踪很多链接，不应直接把逻辑序号加到起始物理块号。

03 易错点

错误说法链接分配随机访问任何逻辑块都 $O(1)$，连续分配天然可无限增长。

自检：索引块是否保存实际全部文件正文?

不是，主要保存数据块等地址。

挂载、访问保护与共享：权限检查不是文件内容　目录可见不代表文件可写；路径各级搜索权限也可能影响打开。　　记忆概念　易错　易混　　要点

核心结论

挂载把文件系统接入统一命名空间；权限/ACL 等保护决定主体可执行的操作，共享与保护共同管理访问。

别混淆目录可见不代表文件可写；路径各级搜索权限也可能影响打开。

详解

01 解析

读取目录和元数据不等于把全部数据加载。用户身份、操作类型、目录搜索权和文件权限可能共同决定访问；硬链接、打开引用和映射分别带来不同共享关系。

02 例子

路径中的某目录无搜索权限时，即使知道目标文件名，也不一定能成功打开。

03 易错点

错误说法挂载必然把文件系统每个文件全部复制到内存，文件共享不需保护。

自检：“能看到目录项”能否必然推出能写该文件?

不能，写权限需另行检查。

缓存、写回与一致性：日志的目的　日志不等于备份；close 不自动保证所有数据已永久落盘。　　记忆概念　易错　易混　　要点

核心结论

缓存可提高 I/O 性能，写回策略决定落盘时机；文件系统日志/一致性机制帮助处理异常中断后的元数据或事务状态。

别混淆日志不等于备份；close 不自动保证所有数据已永久落盘。

详解

01 解析

缓存脏块、内核缓冲、设备内部缓存的持久化保证不同。关闭文件不等于所有实现中硬件永久保存全部写入；同步语义要看规定。日志也不能替代独立备份。

02 例子

目录项已改但分配表未完全落盘，崩溃可能产生一致性问题，需恢复机制。

03 易错点

错误说法写入缓存后立即断电，磁盘必已有全部新数据；日志就是文件正文的完整备份。

自检：有日志能否推出永不丢失任何最近数据?

不能，取决于日志模式、提交与持久化边界。

打开文件表：进程记用法，系统记公共信息 教材二表模型：读写指针在进程表；外存位置在系统级公共信息中。 记忆概念 易错 易混 图解 要点

核心结论

按本页教材二表模型：进程打开文件表记录读写位置、访问方式和系统表索引；系统打开文件表记录文件公共信息及打开计数。Unix/Linux 的字段归属另见实现差异。

别混淆教材二表模型：读写指针在进程表；外存位置在系统级公共信息中。

详解

01 解析

1. 先读题设模型：若采用教材二表结构，fd 索引进程打开文件表，读写指针按该教材放在进程表中。
2. 若题目明确 Unix/Linux，则 fd 表指向打开文件描述，偏移量在后者；dup 或 fork 继承可让多个 fd 共享该偏移。
3. 两个独立 open 通常创建两个打开文件描述，偏移各自独立。先分清模型与打开来源，再判断一个进程读文件会不会改变另一个的读写位置。

补充原理与条件

按 408 常见教材二表模型记忆。读写指针是当前偏移量，不是磁盘地址；本次只读/读写权限，也不同于文件本身的 rwx 权限。

02 例子

P、Q 打开同一文件：各自读写位置和访问方式可不同，但引用相同的公共文件信息。

03 易错点

错误说法把读写指针和文件的外存地址混为一谈；把本次打开权限和文件权限位混为一谈。

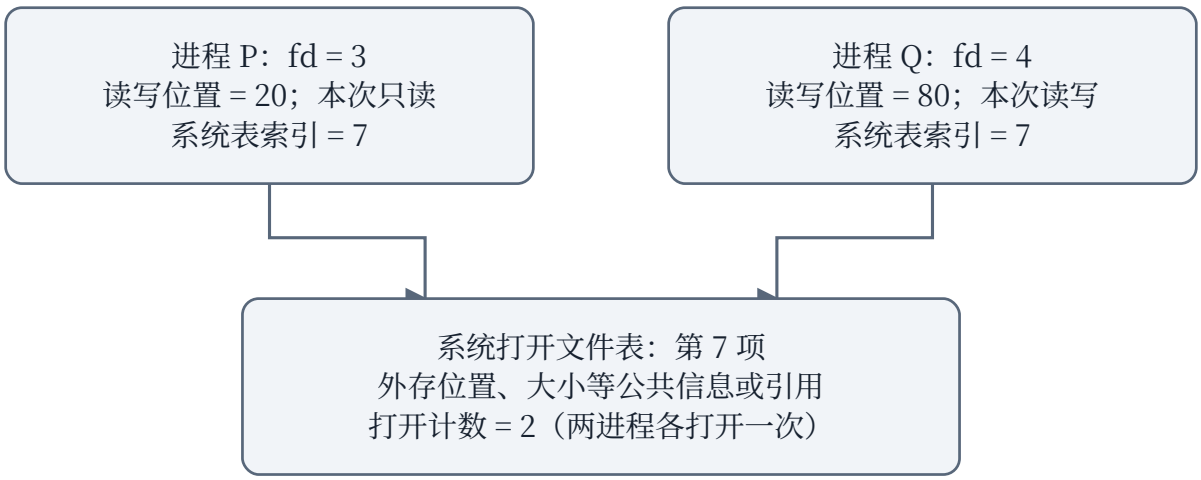
04 图表

要点对照

表	重点字段	作用
进程打开文件表	读写指针；本次访问权限；系统表索引	记录“这个进程怎样使用文件”
系统打开文件表	外存位置；大小、时间等公共信息或其引用；打开计数	管理“被打开文件的公共信息”

打开文件表：进程记用法，系统记公共信息

408 常见教材二表模型



fd → 进程表项 → 系统表项；同一文件，使用位置和权限可不同。

教材二表模型：两份进程表可以指向同一系统表项，读写位置与本次权限可不同。单独查看图片

教材模型与实现差异

UNIX/Linux 的实现另分三层：fd 表→打开文件描述→inode。偏移量和打开方式在打开文件描述中；dup/fork 继承可共享它，独立 open 通常不共享偏移。遇到此类题按题设模型判断。

变式

自编模拟变式 · 独立 open 与 dup 的偏移区别

明确采用 Unix/Linux：fd1=open(…) 后，分别考虑 fd2=dup(fd1) 与再次独立 open 同一文件。fd1 读 10 B 后 fd2 的位置是否都变?

答案与解析

dup 得到的 fd2 通常共享打开文件描述及偏移，位置随之改变；独立 open 得到另一打开文件描述，偏移通常独立。

关键区别：“同一文件”不足以判断偏移共享；关键是是否指向同一打开文件描述，题设若使用教材二表模型则先按其定义。

参考资料

参考：Operating System Concepts 配套资料：Open Files（教材抽象）；Linux man-pages：open(2)、打开文件描述与访问模式

自检：文件描述符 fd 索引哪张表？读写指针放在哪里?

fd 索引进程打开文件表；在本页教材二表模型中，读写指针也在进程表。

SPOOLing、缓冲与磁盘调度　SCAN 到边界反向；LOOK 到最后请求处反向。　冷门　易混　近年考查　要点

核心结论

SPOOLing 用磁盘队列虚拟共享独占设备；缓冲用于速率匹配。SCAN 走到边界再反向，LOOK 在该方向最后请求处反向。

别混淆SCAN 到边界反向；LOOK 到最后请求处反向。

详解

01 解析

1. 遇到磁盘移动距离题，先画当前磁头位置、初始方向、请求位置和两端边界。
2. SCAN 在没有后续请求时仍到该方向边界才反向；LOOK 到最远的待服务请求即反向。
3. 逐段加实际移动距离；循环算法若给回程，检查题设是否把返回过程也计入总距离。

补充原理与条件

SCAN/LOOK 的差别会影响移动距离；C-SCAN/C-LOOK 还需确认回程是否计距离。机制概念与题目计费约定应分开。

02 例子

当前方向上最后请求是 80、磁盘边界是 99：LOOK 可在 80 反向，SCAN 到 99。

03 易错点

错误说法SPOOLing 把一台打印机变成多台物理设备；SCAN 就是访问最远请求后必回头。

自检：磁盘题只给“电梯算法”时应核对什么?

方向、边界、SCAN/LOOK 约定及回程距离是否计入。

I/O 软件层次、设备独立性与缓冲　中断接到内核缓冲，不代表已复制到用户缓冲。　记忆　概念　易错　易混　要点

核心结论

常见层次包括用户 I/O、设备独立软件、驱动、中断处理与硬件。设备独立性用统一接口隐藏设备细节；缓冲协调速率与粒度。

别混淆中断接到内核缓冲，不代表已复制到用户缓冲。

详解

01 解析

逻辑设备到物理设备映射、权限、错误处理、缓冲可由不同层承担。单/双/循环缓冲影响并行与等待；中断服务结束时数据常先在内核缓冲，未必已经复制到用户缓冲。

02 例子

键盘驱动处理中断数据后，应用仍可能需通过系统调用取得它。

03 易错点

错误说法设备驱动等于所有应用程序的读写函数；缓冲完全不消耗内存。

自检：两块缓冲能否同时用于填入和消费?

可以按正确同步交替使用，但不能未经协调读写同一缓冲。

磁盘时延与 SSD：机械部件的公式不通用　HDD 平均旋转等待常为半圈；SSD 不套机械寻道公式。　　记忆概念　易错　易混　　要点

核心结论

机械盘访问常含寻道、旋转等待和传输时间；SSD 无同样机械寻道，但有闪存擦写、映射和垃圾回收等开销。

别混淆HDD 平均旋转等待常为半圈；SSD 不套机械寻道公式。

详解

01 解析

$t_{\text{rot,avg}} = \text{s}$

机械盘均匀位置模型，r 为 rpm 转速；7200 rpm 时约为 4.17 ms。

1. 转速先从 rpm 除以 60 换成圈/s，再取倒数得到一整圈时间。
2. 均匀位置模型的平均旋转等待为半圈；加上平均寻道和实际数据传输时间才是完整访问时延。
3. 7200 rpm 对应 120 圈/s，半圈约 4.17 ms。SSD 没有机械旋转，不能给它照加此项。

补充原理与条件

平均旋转等待常为半圈，转速单位需转换。调度仍涉及访问模式和设备内部行为；不要把 HDD 的距离模型无条件用于 SSD。

02 例子

转速 7200 rpm 等于 120 圈/s，一圈约 8.33 ms，平均等待约 4.17 ms（均匀位置模型）。

03 易错点

错误说法SSD 的访问时延也必须按磁头寻道加旋转等待计算。

自检：“传输速率高”是否证明小随机 I/O 总时延很低?

不能，定位/控制等固定开销可能占主要部分。

408 · NET

计算机网络

73 条 · 6 个章节

网络体系结构

OSI/TCP-IP 分层与封装　OSI 七层，TCP/IP 可按四/五层讲，别混同层名。　记忆概念　易错　易混　要点

核心结论

常用五层：应用、传输、网络、数据链路、物理；封装逐层增加所需控制信息，接收端按协议解封装。

别混淆OSI 七层，TCP/IP 可按四/五层讲，别混同层名。

详解

01 解析

OSI 为七层，TCP/IP 教材可用四层/五层模型，不应因名称不同就认为协议所属层不同。典型路由器主要网络层转发，普通二层交换机主要链路层转发。

02 例子

TCP 段封装进 IP 数据报，再进链路帧，通过比特信号发送。

03 易错点

错误说法交换机、路由器和 TCP 都工作在同一层；每层都把数据叫帧。

自检：路由器普通转发是否必须解析完整 HTTP 请求?

不需要，转发依据主要是网络层相关信息。

协议、服务、接口与实体　协议是对等规则，服务向上提供，接口供邻层调用。　记忆概念　易错　易混　要点

核心结论

协议约定对等实体通信规则；服务是本层向上层提供的能力；接口描述相邻层如何调用该服务。

别混淆协议是对等规则，服务向上提供，接口供邻层调用。

详解

01 解析

服务由下层向上层提供，而协议在对等层之间逻辑执行；实际数据经下层传输。协议的语法、语义、时序是不同内容。

02 例子

TCP 向应用提供可靠字节流，是服务；TCP 首部与状态转换规则属于协议。

03 易错点

错误说法同层实体之间的规则叫接口，邻层提供的功能叫协议。

自检：面向连接服务必然要求底层网络也面向连接吗?

不必，TCP 可在无连接 IP 上实现连接服务。

电路、报文和分组交换：建立与存储转发 分组可流水化，但首部、排队等开销仍要计入。 记忆概念 易错 易混 要点

核心结论

电路交换先建立专用通路；报文交换完整报文逐跳存储转发；分组交换分块转发，可流水化并统计复用。

别混淆分组可流水化，但首部、排队等开销仍要计入。

详解

01 解析

电路需建连但后续传输可少逐跳存储；分组有首部、排队和丢失重排代价。比较时统一文件大小、分组大小、链路速率与存储转发约定。

02 例子

一条长报文拆成多包后可在不同链路重叠传送，而不是每跳都等待全文件。

03 易错点

错误说法分组交换不需排队，也总比分组之外所有方式耗时短。

自检：只给“分组交换”能否断言其端到端时延总最小?

不能，需要具体参数和流量条件。

各层协议总览：五层职责、数据单位与设备 分清功能、数据单位、地址：端口属于传输层，IP 地址属于网络层，MAC 地址用于链路层。 记忆概念 易混 要点

核心结论

按五层记忆：应用层服务进程，传输层提供进程间通信，网络层实现跨网络的主机间分组传送，链路层传送相邻结点间的帧，物理层传送比特。

别混淆分清功能、数据单位、地址：端口属于传输层，IP 地址属于网络层，MAC 地址用于链路层。

详解

01 解析

层次是功能抽象。以太网和 IEEE 802.11 都同时包含物理层与链路层规范，不能把整个标准只放进某一层。网卡通常涉及物理层和链路层；普通路由器转发数据时处理到网络层，也实现下两层。设备可具有更高层功能，应依题设判断。

02 例子

PPP 点到点链路不需要像以太网那样通过 MAC 地址选择多个接收者。

03 易错点

错误说法每层都用端口寻址；所有链路都必须有 MAC 地址。

04 图表

要点对照

层次	核心职责 / 数据单位	典型协议或标准	寻址与典型设备
应用层	定义应用报文与交互规则 / 报文	HTTP、DNS、FTP、SMTP、POP3、IMAP、DHCP、SNMP	域名、资源名等应用标识；代理服务器等
传输层（运输层）	复用分用；按协议提供可靠性、流量及拥塞控制 / TCP 段、UDP 数据报	TCP、UDP	端口；端系统协议栈
网络层	逻辑寻址、路由选择、分组转发 / IP 数据报（分组）	IPv4、IPv6、ICMP、IGMP；路由选择协议	IP 地址；路由器、三层交换机
数据链路层	成帧、差错检测、介质访问 / 帧	以太网 MAC、802.11 MAC、PPP、HDLC、STP、802.1Q	以太网等使用 MAC；网桥、二层交换机
物理层	机械、电气、功能和规程特性；编码、调制 / 比特	以太网 PHY、802.11 PHY、RS-232 等接口标准	信号与介质；中继器、集线器

参考资料

参考：RFC 1122；RFC 1123

自检：相邻路由器之间传帧与源、目的主机之间传 IP 分组分别是哪层功能?

前者是链路层；后者是网络层。

OSI 七层与 TCP/IP 四层：模型对应与封装　OSI 是七层参考模型；408 常用五层讲解；TCP/IP 常用四层。不要混用层号。　　记忆概念　易混　　要点

核心结论

OSI 的应用、表示、会话三层，在 TCP/IP 中由应用层承担相关功能；五层模型把 TCP/IP 网络接口层拆为链路层和物理层。

别混淆OSI 是七层参考模型；408 常用五层讲解；TCP/IP 常用四层。不要混用层号。

详解

01 解析

发送端逐层封装，接收端逐层解封装。典型顺序为应用报文 → TCP 段或 UDP 数据报 → IP 数据报 → 帧 → 比特流。链路层通常加首部和尾部；网络层、传输层加各自首部。TLS 提供加密和认证，但在实际 TCP/IP 中不因此成为独立的 OSI 表示层协议。

02 例子

普通路由转发每一跳重新封装链路帧；不经 NAT、隧道等处理时，源、目的 IP 通常不变，但 TTL 等字段会改变。

03 易错点

错误说法TCP/IP 应用层不涉及表示或会话功能；TLS 必须单独归入表示层。

04 图表

要点对照

OSI 七层（自上而下）	五层模型	TCP/IP 四层	主要特点
7 应用层	应用层	应用层	向应用进程提供网络应用服务
6 表示层	应用层	应用层	数据表示转换、压缩、加密等功能
5 会话层	应用层	应用层	会话建立、管理、同步等功能
4 传输层	传输层	传输层	端到端的进程通信
3 网络层	网络层	网际层	跨网络寻址与转发
2 数据链路层	数据链路层	网络接口层（链路层）	相邻结点间帧传送
1 物理层	物理层	网络接口层（链路层）	比特流传输

参考资料

参考：RFC 1122；RFC 1123

自检：TCP/IP 四层中的网络接口层对应五层模型哪两层?

数据链路层与物理层。

协议归层与号码辨析：端口号、IP 协议号、EtherType　UDP 端口 89 与 IP 协议号 89 不是一回事；ARP 没有 TCP/UDP 端口。　　记忆概念　易混　　要点

核心结论

端口号在 TCP/UDP 等传输层首部中标识进程；IP 协议号（IPv6 为下一首部字段）区分上层协议；EtherType 区分以太网载荷类型。

别混淆UDP 端口 89 与 IP 协议号 89 不是一回事；ARP 没有 TCP/UDP 端口。

详解

01 解析

1. 从外向内逐层检查分用字段：以太网 EtherType 决定帧中是 IPv4、ARP 等哪类载荷。
2. 进入 IPv4 后，协议号决定交给 TCP、UDP、OSPF 等哪个上层模块；进入 TCP/UDP 后再按端口定位服务。
3. 例如 0x0800→6→443 依次表示 IPv4、TCP、常用 HTTPS 服务端口。不能把端口 443 写成 IP 协议号，也不能只凭 443 断言加密协商必定成功。

补充原理与条件

典型浏览网页过程：需要时 DHCP 获取配置 → DNS 解析域名 → 路由确定下一跳 → ARP（IPv4）或 NDP（IPv6）解析邻居 → TCP+TLS 或 QUIC 建立通信 → HTTP 请求响应。该过程受缓存、静态配置和连接复用影响，并非每次都完整重复。交换机按二层地址转发，路由器按 IP 前缀查表；NAT/NAPT 会改变相应 IP/端口和校验字段。

02 例子

以太网载荷为 IPv4（0x0800），IP 协议号为 6，TCP 目的端口为 443，分别代表三个不同层次的分用。

03 易错点

错误说法网络协议都可用端口号区分；每次 HTTP 请求都必须重新 DHCP、DNS 和握手。

04 图表

要点对照

字段 / 标识	常见值	用途与边界
以太网 EtherType	IPv4 0x0800；ARP 0x0806；IPv6 0x86DD	说明帧载荷是什么；不是端口

字段 / 标识	常见值	用途与边界
IPv4 协议号 / IPv6 下一首部	ICMP 1；IGMP 2；TCP 6；UDP 17；OSPF 89；ICMPv6 58	IPv6 下一首部也可能指向扩展首部；不是服务端口
TCP / UDP 服务端口	DNS 53；HTTP 80；HTTPS 常用 443；RIP UDP 520；BGP TCP 179	TCP 和 UDP 端口空间分别独立；需同时注明传输协议
MAC / IP / 端口	链路接口标识 / 网络地址 / 进程复用分用	跨路由转发通常更换帧 MAC；无 NAT 等情况下端到端 IP 和端口通常保持

参考资料

参考：RFC 791；RFC 8200；RFC 826；RFC 1122

自检：ICMP 是端口 1 还是 IP 协议号 1?

IPv4 中 ICMP 使用 IP 协议号 1，没有 TCP/UDP 端口。

物理层与链路层

发送时延、传播时延与吞吐率　速率影响发送时延；距离影响传播时延。　　易错　易混　近年考查　　要点

核心结论

发送时延=帧长/链路速率；传播时延=距离/信号传播速度。吞吐率是实际完成的传输率。

别混淆速率影响发送时延；距离影响传播时延。

详解

01 解析

$t_{send} = \frac{L}{R}, t_{prop} = \frac{d}{v}$

L 用 bit，R 用 bit/s；d 为距离，v 为传播速度，注意与排队/处理时延区分。

1. 帧长 L bit、速率 R bit/s：L/R 是第一位开始发出到最后一位发出的时间。
2. 距离 d、传播速度 v：d/v 是同一位从链路一端到另一端的时间，和帧长无关。
3. 单链路、忽略处理与排队时，最后一位到达耗时 L/R+d/v；提高 R 只缩短前一项。

补充原理与条件

加大链路速率减少把比特推入链路的时间，不直接提高介质中的传播速度。还要区分排队和处理时延。

02 例子

光纤长度固定时，从 100 Mb/s 升到 1 Gb/s 不会把传播时延降为原来的十分之一。

03 易错点

错误说法带宽越大，传播时延一定越短。

自检：最后一位到达接收端至少要考虑哪两段时间?

发送时延与传播时延；具体题设还可能有其他时延。

奈奎斯特与香农：不同限制共同作用　信噪比先由 dB 转线性值；两种上限均适用时取较小者。　　易混　近年考查　　要点

核心结论

理想低通信道奈奎斯特 $R\leq 2B\log_2 M$；含噪声香农 $C=B\log_2(1+S/N)$，B 用 Hz，信噪比代入线性值。

别混淆信噪比先由 dB 转线性值；两种上限均适用时取较小者。

详解

01 解析

$R\leq 2B\log_2 M$

理想无噪声低通信道；B 为带宽，M 为码元状态数。

$C=B\log_2\left(1+\frac{S}{N}\right)$

带噪声信道；S/N 必须使用线性比值，dB 值先换算。

1. 先换单位：B 用 Hz，信噪比若给 dB，先算 $S/N=10^{\mathrm{dB}/10}$。
2. 无噪声低通信道的码元/调制限制用 $2B\log_2 M$；带噪声容量限制用 $B\log_2(1+S/N)$。
3. 两类限制都适用时取同时满足的上限，不把码元数 M 当作香农公式里的信噪比。

补充原理与条件

一个限制码元速率与调制等级，一个限制带噪声信道的信息容量。若两者条件均给出，应同时满足；有带通等模型须按题设。

02 例子

信噪比 30 dB 对应线性比 1000，不能把 30 直接代入香农式。

03 易错点

错误说法用任意一个公式就能得到最终可达速率。

自检：信噪比 dB 如何转线性值?

$S/N=10^{\mathrm{dB}/10}$。

CRC 与纠错：检错不等于自动纠正　CRC 余数为 0 只代表未检出错误，不保证绝对无错。　冷门　易混　要点

核心结论

通常 CRC 用于检测，接收端检查余数；检出能力取决于生成多项式与差错模式。纠错需要额外编码机制。

别混淆CRC 余数为 0 只代表未检出错误，不保证绝对无错。

详解

01 解析

模 2 除法使用异或，不进位、不借位。r 次生成多项式对应 r 位 CRC；余数为 0 也不能无条件证明绝无差错。

02 例子

某些错误多项式是生成多项式的倍数，可能检测不出。

03 易错点

错误说法CRC 可以自动修复所有单比特错误。

自检：CRC 余数为 0 就能证明数据绝对正确吗?

不能，只表示该校验未检测到错误。

GBN 与 SR：接收窗口和序号空间　GBN 接收窗为 1；等大 SR 窗口不超过半个序号空间。　易混　易错　近年考查　记忆概念　要点

核心结论

GBN 通常接收窗口为 1；SR 可缓存失序帧。k 位序号：GBN 发送窗口$\leqslant 2^{k}-1$；等大发送/接收窗口的 SR$\leqslant 2^{k-1}$。

别混淆GBN 接收窗为 1；等大 SR 窗口不超过半个序号空间。

详解

01 解析

$W_t+W_r\leq 2^k$

SR 的序号空间约束；发收等窗时 W$\leqslant 2^{k-1}$，GBN 的接收窗为 1。

1. k 位序号只有 2^k 个不同值，窗口不能大到让重传旧帧与下一轮新帧无法区分。
2. GBN 接收窗口为 1，发送窗口最多 2^k-1；SR 若发收等大，则 $2W\leq 2^k$。
3. 3 位 SR 若 W=5，旧窗口与回绕后的新窗口会重叠；应取 W$\leqslant$4。不同大小的 SR 窗口要分别相加，不能一律各自除以二。

补充原理与条件

SR 一般需要发送窗口+接收窗口≤序号空间，避免新旧帧混淆。窗口上限不是实际吞吐率，超时重传策略也不同。

02 例子

3 位序号的等大 SR 窗口最多 4，而 GBN 发送窗口最多 7。

03 易错点

错误说法两种协议的接收窗口都与发送窗口相同。

考点联系

选择重传的独立确认，不等于发送窗可以越过缺口。

变式

自编模拟变式 · 收到后续 ACK 后能否跳过缺口

3 位序号、等大发收窗口的 SR，初始发送窗为 0～3；只收到 ACK2、ACK3，能否新发序号 4?

答案与解析

不能。窗口左边界仍是未确认的 0，收到后续独立 ACK 只标记相应帧，不能越过 0、1 滑动发送窗。

关键区别：窗口大小不等于“未确认帧的数量上限可以任意补齐”；新帧序号也必须位于当前窗口。

自检：GBN 收到正确但失序的帧通常如何处理?

丢弃，并按协议重复确认此前连续收到的帧。

交换机、集线器与全双工以太网　交换机隔离碰撞域；同 VLAN 通常仍共用广播域。　　冷门　易混　近年考查　　要点

核心结论

集线器共享碰撞域；普通二层交换机隔离端口碰撞域，但同 VLAN 通常同属广播域。全双工点对点链路无需 CSMA/CD。

别混淆交换机隔离碰撞域；同 VLAN 通常仍共用广播域。

详解

01 解析

广播域通常靠 VLAN 或路由器划分。CSMA/CD 属于共享半双工以太网的冲突检测机制；无线通常采用不同机制。

02 例子

两个交换机端口属于同一 VLAN，广播帧仍可被转发到另一端口。

03 易错点

错误说法交换机每个端口划分一个广播域；全双工仍靠 CSMA/CD。

自检：普通二层交换机是否天然隔离广播域?

不是，需要 VLAN 等机制。

CSMA/CA 与 RTS/CTS：隐藏站与 NAV　RTS/CTS 缓解隐藏站，不能保证完全无碰撞。　冷门　易混　近年考查　　要点

核心结论

802.11 采用碰撞避免、确认及退避；RTS/CTS 可缓解隐藏站问题，其他站利用持续时间字段设置 NAV。

别混淆RTS/CTS 缓解隐藏站，不能保证完全无碰撞。

详解

01 解析

隐藏站未必能听见发送端，但可能听见接收端的 CTS。SIFS 通常短于 DIFS，便于 ACK 等响应优先发送；RTS 本身仍可能碰撞。

02 例子

B 听不到 A 却听到 AP 的 CTS，可以据 NAV 延迟发送，避免干扰 A→AP。

03 易错点

错误说法无线网络也边发送边可靠检测碰撞；RTS/CTS 能消除一切碰撞。

考点联系

NAV 要按观察时刻统计尚未完成的交换。

变式

自编模拟变式 · 从 CTS 结束时刻计算 NAV

自编参数：数据帧 1500 B、速率 12 Mb/s，SIFS=16 μs、ACK 传输 20 μs，忽略传播。从 CTS 接收完毕起还应保留多久?

答案与解析

数据发送用时 1000 μs；剩余顺序为 SIFS、DATA、SIFS、ACK，共 1052 μs。

关键区别：RTS、CTS 已经完成，DIFS 属于更早的竞争阶段，均不重复加入这段剩余预约。

自检：RTS/CTS 是否保证完全无冲突?

不保证，主要缓解特定竞争问题。

码元、比特、编码与调制　M 种码元理想承载 $\log_2 M$ bit，波特率不等于比特率。　记忆概念　易错　易混　要点

核心结论

码元率以 Baud 计，信息速率以 bit/s 计；M 种可区分符号可承载 $\log_2 M$ bit/码元。编码与调制分别组织基带表示或载波特性。

别混淆M 种码元理想承载 $\log_2 M$ bit，波特率不等于比特率。

详解

01 解析

ASK 改幅度、FSK 改频率、PSK 改相位；Manchester 每位中部有跳变便于时钟提取，速率与带宽需依题设口径。不能把波特率直接当比特率。

02 例子

4 种符号可表示 2 bit，因此 1000 Baud 理想信息率为 2000 bit/s（未另计开销）。

03 易错点

错误说法一个码元始终只有 1 bit；Manchester 完全不需要更多信号变化。

自检：8 种符号理想可承载几 bit/码元?

3 bit/码元。

单工、半双工、全双工与传输介质　全双工允许双向同时，但不自动翻倍单向吞吐。　记忆概念　易错　易混　要点

核心结论

单工仅单方向；半双工双向但不能同时；全双工允许双向同时。双绞线、同轴、光纤、无线在带宽、干扰与距离方面不同。

别混淆全双工允许双向同时，但不自动翻倍单向吞吐。

详解

01 解析

光传播仍有有限速度，链路距离产生传播时延。复用用于共享资源；全双工不是将一条流自动翻倍为两倍单向吞吐。

02 例子

对讲机常为半双工，不能双方同时讲话；全双工点对点以太网可同时发收。

03 易错点

错误说法全双工等于一根线永远只有一个方向；光纤完全没有传播时延。

自检：抗电磁干扰强是否等于光纤不存在任何衰减和距离限制?

不等于，仍有实际物理限制。

信道复用：FDM、TDM、WDM 与 CDM　统计 TDM 动态分配；码分须检验码片正交条件。　　记忆概念　易错　易混　　要点

核心结论

频分分不同频带，时分分时隙，波分分光波长，码分利用可区分码序列。统计时分按需求动态组织。

别混淆统计 TDM 动态分配；码分须检验码片正交条件。

详解

01 解析

正交码的内积用于分离信号，映射 0/1 到 ±1 及是否归一化需按题设。不是所有随机码字都满足正交；保护带/时隙标识等开销降低有效利用率。

02 例子

普通同步 TDM 用户闲置时仍可能占固定槽，统计 TDM 可把槽给活跃用户。

03 易错点

错误说法统计时分复用给每个用户永久固定时隙；CDM 任意码片都正交。

自检：WDM 常用于什么介质?

光纤，以不同光波长复用。

成帧：字符填充、比特填充与透明传输　比特填充：数据连续五个 1 后插 0，接收端再去掉。　　记忆概念　易错　易混　　要点

核心结论

透明传输让任意数据不与边界控制混淆；字符填充用转义，典型比特填充在连续五个 1 后插 0。

别混淆比特填充：数据连续五个 1 后插 0，接收端再去掉。

详解

01 解析

发送端填充、接收端解除，填充位不属于原数据。PPP 等协议还需按实际链路和帧规则区分；不能在遇四个 1 时就插位。

02 例子

原数据中的 01111110 可以通过填充避免被误认为 HDLC 标志。

03 易错点

错误说法数据中出现帧界定符，就必须禁止该数据发送。

自检：接收端去掉填充位后，是否改变原始业务数据?

不应，正确解除恢复原数据。

ARQ：检错、确认、序号与重传　ACK 丢失会重传，序号用于去重，CRC 不负责重传。　记忆概念　易错　易混　　要点

核心结论

ARQ 用检错配合 ACK、序号和重传计时提供可靠传送；停止等待、GBN、SR 有不同窗口与重传范围。

别混淆ACK 丢失会重传，序号用于去重，CRC 不负责重传。

详解

01 解析

停止等待要区分新帧和重传，常用交替位；ACK 丢失会导致重传，接收端需去重。丢数据、丢 ACK、超时过早的时间线要分别画。

02 例子

接收端已收到帧 0，但 ACK 丢失；再次收到帧 0 不能重复交付。

03 易错点

错误说法CRC 检出错误就由 CRC 本身重新发送帧；停止等待可以不编号。

自检：接收端的序号判定只用于检查 CRC 吗?

不是，还用于识别新帧、重复与顺序。

交换机自学习：学源 MAC，转发看目标 MAC　学源 MAC 与入端口，按目标 MAC 决定转发。　　记忆概念　易错　易混　　要点

核心结论

自学习通常用源 MAC 和入端口建立映射；已知目标按表转发，未知单播在相应广播域/VLAN 内泛洪。

别混淆学源 MAC 与入端口，按目标 MAC 决定转发。

详解

01 解析

若目标就在入端口关联网段可过滤。表项有老化机制，搬迁后可更新；泛洪不等于路由器跨网广播，也不能无限循环而不控制二层环路。

02 例子

H1 从端口 1 发往未知 H2，交换机先学 H1→1，再泛洪寻找 H2。

03 易错点

错误说法交换机根据目标 MAC 学习发送端位置，未知单播直接丢弃。

自检：未知目标时是否先把目标 MAC 绑定到入端口?

不应，入端口关联的是本次帧源端。

VLAN、Trunk 与广播域　VLAN 是二层域，IP 子网是三层范围，不能互相替代。　　记忆　概念　易错　易混　　要点

核心结论

VLAN 逻辑划分二层广播域；Trunk 可承载多个 VLAN 并标记所属关系；跨 VLAN 通常需三层转发。

别混淆VLAN 是二层域，IP 子网是三层范围，不能互相替代。

详解

01 解析

IP 子网与 VLAN 是不同层的组织方式，常配套但不能互相替代。端口成员、标签与路由决定可达性；同交换机上的两个 VLAN 也可隔离广播。

02 例子

H1、H2 配相同 IP 前缀但在隔离 VLAN 中，普通二层 ARP 不必可达对方。

03 易错点

错误说法只要 IP 同网段，就可以跨不同 VLAN 直接通过二层广播 ARP。

自检：一个 VLAN 必须只包含一台物理交换机的端口吗?

不必，可通过相应链路跨交换机延伸。

物理层标准速查：比特、接口、编码与调制　编码/调制是技术，接口/PHY 是标准；不要把 CRC、成帧、MAC 寻址归入物理层。　　记忆概念　易混　　要点

核心结论

物理层解决如何在介质上传送比特，规定接口和信号特性，不解释 IP 地址、端口或应用报文。

别混淆编码/调制是技术，接口/PHY 是标准；不要把 CRC、成帧、MAC 寻址归入物理层。

详解

01 解析

物理层的四类特性常概括为机械、电气、功能、规程。码元是信号单位，比特是信息单位；若每码元承载 $\log_2 M$ 比特，理想条件下比特率等于波特率乘 $\log_2 M$。光纤、双绞线和无线信道是传输介质，不是应用协议。

02 例子

使用 16 种码元状态且每码元编码 4 比特时，理想 1000 Baud 对应 4000 bit/s。

03 易错点

错误说法集线器能够按目的 MAC 转发帧。

04 图表

要点对照

标准或技术	作用与特点	考点提醒
IEEE 802.3 PHY	规定以太网物理介质、信号和速率等	802.3 还包含 MAC，整个以太网不只属于物理层
IEEE 802.11 PHY	规定无线局域网物理信号传输	CSMA/CA 位于 MAC 子层
RS-232	串行通信接口标准，规定电气等接口特性	属于接口标准，不是 TCP/IP 应用协议
编码与调制	编码组织基带信号；调制改变载波参数以承载信息	比特率与波特率不能直接等同
中继器、集线器	再生或转发物理信号	不识别 MAC；集线器不隔离冲突域

自检：物理层是否负责检查以太网帧 CRC?

不负责；帧的 CRC 检验属于链路层功能。

PPP 工作阶段：静止、建立、认证、网络层协议、终止　LCP 建链，NCP 配网络层；认证可跳过。　　记忆概念　易混　图解　　要点

核心结论

PPP 主阶段为 Dead→Establish→可选 Authenticate→Network→Terminate→Dead；教材可把 Network 中的网络层配置与链路打开单独画出。

别混淆LCP 建链，NCP 配网络层；认证可跳过。

详解

01 解析

LCP 建立、配置、测试和终止链路；认证可选；NCP 家族配置相应网络层协议，例如 IPCP 配置 IPv4。必须区分“PPP 工作阶段”和 LCP 更细的状态机，不要求所有实现采用教材的同一张简图。

02 例子

物理层就绪后由 LCP 协商；若需要认证，则认证成功后再进行网络层协议配置。

03 易错点

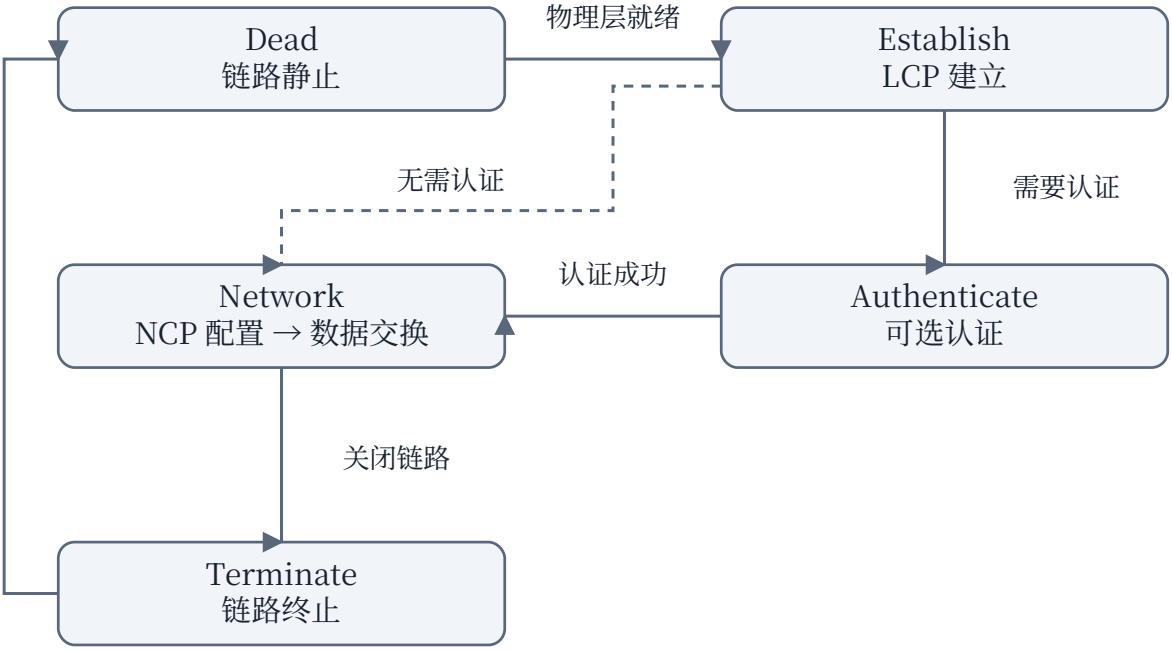
错误说法PPP 的认证每次必需；LCP 与 NCP 是同一个协议。

04 图表

要点对照

阶段	中文名称	主要工作
Dead	链路静止	物理层未准备好
Establish	链路建立	LCP 协商
Authenticate	认证（可选）	验证对端身份
Network	网络层协议阶段	NCP 配置，配置成功后交换相应数据
Terminate	链路终止	LCP 终止，回到静止

PPP：LCP 建链，NCP 配网络层



认证可跳过；任意阶段发生失败或断链可终止，图中只画主路径。

PPP 主流程为静止、建立、可选认证、网络层协议、终止。无需认证可从建立直接到网络层协议阶段。网络层协议阶段先由 NCP 配置，成功后交换相应数据。单独查看图片

参考资料

参考：RFC 1661 §3：PPP 工作阶段

自检：LCP 协商成功是否就一定能立即传 IP 数据?

不一定，还可能需要认证及相应 NCP 配置。

介质访问阶段：CSMA/CD 退避与 CSMA/CA 等待　无线退避遇忙冻结；SIFS 比 DIFS 短，优先响应。　　记忆概念　易混　图解　　要点

核心结论

CSMA/CD：监听→发送并检测→碰撞后强化信号、停止→退避重试；CSMA/CA 使用帧间间隔、随机退避及确认等降低碰撞概率。

别混淆无线退避遇忙冻结；SIFS 比 DIFS 短，优先响应。

详解

01 解析

1. 无线退避期间只有空闲时隙才让计数递减；一听到忙就冻结剩余值。
2. 信道重新连续空闲 DIFS 后，恢复原来的剩余计数；不是每经过一个空闲时隙都重新等 DIFS，也不是每次都重新抽随机数。
3. ACK/CTS 等响应使用更短的 SIFS 优先获得信道。这里是常见 DCF 模型，不能套用以太网 CSMA/CD 的碰撞后流程。

补充原理与条件

以常见 DCF 模型记忆：DIFS 用于普通竞争，SIFS 用于 ACK/CTS 等优先响应，SIFS 比 DIFS 短；无线退避计数在信道忙时冻结。二进制指数退避与无线退避规则要分别按题设使用。

02 例子

无线站等待退避时信道变忙，应冻结剩余计数，待满足空闲间隔后继续。

03 易错点

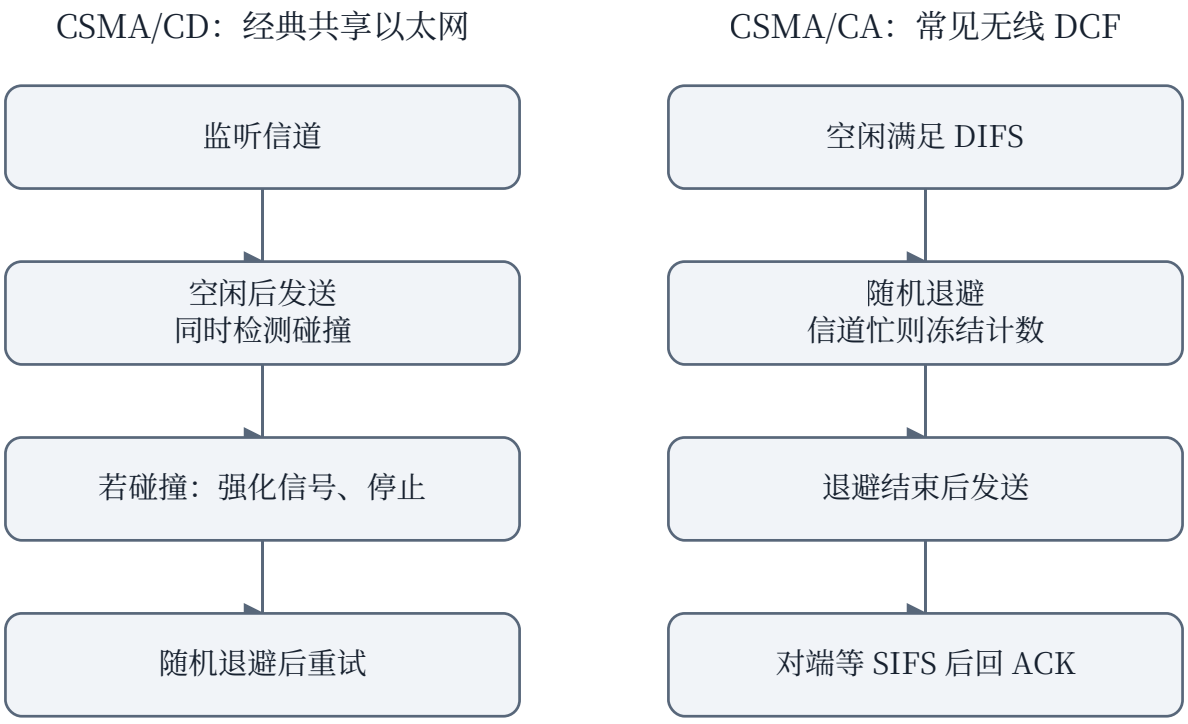
错误说法CSMA/CA 发送时也靠检测碰撞中止；DIFS、SIFS 是 TCP 状态。

04 图表

要点对照

名称	含义	易混点
CSMA/CD	载波监听多路访问/碰撞检测	经典共享半双工以太网；全双工不使用碰撞检测竞争
CSMA/CA	载波监听多路访问/碰撞避免	不保证绝无碰撞
SIFS	短帧间间隔	ACK、CTS 等优先响应
DIFS	DCF 帧间间隔	普通竞争等待
Backoff	随机退避	无线信道忙时冻结计数

介质访问：检测碰撞与避免碰撞



CA 不保证绝无碰撞；SIFS < DIFS；全双工交换以太网不使用 CD 竞争。

左边为 CSMA/CD 监听、发送并检测、碰撞后停止、退避重试；右边为常见无线 DCF 的 DIFS、退避、发送、SIFS 后 ACK。无线退避在忙时冻结。单独查看图片

变式

自编模拟变式 · 冻结的是剩余计数

退避还剩 5 个时隙时信道变忙；之后连续空闲达到 DIFS。此时重新抽随机数、重数 DIFS，还是恢复剩余值?

答案与解析

恢复剩余 5 个空闲时隙的计数；后续每过一个空闲时隙递减，不在每个时隙前重新插 DIFS。

关键区别：再次忙时再次冻结，新的 DIFS 用于忙后恢复竞争；它不是每个倒计时时隙的附加开销。

自检：SIFS 与 DIFS 谁更短，目的是什么?

SIFS 更短，使应答等获得优先发送机会。

链路层协议速查：以太网、Wi-Fi、PPP 与可靠传输 CSMA/CD 用于传统共享半双工以太网；CSMA/CA 用于无线局域网；全双工交换式以太网不使用 CSMA/CD。 记忆概念 易混 要点

核心结论

链路层负责相邻结点间帧传送；检错、可靠交付和介质访问是不同功能，并非所有链路协议都提供确认重传。

别混淆CSMA/CD 用于传统共享半双工以太网；CSMA/CA 用于无线局域网；全双工交换式以太网不使用 CSMA/CD。

详解

01 解析

CSMA/CA 基本 DCF 中，退避计数只在空闲时隙递减；信道变忙时冻结，重新连续空闲 DIFS 后继续剩余计数，而不是每次都重新抽签。SIFS 用于 ACK、CTS 等优先响应。链路可靠性只覆盖本段链路，不能代替端到端可靠性。

02 例子

退避计数剩 5 时信道变忙；再次连续空闲 DIFS 后从剩余 5 继续。

03 易错点

错误说法CRC 能自动纠错；Wi-Fi 退避冻结后必须重新随机选数。

04 图表

要点对照

协议 / 机制	特点	408 易错点
以太网 MAC（IEEE 802.3）	MAC 寻址、CRC 检错；传统共享介质采用 CSMA/CD	基本 MAC 服务不通过逐帧 ACK/重传保证可靠交付
Wi-Fi MAC（IEEE 802.11）	CSMA/CA；典型单播有 ACK 和重传，可选 RTS/CTS	不边发边检测碰撞；RTS/CTS 缓解隐藏站，不保证无碰撞
PPP	点到点；LCP 建链与协商，NCP 配置网络层协议，认证可选	检错但不提供链路层可靠重传；异步链路字节填充，同步链路比特填充
HDLC	面向比特，使用标志字段和比特填充	具体帧格式、控制方式以题设为准
STP（IEEE 802.1D）	生成无环逻辑拓扑，防止二层转发环路	不是 IP 路由选择协议
IEEE 802.1Q	VLAN 标记；Trunk 承载多个 VLAN	VLAN 划分广播域；跨 VLAN 通常需三层转发
停止等待 / GBN / SR	可靠传输机制；利用序号、确认、超时与重传	GBN 接收窗口通常为 1；SR 缓存失序帧并选择重传

参考资料

参考：RFC 1661；RFC 1662

自检：PPP 是否使用 TCP 式超时重传保证帧可靠交付?

不使用；PPP 提供检错，不提供这种链路层可靠重传。

网络层

ARP：查下一跳 MAC，不一定查目标主机 跨网发帧先找网关 MAC，目的 IP 仍指远端。 易错 易混 近年考查 要点

核心结论

先根据路由决定下一跳，再用 ARP 获取本链路上下一跳 IPv4 地址对应的 MAC。

别混淆跨网发帧先找网关 MAC，目的 IP 仍指远端。

详解

01 解析

1. 先将目的 IP 与本机掩码结合，判断目标是否在本地子网，并按路由表确定下一跳。
2. 同网段才查询目标主机 MAC；跨网段查询所选下一跳的 MAC，通常是默认网关。
3. 链路帧填下一跳 MAC，IP 首部仍填最终目的 IP；若无 NAT，不因换了一跳就把 IP 目的地址改成网关。

补充原理与条件

目标不同网段时下一跳通常是网关。路由器每跳重封装链路层首部；通常端到端 IP 地址保持，NAT 是另一个机制。

02 例子

访问远端网站时，局域网帧的目标 MAC 通常是网关 MAC，IP 目标仍是网站地址。

03 易错点

错误说法跨网通信时 ARP 获取远端主机的 MAC。

自检：ARP 的广播能否被路由器普通转发到整个互联网?

不能，通常限于本地广播域。

最长前缀匹配与默认路由 先匹配目的地址，再选最长前缀。 易错 要点

核心结论

在所有匹配目的地址的路由中，选择前缀最长者；默认路由 /0 是最宽泛的后备项。

别混淆先匹配目的地址，再选最长前缀。

详解

01 解析

1. 对每条候选前缀检查“目的地址按该掩码与运算”是否等于路由前缀；不匹配的先排除。
2. 只在匹配项中选最长前缀，不比较目的网段数值大小，也不先选默认路由。
3. 选中条目后再读下一跳及出接口。/0 只在没有更具体匹配项时作为后备。

补充原理与条件

先检查地址是否匹配，再比较前缀长度。下一跳和出接口可能单独指定，不能把目的网段地址直接当下一跳。

02 例子

10.1.2.3 同时匹配 10.0.0.0/8 与 10.1.2.0/24，选择 /24。

03 易错点

错误说法选路时选择数值最大的掩码或第一条匹配。

自检：更长前缀但不匹配目的地址的路由能否被选择?

不能。

IPv4 分片：偏移单位与首部 总长度含首部；片偏移按原数据部分每 8 B 编号。 易错 易混 要点

核心结论

片偏移单位为 8 字节；总长度含该片 IP 首部；除最后一片外的数据部分通常是 8 的倍数。

别混淆总长度含首部；片偏移按原数据部分每 8 B 编号。

详解

01 解析

1. 先用原 IP 总长度减原首部长度，得到需要切分的数据字节数。
2. 每个非末片的数据容量取 $8\left\lfloor \frac{MTU-H}{8} \right\rfloor$，其中 H 表示该片首部字节数；每片总长度还须加自己的首部。
3. 片偏移=该片数据起点相对原始数据部分的字节偏移/8。只有最后一片 MF=0；不要把各片首部算进累计数据偏移。

补充原理与条件

偏移相对原始 IP 数据部分，不能把首部计入偏移。MF=1 表示后面还有片，最后片 MF=0；重组通常在目的主机完成。

02 例子

片偏移 185 对应原数据部分从 0 计的偏移 1480 B，不是 185 B。

03 易错点

错误说法片偏移以字节计，分片总长度不含首部。

自检：首片的 MF 一定是 0 吗?

不是；若后面还有分片，首片 MF=1。

TTL、首部校验与 ICMP 差错　IPv4 校验仅覆盖首部；TTL 变化需要更新校验。　冷门　易混　要点

核心结论

IPv4 首部校验和仅覆盖首部；转发时 TTL 递减导致首部校验需更新。ICMP 差错有发送限制。

别混淆IPv4 校验仅覆盖首部；TTL 变化需要更新校验。

详解

01 解析

TTL 归零时通常丢弃并可产生超时差错；不是所有数据包都允许返回差错，例如不对另一条 ICMP 差错再生成差错。IPv6 不使用同样首部校验和。

02 例子

TCP 校验覆盖 TCP 首部与数据及伪首部，而 IPv4 的校验不覆盖 IP 负载。

03 易错点

错误说法IPv4 首部校验和覆盖用户数据；每次丢包都发 ICMP。

自检：修改 IPv4 TTL 后能否原样保留首部校验和?

不能，需要相应更新。

距离向量、链路状态与路由环路　距离向量不是完整拓扑；毒性逆转不能消除所有环路。　冷门　易混　要点

核心结论

距离向量交换距离及下一跳相关信息；链路状态发布本地链路状态，节点建立拓扑并计算最短路。

别混淆距离向量不是完整拓扑；毒性逆转不能消除所有环路。

详解

01 解析

距离向量可能发生无穷计数，水平分割/毒性逆转用于缓解但不解决所有拓扑中的环路。链路状态也有传播与收敛时间。

02 例子

RIP 属于距离向量；OSPF 属于链路状态。

03 易错点

错误说法距离向量交换完整拓扑；链路状态只交换到目的地的距离。

自检：毒性逆转能无条件消除所有路由环路吗?

不能。

子网、广播地址与默认网关　主机数减 2 有适用范围；/31、/32 另看用途。　　易错　易混　近年考查　记忆概念　　要点

核心结论

IPv4 网络地址由 IP 与掩码按位与得到；常规子网排除全 0/全 1 主机号，默认网关通常需在本地可达链路。

别混淆主机数减 2 有适用范围；/31、/32 另看用途。

详解

01 解析

传统可用主机数规则对 /31 点到点与 /32 等特殊用途不直接适用。先看具体网段和用途，不能拿远端路由器地址作直接下一跳。

02 例子

192.168.1.60/27 属于 192.168.1.32/27，广播为 .63；网关 .62 在该子网内。

03 易错点

错误说法默认网关随便填一个路由器地址；可用主机数一律为 $2^{h}-2$。

自检：192.168.1.1 能否作为上述主机通常的本链路网关?

它不在该 /27 子网内，普通配置下不能直接这样使用。

SDN：控制平面与南北向接口　南向连控制器与设备；北向连应用与控制器。　冷门　易混　近年考查　要点

核心结论

SDN 分离控制与数据平面；南向接口连接控制器与转发设备，北向接口连接应用与控制器。

别混淆南向连控制器与设备；北向连应用与控制器。

详解

01 解析

数据平面根据流表匹配和转发，控制平面制定策略与下发规则。OpenFlow 是常见南向接口协议之一，不是所有 SDN 的唯一实现。

02 例子

控制器向交换机下发流表属于南向交互。

03 易错点

错误说法南向接口连接业务应用与控制器，交换机自己承担全部集中控制。

自检：应用向控制器请求策略属于哪个方向?

北向接口。

网络层计时：TTL、重组超时与路由老化　IP TTL、重组超时、路由老化是不同机制。　记忆概念　易错　易混　图解　要点

核心结论

IPv4 TTL 是首部中的生存限制字段；IP 重组超时用于清理不完整分片；RIP 更新/失效计时用于维护路由。

别混淆IP TTL、重组超时、路由老化是不同机制。

详解

01 解析

TTL 在现代转发中主要按跳数扣减，并非主机上“到时重发”的 RTO。IP 不因重组超时自动重传原数据报；端到端可靠性由上层协议另行处理。

02 例子

路由器 TTL 归零丢包与目的主机分片迟迟不全，是不同的失效场景。

03 易错点

错误说法TTL 就是一个 TCP 计时器；网络层所有时间参数都计数据重传。

04 图表



单独查看图片

参考资料

参考：RFC 1122：IP 重组与 TCP 计时机制

自检：分片重组计时器通常由中间路由器还是重组端维护?

通常由执行重组的目的主机 IP 层维护。

IP 分片重组超时：等不齐就丢弃　IP 等不齐分片则超时丢弃，不自动补传缺片。　　记忆概念　易错　易混　　要点

核心结论

重组必须设置超时，届时丢弃不完整的数据报；IP 本身不提供按缺片自动重传机制。

别混淆IP 等不齐分片则超时丢弃，不自动补传缺片。

详解

01 解析

超时不是片偏移，也不等于 RTT。不要将 IPv4 TTL 字段机械当成重组等待秒数。收到首片且满足 ICMP 发送条件时，可报告重组超时。

02 例子

到达第 1、3 片而第 2 片一直未到，等待过期后不能把不完整内容直接交给 TCP。

03 易错点

错误说法缺一个分片时，目的主机会永久保存已到分片，或重传缺片。

参考资料

参考：RFC 1122：IP 重组与 TCP 计时机制

自检：能否将不完整重组数据直接交给上层作正常数据?

不能，应按重组失败处理。

RIP 计时器：30 / 180 / 120 的标准例子　RIP：约 30 s 更新，180 s 失效，再等 120 s 删除。　　记忆概念　易错　易混　　要点

核心结论

RFC 2453 的典型参数：约每 30 s 定期更新，180 s 未获有效更新使路由超时，失效后再保留 120 s 供垃圾收集。

别混淆RIP：约 30 s 更新，180 s 失效，再等 120 s 删除。

详解

01 解析

路由失效时度量设为 16，停止用于正常转发，但暂保留并通告不可达。垃圾收集从失效阶段开始，不是与最初的 180 s 同时开始；厂商的保持/清除参数可能另有设置。

02 例子

无新路由等条件下，若从最近一次更新 t=0 计，t≈180 s 失效，再过 120 s 删除。

03 易错点

错误说法路由 180 秒失效后立刻从表中物理删除；120 秒从最初更新时间起计。

04 图表

要点对照

阶段	标准示例值	动作
周期更新	约 30 s	向邻居通告路由
路由超时	180 s	度量设 16，停止正常使用
垃圾收集	失效后 120 s	删除失效表项

参考资料

参考：RFC 2453 §3.8：RIP 定时器

自检：RIP 最大有效跳数与不可达值分别是什么?

最大有效跳数 15，不可达度量 16。

IPv4 地址与 NAT：私网、前缀和端口映射　172.16/12 仅覆盖第二字节 16～31；NAT 不等于普通转发。　　记忆概念　易错　易混　　要点

核心结论

IPv4 地址定位网络接口，CIDR 用前缀划网段。私网范围常见 10/8、172.16/12、192.168/16；NAT 可改 IP，NAPT 还映射端口。

别混淆172.16/12 仅覆盖第二字节 16～31；NAT 不等于普通转发。

详解

01 解析

NAT 可能影响端到端地址与校验和；它不是路由器正常转发时必须执行的操作。端口用于区分传输端点，不能替代 IP 路由前缀。

02 例子

多个内网主机共享一个公网地址时，可用不同外部端口区分会话。

03 易错点

错误说法私网地址可在全互联网不经转换唯一可路由；NAT 只修改 MAC。

自检：172.32.0.1 是否属于 172.16.0.0/12 私网范围?

不属于；该范围第二字节为 16～31。

IPv6：地址长度、首部与分片　IPv6 为 128 位地址、40 B 基本首部；中间路由器不分片。　　记忆概念　易错　易混　　要点

核心结论

IPv6 地址为 128 位，基本首部固定 40 B；中间路由器不按 IPv4方式分片，源端可使用分片扩展机制。

别混淆IPv6 为 128 位地址、40 B 基本首部；中间路由器不分片。

详解

01 解析

IPv6 无基本首部校验和，用邻居发现等代替 IPv4 ARP 的部分职责。地址压缩 :: 一处最多用一次，不能因此失去组数信息。

02 例子

一个 IPv6 基本首部含地址与下一首部等字段，额外功能可用扩展首部。

03 易错点

错误说法IPv6 地址仍为 32 位，路由器像 IPv4 一样按 MTU 随处分片。

自检：IPv6 是否使用 IPv4 的广播地址机制?

没有同样的广播地址类型，常通过组播等机制实现相应用途。

ICMP：差错报告与查询，不负责可靠重传　ICMP 报告差错，不替代 TCP 做可靠重传。　记忆概念　易错　易混　　要点

核心结论

ICMP 传递网络差错和诊断/查询信息，通常作为 IP 负载。ping 常使用 Echo 请求/应答。

别混淆ICMP 报告差错，不替代 TCP 做可靠重传。

详解

01 解析

ICMP 不等于网络层可靠传输。TTL 到期、目的不可达等可报告，但有发送限制；不能对 ICMP 差错再层层生成差错。traceroute 利用逐跳 TTL 等机制及响应。

02 例子

发出 ICMP 超时并不保证原数据报自动成功重发。

03 易错点

错误说法ICMP 替代 TCP 进行所有丢包重传，ping 属于 TCP 应用数据。

自检：ICMP 报文是否直接取代以太网 MAC 首部?

不是，仍要封装在 IP 和相应链路层之中。

RIP、OSPF、BGP：域内、域间与选路依据 RIP 跳数、OSPF 代价、BGP 路径与策略，度量不同。 记忆概念 易错 易混 要点

核心结论

RIP 是域内距离向量，常按跳数；OSPF 是域内链路状态，按配置代价；BGP 是域间路径向量，考虑 AS 路径与策略。

别混淆RIP 跳数、OSPF 代价、BGP 路径与策略，度量不同。

详解

01 解析

协议用途、交换信息和路由度量不同。BGP 的策略不等于最短地理距离；OSPF 区域组织也不是 TCP 连接窗口。RIP/OSPF 并非应用文件传输协议。

02 例子

RIP 的 16 代表不可达，不能因此认为其他协议也最大只有 15 跳。

03 易错点

错误说法BGP 是链路状态协议；所有协议都按最少跳数选全网路径。

自检：OSPF 与 BGP 是否都主要用于同一自治系统内部?

不，BGP 主要交换自治系统之间的可达与策略路由信息。

拥塞控制、流量控制与 QoS：链路也可控流 rwnd 大不证明网络不拥塞；流控不限于传输层。 记忆概念 易错 易混 要点

核心结论

流量控制匹配发送/接收能力，拥塞控制处理网络资源负载；链路层和传输层均可有各自的流控机制，QoS 涉及时延/吞吐等服务指标。

别混淆rwnd 大不证明网络不拥塞；流控不限于传输层。

详解

01 解析

瓶颈路由队列因网络拥塞增长不等于终端 rwnd=0。资源预留、调度、整形等可影响服务；不要把不同层机制混为一个计时器。

02 例子

发送端路由器出口拥挤时，终端接收缓冲仍可能有大量空间。

03 易错点

错误说法只有 TCP 才能出现流量控制；队列变长一定是接收主机缓冲不足。

自检：rwnd 大能否证明路径上完全不拥塞?

不能。

IPv4 首部：长度、标识、DF/MF 与片偏移 IPv4 首部长度 4 B、总长度 1 B、片偏移 8 B 为单位。 记忆概念 易错 易混 要点

核心结论

IPv4 首部长度以 4 B 为单位，总长度以 B 为单位，片偏移以 8 B 为单位；标识协助同一数据报分片重组，DF 禁止分片，MF 表示还有后续片。

别混淆IPv4 首部长度 4 B、总长度 1 B、片偏移 8 B 为单位。

详解

01 解析

1. 首部长度字段先乘 4 得首部字节数，总长度减去它才是 IP 数据部分长度。
2. DF=1 表示不许分片；若数据报超过下一链路 MTU，不能继续按普通分片计算，应按题设丢弃并处理差错。
3. 允许分片时再按 8 B 对齐非末片数据，并用 MF 标记是否还有后续片；总长度、首部长度、片偏移的单位各不相同。

补充原理与条件

基本首部 20 B，带选项可更长；MTU 限制所承载的 IP 数据报大小。源/目的地址、协议号、TTL 与首部校验字段作用不同，不能把端口写成 IP 首部固有字段。

02 例子

DF=1 且需要更小 MTU 而无法直接发送时，不能照常让路由器分片。

03 易错点

错误说法首部长度、总长度和片偏移全都以字节为单位。

自检：目的 TCP 端口在 IPv4 基本首部里吗?

不在，端口属于传输层首部。

ARP 缓存老化：地址映射不是永久不变　ARP 表是 IP→MAC；交换机表是 MAC→端口。　记忆概念　冷门　易混　要点

核心结论

ARP 缓存需淘汰或验证失效的 IP→MAC 映射；若用老化定时，超时值应允许按实现或配置调整。

别混淆ARP 表是 IP→MAC；交换机表是 MAC→端口。

详解

01 解析

换网卡、主机移动等可能使旧映射失效。缓存老化、等待 ARP 回答和 TCP 重传是不同机制；交换机 MAC 表老化也不是同一个表。不要背成所有设备固定若干分钟。

02 例子

同一 IP 后来对应新网卡时，旧 MAC 映射不能永久沿用。

03 易错点

错误说法ARP 解析一次就可永远不再验证，所有 OS 老化时间固定相同。

参考资料

参考：RFC 1122：ARP 缓存验证

自检：ARP 表老化等同于交换机 MAC 表老化吗?

不同，前者映 IP→MAC，后者常映 MAC→端口。

OSPF Hello/Dead：邻居存活与 LSA 老化　OSPF Dead 与 LSA Age 不同，不能套 RIP 的失效时间。　记忆概念　冷门　易混　要点

核心结论

Hello 周期发送以发现/维护邻居；Dead 等待过期表示一段时间未获邻居 Hello；LSA Age 管理链路状态通告的年龄，职责不同。

别混淆OSPF Dead 与 LSA Age 不同，不能套 RIP 的失效时间。

详解

01 解析

相邻接口的相关参数需匹配以正常形成邻居关系。10 s/40 s 是常见广播网配置例子，不是所有网络类型的固定值；具体题目给值时按题设。

02 例子

接口 Hello 为 10 s、Dead 为 40 s 时，不能只因漏掉一次 Hello 就立即按 Dead 超时处理。

03 易错点

错误说法OSPF 不收到一个 Hello 就必立刻宣告邻居失效；Dead 与 LSA Age 是同一个时间字段。

参考资料

参考：RFC 2328：OSPF 邻居与协议计时

自检：OSPF 的邻居存活超时能否直接套成 RIP 的 180 s?

不能，它们属于不同协议和参数体系。

BGP Keepalive/Hold：和 TCP 保活不是同一层　BGP KEEPALIVE 与 TCP 保活不同；Hold=0 时规则另论。　记忆概念　冷门　易混　要点

核心结论

BGP 有自己的 KEEPALIVE 消息和 Hold 机制，用于维护 BGP 会话；它们不同于 TCP 层可选保活。

别混淆BGP KEEPALIVE 与 TCP 保活不同；Hold=0 时规则另论。

详解

01 解析

Hold Time 按本端配置与对端 OPEN 声明取较小值；接收有效的 KEEPALIVE/UPDATE 可维护保持计时，期满会处理会话失效。协商为 0 时不启动 Hold Timer，也不定期发送 KEEPALIVE；不能硬套厂商默认值。

02 例子

即使 TCP 底层尚未给出断连错误，BGP 的保持时间也可能先超时。

03 易错点

错误说法BGP 使用 TCP，所以不需要自己的 KEEPALIVE；所有保持时间固定 180 s。

参考资料

参考：RFC 4271：BGP KEEPALIVE 与 Hold Timer

自检：TCP 保活开启是否就与 BGP KEEPALIVE 完全等价?

不等价，协议层、报文和维护对象均不同。

网络层协议速查：IPv4、IPv6、ICMP、IGMP 与 ARP　ARP 的章节归属与严格分层口径可能不同：复习常放网络层讲，RFC 1122 放链路层；ARP 报文直接由链路帧承载。　记忆概念　易混　要点

核心结论

IP 提供无连接、尽最大努力的分组交付；ICMP 报告差错和提供查询，不能把 IP 变成可靠交付服务。

别混淆ARP 的章节归属与严格分层口径可能不同：复习常放网络层讲，RFC 1122 放链路层；ARP 报文直接由链路帧承载。

详解

01 解析

若目标在同一子网，ARP 查询目标主机 MAC；否则先按路由表确定下一跳，再查询下一跳（通常为网关）的 MAC。ARP 请求通常广播、正常应答通常单播。教材中将 ARP 放在网络层辅助协议章节，并不表示它由 IP 承载。ping 通常使用 ICMP 回显；traceroute 的探测报文可因实现使用 UDP、ICMP 或 TCP。

02 例子

跨网访问时，以太网帧目的 MAC 可为本地网关，IP 数据报目的地址仍为远程服务器。

03 易错点

错误说法访问远程网站时，ARP 要获得远程服务器的 MAC；ICMP 有端口号。

04 图表

要点对照

协议	作用与特点	承载 / 易错点
IPv4	32 位地址；可变首部；尽最大努力交付	DF 允许时源主机及路由器可分片，目的主机重组；只校验首部
IPv6	128 位地址；固定基本首部 40 B；无首部校验和	路由器不分片；源主机可用分片扩展首部；无广播
ICMP / ICMPv6	差错报告、诊断和控制消息	IPv4 协议号 1；ICMPv6 下一首部值 58；无 TCP/UDP 端口
IGMP	IPv4 主机与相邻组播路由器管理组成员关系	IPv4 协议号 2；不负责自治系统间选路
ARP	本地链路上将下一跳 IPv4 地址解析为 MAC	以太网 EtherType 0x0806；不封装在 IP 数据报中；不跨路由转发
NDP（邻居发现）	IPv6 邻居解析、路由器发现、重复地址检测等	基于 ICMPv6，替代 IPv6 中的 ARP 等功能

参考资料

参考：RFC 791；RFC 8200；RFC 792；RFC 4861；RFC 1122

自检：IPv6 是否通过 ARP 解析邻居链路层地址?

否，使用基于 ICMPv6 的邻居发现。

路由协议速查：RIP、OSPF、BGP 的功能与承载 RIP：UDP 520；OSPF：直接用 IP，协议号 89；BGP：TCP 179。功能归类与报文封装要分开作答。 记忆概念 易混 要点

核心结论

路由选择属于网络层控制功能；RIP、OSPF、BGP 的实际报文承载不同，不能仅凭用途断定它们都直接封装在 IP 中。

别混淆RIP：UDP 520；OSPF：直接用 IP，协议号 89；BGP：TCP 179。功能归类与报文封装要分开作答。

详解

01 解析

1. 问“承担什么功能”时，RIP、OSPF、BGP 都是路由选择，放在网络层控制功能中讨论。
2. 问“报文由谁封装”时另看承载：RIP 用 UDP，BGP 用 TCP，OSPF 直接封装在 IP 中。
3. 因此端口 179 是 TCP 的 BGP 服务端口，89 是 IP 首部的 OSPF 协议号；两类号码不能在同一字段中比较。

补充原理与条件

408 按路由功能把三者集中在网络层学习。若问 TCP/IP 实际协议栈层次，RIP 和 BGP 运行于 UDP/TCP 之上，可按应用层协议理解；OSPF 直接运行于 IP 之上。必须说明采用的口径。OSPF 交换链路状态通告，不是定期把整个路由表发给邻居；BGP 不保证选择跳数最少或时延最低的路径。

02 例子

抓包中 IP 协议字段为 89 代表 OSPF；TCP 目的端口 179 才表示 BGP 常用监听端口。

03 易错点

错误说法BGP 总选最短路；OSPF 用 UDP 89 端口。

04 图表

要点对照

协议	范围 / 算法	主要特点	承载
RIPv2	域内 IGP；距离向量	按跳数；最多 15 跳，16 为不可达；周期和触发更新；收敛较慢	UDP 520
OSPFv2	域内 IGP；链路状态 / Dijkstra	按代价；区域划分；洪泛 LSA；同一区域路由器维护区域链路状态信息	IPv4 协议号 89，无端口
BGP-4	域间 EGP；路径向量	传播前缀与路径属性；按策略选路；AS_PATH 等属性帮助避免 AS 级环路	TCP 179

参考资料

参考：RFC 2453；RFC 2328；RFC 4271

自检：路由协议都属于网络层功能，为什么 BGP 仍使用 TCP?

功能描述与封装描述不同；BGP 用 TCP 可靠传送路由信息。

传输层

TCP 序号与确认号：字节计数　ACK 是下一个期待字节；SYN/FIN 各耗一个序号。　易错　易混　要点

核心结论

确认号表示下一期待的字节序号；数据按字节编号；SYN 和 FIN 各占一个序号，纯 ACK 不占。

别混淆ACK 是下一个期待字节；SYN/FIN 各耗一个序号。

详解

01 解析

1. 先找接收端已连续收到的最后一个字节，ACK 写它的序号加 1。
2. 若中间存在缺口，后面即使已缓存更高序号数据，累计 ACK 仍停在缺口起点。
3. 计算段结束序号时把 SYN、FIN 各算 1；纯 ACK 只确认、不消耗序号。

补充原理与条件

TCP 累积确认只确认连续前缀，不能因更高序号的数据到达就跨越缺口确认。序号按 32 位模空间处理。

02 例子

收到从 100 开始的 20 字节连续数据后，确认号为 120。

03 易错点

错误说法ACK 确认号表示最后收到的字节；纯 ACK 占一个序号。

自检：不带数据且无 SYN/FIN 的 ACK 是否消耗序号?

不消耗。

流量控制与拥塞控制：限制来源不同　还能新发的量，要从窗口上限扣掉在途未确认量。　易混　要点

核心结论

rwnd 限制接收端可接收量，cwnd 限制网络中的在途量。在题设的简化窗口模型中，新发上限= $\max\{0,\min(\text{rwnd},\text{cwnd})-F\}$，各项须使用相同单位。

别混淆还能新发的量，要从窗口上限扣掉在途未确认量。

详解

01 解析

$$W_{\text{new}} = \max(0, \min(\text{rwnd}, \text{cwnd}) - F)$$

常见考题简化模型：F 为已发送但未确认的数据量；所有量使用相同单位。

1. 先统一 rwnd、cwnd 与未确认数据量的单位，不能把 MSS 个数直接与字节数相减。
2. 取 $\min(\text{rwnd}, \text{cwnd})$ 得到题设简化模型的在途上限；再减去已发送但未确认的部分。
3. 余量小于 0 时按 0 处理，不存在“还能发负数个字节”。例如上限 4 MSS、在途 3 MSS，只能再新发 1 MSS。

补充原理与条件

rwnd 由接收端通告，cwnd 由发送端的拥塞控制维护。窗口表示允许在途的数据上限，不是每次收到 ACK 都能新增发送的数量；还要减去已发送但未确认的部分。该式是常见考题简化模型，实际发送还受应用数据量、发送策略及其他条件限制。

02 例子

rwnd=8 MSS，cwnd=4 MSS，限制通常为 4 MSS；已有 3 MSS 在途时只剩约 1 MSS 余量。

03 易错点

错误说法rwnd 与 cwnd 都由接收端直接通知。

变式

自编模拟变式 · 窗口缩小时的新增发送量

题设简化模型中 rwnd=1000 B、cwnd=3000 B，已有 1400 B 在途，是否还可新发 2000 B?

答案与解析

不可新发，余量按 0 计；先取上限 1000 B，再减在途 1400 B，而不是只看 cwnd。

关键区别：窗口变小不会把已经在途的数据瞬间收回；它限制接下来可新发的量。

参考资料

参考：RFC 9293：TCP 发送窗口

自检：收到较大 rwnd 是否必然可以按该值发送?

不一定，仍受 cwnd 和在途数据限制。

慢开始与拥塞避免：按 RTT 与 ACK 区分　翻倍按 RTT 估计，不是每收到一个 ACK 就翻倍。　冷门　易错　近年考查　要点

核心结论

理想条件下慢开始约每 RTT 翻倍，拥塞避免约每 RTT 增 1 MSS；每 ACK 的更新不同。

别混淆翻倍按 RTT 估计，不是每收到一个 ACK 就翻倍。

详解

01 解析

1. 先确认横轴按 RTT 还是按单个 ACK：慢开始是每个新 ACK 常增加约 1 MSS，从而整轮约翻倍。
2. 拥塞避免阶段按经典模型每个 RTT 约增加 1 MSS，不能把每个 ACK 都加 1 MSS。
3. 出现丢包时先区分超时与三次重复 ACK，再按题设 Tahoe/Reno 规则重设门限和窗口；不要把两种丢包事件统一成同一条折线。

补充原理与条件

经典教材模型与具体 TCP 实现有差异。超时与三次重复 ACK 后的更新也不同，题目若给 Tahoe/Reno 或简化规则应按其处理。

02 例子

当前窗口含多个段时，某一个 ACK 不会使整个窗口立即翻倍。

03 易错点

错误说法慢开始每收到一个 ACK 就翻倍；所有丢包处理都相同。

自检：“每 RTT 翻倍”能否改成“每 ACK 翻倍”？

不能。

连接关闭、半关闭与 TIME_WAIT　FIN 只关闭一个发送方向；收到 FIN 后仍可发剩余数据。　冷门　易混　近年考查　记忆概念　要点

核心结论

FIN 关闭该方向的发送，另一方向可继续；ACK 与 FIN 有时合并。主动关闭方通常进入 TIME_WAIT，标准时长为 2MSL。

别混淆FIN 只关闭一个发送方向；收到 FIN 后仍可发剩余数据。

详解

01 解析

1. 分别追踪两端的发送方向：一端发 FIN 只表示它不再发新数据，不禁止接收对方数据。

2. 收到 FIN 的一端先确认，若本地应用尚未关闭，仍可发送剩余数据，此时为半关闭。
3. 通常主动关闭方在收到对方 FIN 并发最后 ACK 后进入 TIME_WAIT；不要按谁是客户端或谁最后发报文死记。

补充原理与条件

TCP 是双向连接，两方向独立关闭。TIME_WAIT 有助于处理旧段和重传最后 ACK；不能仅用最后报文发送者判断所有状态。

02 例子

A 发 FIN 后，B 确认并继续发送剩余数据，之后 B 再发 FIN。

03 易错点

错误说法收到 FIN 后就不能再发送任何数据；四次挥手绝不可合并。

自检：被动方收到对方 FIN 后还能发送应用数据吗?

其发送方向尚未关闭时可以。

UDP/TCP 校验：伪首部不真正发送　伪首部参与校验但不单独发送；进位须回卷相加。

冷门　易错　近年考查　记忆概念　　要点

核心结论

UDP/TCP 使用反码和校验，计算时包含伪首部；伪首部借用 IP 地址等信息，不作为该传输层首部单独发送。

别混淆伪首部参与校验但不单独发送；进位须回卷相加。

详解

01 解析

16 位反码和出现最高位进位要回卷相加，最后按位取反。奇数字节长度需为校验补 0，补字节不算有效数据。

02 例子

算得 17 位和后，不能简单丢弃最高位而不回卷。

03 易错点

错误说法伪首部是另一个真正发送的 UDP 首部；UDP 校验与 CRC 相同。

自检：伪首部是否算入 UDP 长度字段?

不算，UDP 长度包含 UDP 首部和数据。

TCP 四类计时器：重传、持续、保活与时间等待 零窗口想持续，长期空闲想保活，未确认数据想 RTO。 记忆概念 易错 易混 图解 要点

核心结论

四类机制分别解决：未确认数据重传、零窗口探测、空闲连接存活探测、关闭后的 2MSL 等待。

别混淆零窗口想持续，长期空闲想保活，未确认数据想 RTO。

详解

01 解析

这些计时器不是统一的固定时长。RTO 由 RTT 等估计；持续计时器不是拥塞控制；保活通常可选；TIME_WAIT 不是接收窗口探测。

02 例子

题目说“窗口恢复通知可能丢失”应联想到持续计时器，而不是保活。

03 易错点

错误说法所有 TCP 定时器都是为了重传数据，超时就断开连接。

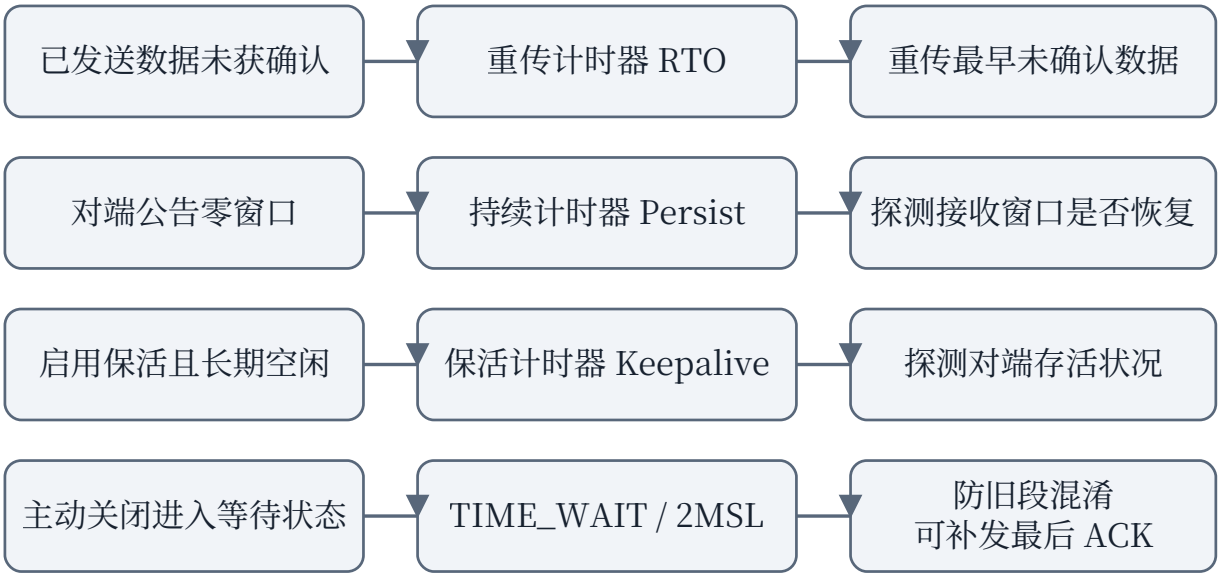
04 图表

要点对照

计时机制	触发场景	主要动作/目的
重传 RTO	已发送数据未及时确认	重传最早未确认数据，超时退避
持续 / Persist	对端公告零窗口	周期探测窗口，防止永久停发
保活 / Keepalive	启用保活且连接长期空闲	探测对端是否仍存活
TIME_WAIT	主动关闭进入等待状态	等待 2MSL；处理旧段，应答重传 FIN、补发最后 ACK

TCP 计时器：四种场景对照

同属 TCP，但触发条件不同；不是一套固定的秒数



“窗口更新丢失”想持续；“闲置连接”想保活

单独查看图片

参考资料

参考：RFC 9293：TCP 窗口、保活与关闭

自检：持续计时器和保活计时器的根本区别?

持续处理零窗口带来的停发风险；保活探测长期空闲连接的对端是否仍可达。

RTO、RTT 与 Karn：重传超时不是固定 RTT　有重传歧义的样本不估 RTT；波动更新用旧 SRTT。　　记忆概念　易错　易混　　要点

核心结论

RTO 依据平滑 RTT 和波动估计；无消除歧义手段时，不用已重传报文段测 RTT。

别混淆有重传歧义的样本不估 RTT；波动更新用旧 SRTT。

详解

01 解析

RFC 模型：RTO=SRTT+max(G,4×RTTVAR)，G 是计时精度。后续更新 RTTVAR 使用更新前的 SRTT。考试给简化加权公式或常量时，按题设。

02 例子

ACK 到来时若原报文和重传报文都可能导致它，无法明确对应哪一次发送时间。

03 易错点

错误说法RTO 总等于一次测得的 RTT；重传后任何 ACK 都可用来估计 RTT。

参考资料

参考：RFC 6298：TCP 重传计时器

自检：Karn 算法为什么排除重传段样本?

避免把 ACK 归属不明确的往返时间作为样本；时间戳等消歧方案另论。

超时重传与快速重传：触发条件不同　三次重复 ACK 可早于 RTO 触发重传。　　记忆概念　易错　易混　　要点

核心结论

快速重传由重复 ACK 等信号触发，可在 RTO 到期前重传；超时重传由重传计时器到期触发。

别混淆三次重复 ACK 可早于 RTO 触发重传。

详解

01 解析

经典教材模型中，两种事件对拥塞窗口的处理可以不同。RTO 超时通常指数退避；不能把 TCP Reno 的重复 ACK 处理套成超时处理。

02 例子

三个重复 ACK 已提示缺口时，不必再等待原重传计时器走完。

03 易错点

错误说法收到 3 个重复 ACK 等同于 RTO 到期。

参考资料

参考：RFC 6298：TCP 重传计时器

自检：快速重传一定先等待 RTO 到期吗?

不需要。

持续计时器：零窗口探测不是保活　零窗口探测是问新窗口，不是判断对端掉线。　　记忆概念　易错　易混　　要点

核心结论

零窗口使正常数据发送受限，发送方使用探测机制获取新窗口信息，避免双方永久等待。

别混淆零窗口探测是问新窗口，不是判断对端掉线。

详解

01 解析

对端可能已经空出缓冲区，但携带窗口更新的 ACK 不保证可靠送达。探测有助重新得知窗口；它不代表把 cwnd 当作 rwnd，更不等于认为对端已掉线。

02 例子

接收方窗口从 0 恢复到非 0，但通知丢失，发送方可通过探测发现恢复。

03 易错点

错误说法rwnd=0 后发送方永不再发送任何报文；窗口更新丢失只能等重传定时器。

参考资料

参考：RFC 9293：TCP 窗口、保活与关闭

自检：只要 rwnd=0，就应立即断开 TCP 吗?

不应如此，可继续零窗口探测，具体终止策略以题设或实现为准。

保活计时器：教材数值不等于所有实现　保活可选且可配置，不能把 2 小时当所有环境的常量。　　记忆概念　易错　易混　　要点

核心结论

TCP 保活可选，可按连接启用；长期空闲后探测，单次无响应不足以断言连接死亡。

别混淆保活可选且可配置，不能把 2 小时当所有环境的常量。

详解

01 解析

教材常以约 2 小时等数值举例；RFC 规定默认关闭，启用时默认空闲间隔不小于 2 小时。考试若另给时间和次数，按题设；不要把系统配置当永久标准。

02 例子

一个只是不发送业务数据的健康连接不必关闭；保活与应用层心跳也不是同一层机制。

03 易错点

错误说法保活是每个 TCP 连接必启用的机制；一次探测失败就判掉线。

参考资料

参考：RFC 9293：TCP 窗口、保活与关闭

自检：“保活周期固定为 2 小时”能否作为所有环境的结论?

不能，值可配置，题目有约定时按约定。

TIME_WAIT：2MSL 从哪里开始　2MSL 从进入 TIME_WAIT 计；收到重传 FIN 可重启等待。　　记忆概念　易错　易混　　要点

核心结论

主动关闭方通常在收到对方 FIN 并发送最后 ACK 后进入 TIME_WAIT，按该状态计 2MSL；再次收到 FIN 时可重新发送 ACK 并重启等待。

别混淆2MSL 从进入 TIME_WAIT 计；收到重传 FIN 可重启等待。

详解

01 解析

等待期间既给旧报文消失留时间，也给对方重传 FIN、自己补发 ACK 留时间。FIN 往返和数据传输耗时若题目要求总时长，应另行加上。

02 例子

只问 TIME_WAIT 驻留为 2MSL；问从连接建立到 CLOSED，就不能只答 2MSL。

03 易错点

错误说法2MSL 从发出第一个 FIN 时开始；最后 ACK 本身可靠重传。

考点联系

从建连到关闭，须把传播与 TIME_WAIT 分开计时。

变式

自编模拟变式 · 最后 ACK 丢失后的等待

主动关闭方已经进入 TIME_WAIT，随后又收到对方重传的 FIN，是否只需等最初那次 2MSL 到点?

答案与解析

典型处理是再次发送 ACK 并重新启动 2MSL 等待；不能机械沿用第一次等待的结束时刻。

关键区别：先定位收到重传 FIN 的时刻，再从重启计时处计算剩余等待。

参考资料

参考：RFC 9293：TCP 窗口、保活与关闭

自检：最后 ACK 丢失时，常由谁重传什么报文?

对方重传 FIN，TIME_WAIT 方再确认；纯 ACK 不按数据报文那样独立可靠重传。

延迟 ACK：接收端的延后确认　延迟 ACK 在接收端；RTO 在发送端。　　记忆概念　易错　易混　　要点

核心结论

延迟 ACK 是接收端短时延后发送确认的机制；RTO 是发送端等待确认的机制，职责和所在端不同。

别混淆延迟 ACK 在接收端；RTO 在发送端。

详解

01 解析

延迟 ACK 可减少独立确认或等待捎带，但不能无限等待。时间线若单独给出确认延迟，应按模型计入；若所给 RTT 已包含该延迟，则不能再次相加。确认号仍按累计确认解释。

02 例子

接收端短时等下一段以合并确认，会影响观察到的确认间隔，但不改变 ACK 的累计语义。

03 易错点

错误说法延迟 ACK 与发送端的超时重传计时器是同一个。

参考资料

参考：RFC 1122：IP 重组与 TCP 计时机制

自检：延迟 ACK 会将按字节的序号改成按报文段计数吗?

不会。

UDP/TCP、端口与套接字：报文和字节流　TCP 不保留应用写边界；UDP 保留数据报边界。　　记忆概念　易错　易混　　要点

核心结论

TCP 提供有连接的可靠字节流，不保留应用写入边界；UDP 提供无连接数据报，仍用端口复用/分用。

别混淆TCP 不保留应用写边界；UDP 保留数据报边界。

详解

01 解析

端点通常由地址、端口与协议关联；TCP 连接以双端组合区分。UDP 首部 8 B，TCP 基本首部 20 B，可有选项。可靠与实时适用性取决于需求，不是协议名称直接决定。

02 例子

一次发送 100 B、再发送 50 B，TCP 接收 API 可按不同批次交付，应用需自定义消息边界。

03 易错点

错误说法TCP 保留每次 write 的报文边界；UDP 无连接就不需要端口。

自检：UDP 同一数据报的应用边界是否通常保留?

通常保留；但应用仍需面对丢失、重排及最大长度等约束。

TCP 三次握手：两个方向的序号与确认　双方 SYN 各耗 1 序号；第三次 ACK 可携带数据。　　记忆概念　易错　易混　　要点

核心结论

典型握手：SYN(seq=x)；SYN+ACK(seq=y,ack=x+1)；ACK(seq=x+1,ack=y+1)。双方 SYN 各占 1 个序号。

别混淆双方 SYN 各耗 1 序号；第三次 ACK 可携带数据。

详解

01 解析

1. 客户端 SYN 设 seq=x，占用一个序号；服务器回答 ACK=x+1，同时发自己的 SYN，seq=y。
2. 服务器 SYN 也占一个序号，因此第三次握手确认号为 y+1，客户端序号通常为 x+1。
3. 若第三次只是 ACK，它不额外耗序号；下一段客户端数据仍可从 x+1 开始。

补充原理与条件

第三次 ACK 可带数据，纯 ACK 不耗序号。握手交换初始序号并确认两方向通信，不保证身份认证或消息加密；中途丢失可触发重传。

02 例子

若客户端初始 x=100，SYN 被确认后的下一个数据序号通常从 101 开始。

03 易错点

错误说法第三次握手必再带 SYN，客户端第一个数据从初始序号本身开始。

自检：服务端的初始 y 是否必等于客户端的 x?

不必，两端各有自己的序号空间。

TCP 首部字段：SYN/ACK/FIN/RST 与首部长度　TCP 首部长度按 4 B 计；窗口字段不是 cwnd。　记忆概念　易错　易混　要点

核心结论

ACK 标志指确认字段有效；SYN 建立同步，FIN 正常关闭发送方向，RST 重置连接。首部长度字段以 4 B 为单位。

别混淆TCP 首部长度按 4 B 计；窗口字段不是 cwnd。

详解

01 解析

基本首部 20 B，可含选项至 60 B；确认号、窗口、校验和等职责不同。RST 不是正常半关闭的替代说法，控制标志组合需结合连接状态判定。

02 例子

首部长度值 5 对应 20 B，不是 5 B。

03 易错点

错误说法ACK 标志为 0 时确认号仍总作为有效累积确认；FIN 和 RST 语义相同。

自检：TCP 首部中的窗口字段主要表达 rwnd 还是 cwnd?

表达接收窗口相关值，cwnd 是发送端拥塞控制状态，不直接放在该字段。

TCP 建立：两端何时进入 ESTABLISHED?　第三次 ACK 在途中：主动方可已建立，被动方仍等确认。　记忆概念　易混　图解　要点

核心结论

典型主动打开：CLOSED→SYN-SENT→ESTABLISHED；被动打开：CLOSED→LISTEN→SYN-RCVD→ESTABLISHED。SYN-RCVD 即 SYN-RECEIVED。

别混淆第三次 ACK 在途中：主动方可已建立，被动方仍等确认。

详解

01 解析

1. 分别给客户端和服务器画状态线，每条状态变化都以该端实际收到并处理的报文为准。
2. 客户端收到有效 SYN+ACK 后发送第三次 ACK 并进入 ESTABLISHED；服务器必须收到这个 ACK 才进入 ESTABLISHED。

3. 所以第三次 ACK 在途时两端状态可以不同；不能在图上仅写“第三次握手完成前两端都未建立”。

补充原理与条件

状态属于某一端。客户端处理有效 SYN+ACK 并发出第三次 ACK 时进入 ESTABLISHED；服务器收到有效第三次 ACK 后才进入 ESTABLISHED。判断图中 t_1 时刻必须分别看两端已经发送、收到并处理了什么，不能仅按握手次数笼统判断。

02 例子

第三次 ACK 正在链路中传播时，客户端可为 ESTABLISHED，服务器仍为 SYN-RCVD。

03 易错点

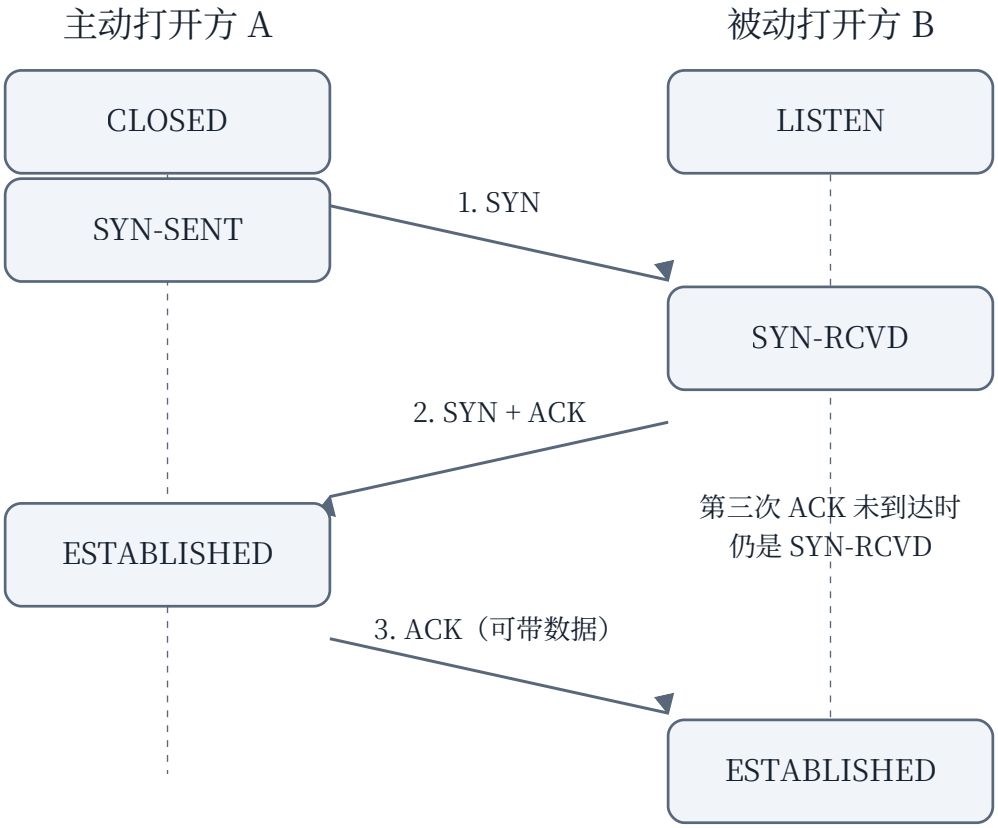
错误说法第二次握手发出后，双方都已处于 ESTABLISHED。

04 图表

要点对照

状态名称	中文含义	典型进入事件
CLOSED	关闭，无连接	初始状态或连接释放完成
LISTEN	监听	服务器被动打开，等候 SYN
SYN-SENT	同步已发送	主动打开并发送 SYN
SYN-RCVD / SYN-RECEIVED	同步已接收	被动方收到 SYN，发送 SYN+ACK
ESTABLISHED	连接已建立	主动方处理 SYN+ACK 并发送 ACK；被动方收到有效第三次 ACK

TCP 建立：分别观察两端状态



时间向下：A 发出第三次 ACK 时已建立；B 收到有效 ACK 后才建立。

典型主动/被动打开，时间向下。A 发送 SYN 后为 SYN-SENT；B 收到后为 SYN-RCVD。A 处理 SYN+ACK 并发送 ACK 时为 ESTABLISHED；B 收到该 ACK 后才为 ESTABLISHED。单独查看图片

参考资料

参考：RFC 9293 §3.3.2、§3.5：TCP 状态与关闭

自检：只看到客户端已发送第三次 ACK，能否断言服务器已建立?

不能；还需确认服务器是否已收到并处理该 ACK。

TCP 释放：各状态究竟在等什么？ FIN-WAIT-2 等对端 FIN，CLOSE-WAIT 等本地关闭。 记忆概念 易混 图解 要点

核心结论

典型主动关闭方：ESTABLISHED→FIN-WAIT-1→FIN-WAIT-2→TIME-WAIT→CLOSED；被动关闭方：ESTABLISHED→CLOSE-WAIT→LAST-ACK→CLOSED。

别混淆FIN-WAIT-2 等对端 FIN，CLOSE-WAIT 等本地关闭。

详解

01 解析

1. 主动方发 FIN 后为 FIN-WAIT-1；本端 FIN 被确认后为 FIN-WAIT-2，随后收到对方 FIN 才进入 TIME-WAIT。
2. 被动方收到 FIN 后为 CLOSE-WAIT，等待的是本地应用关闭，不是在等对方再发一个 FIN。
3. 本地应用关闭后被动方发 FIN，进入 LAST-ACK，等自己 FIN 的确认。遇到合并 FIN+ACK 或同时关闭，再检查相应分支。

补充原理与条件

主动/被动关闭按谁先发 FIN 区分，与客户端/服务器角色无固定绑定。FIN-WAIT-1 等本端 FIN 的确认或对端 FIN；FIN-WAIT-2 中本端 FIN 已获确认，等待对端 FIN。CLOSE-WAIT 等本地应用关闭发送方向；TIME-WAIT 等待 2MSL。

02 例子

A 发 FIN 后到 FIN-WAIT-1；B 收 FIN 并确认后到 CLOSE-WAIT，仍可发送剩余数据。

03 易错点

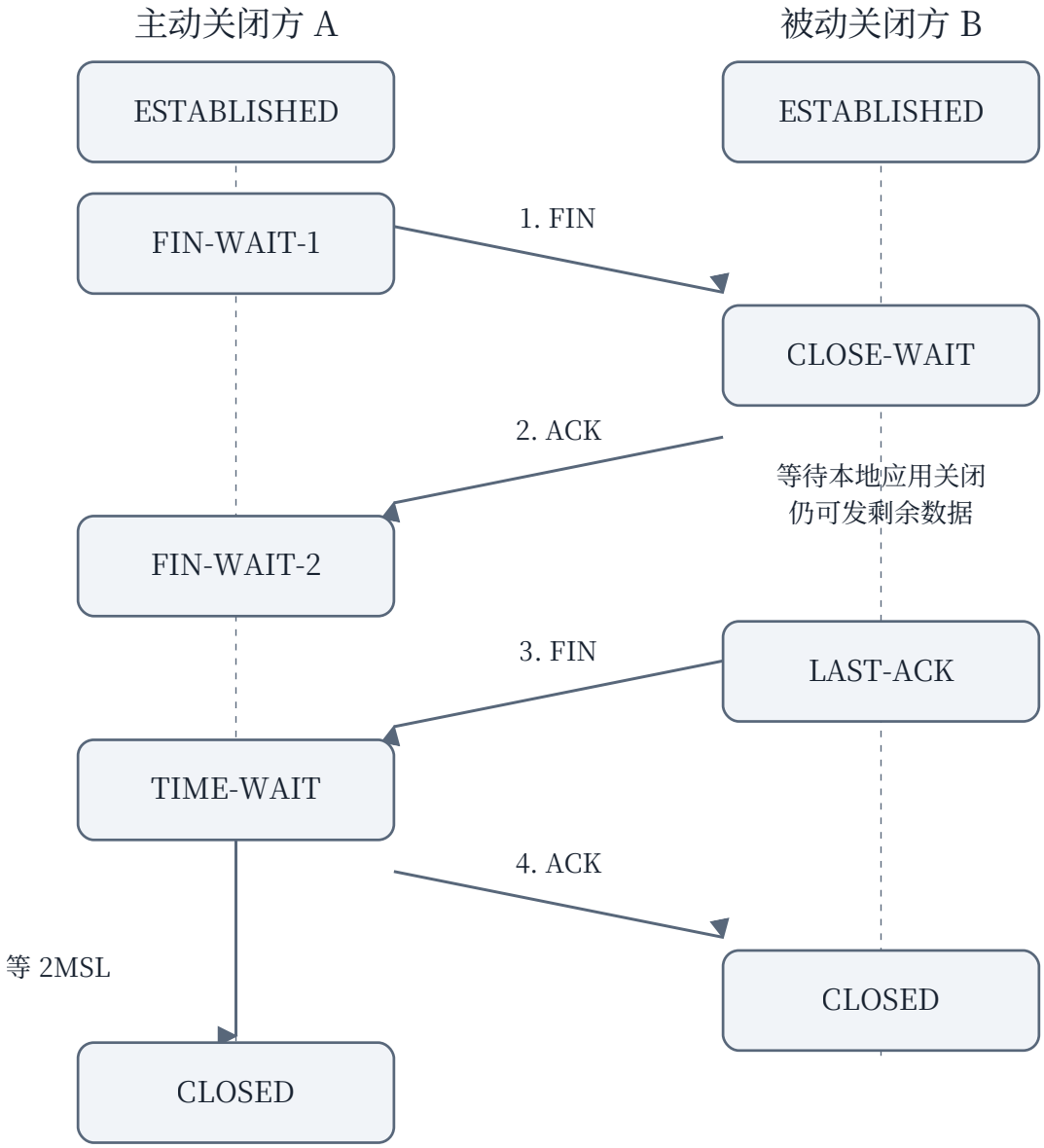
错误说法FIN-WAIT-2 是等待自己 FIN 的 ACK；CLOSE-WAIT 是等待 2MSL。

04 图表

要点对照

状态名称	中文含义	在等什么
FIN-WAIT-1	终止等待 1	本端 FIN 的 ACK 或对端 FIN
FIN-WAIT-2	终止等待 2	对端 FIN
CLOSE-WAIT	关闭等待	本地应用发起关闭
LAST-ACK	最后确认	本端 FIN 的 ACK
TIME-WAIT	时间等待	2MSL，期间可应答重传的 FIN
CLOSED	关闭	连接状态已释放

TCP 释放：谁在等 ACK，谁在等 FIN?



典型四报文过程；ACK 与 FIN 可合并，特殊分支见下一图。

A 先发 FIN 进入 FIN-WAIT-1，收到其确认后进入 FIN-WAIT-2。B 收到 FIN 进入 CLOSE-WAIT，本地关闭后发送 FIN 进入 LAST-ACK。A 收到 B 的 FIN 并确认，进入 TIME-WAIT，等待 2MSL 后关闭；B 收到最后 ACK 即关闭。单独查看图片

参考资料

参考：RFC 9293 §3.3.2、§3.5：TCP 状态与关闭

自检：发送最后 ACK 后，典型主动关闭方处于什么状态?

TIME-WAIT；不是立即 CLOSED。

TCP 特殊分支：CLOSING 与同时打开/关闭　同时关闭可双方 TIME-WAIT；合并 FIN+ACK 可跳过 FIN-WAIT-2。　　记忆概念　易混　图解　　要点

核心结论

先收到对端 FIN、但本端 FIN 尚未被确认时可进入 CLOSING；本端 FIN 再获确认后进入 TIME-WAIT。

别混淆同时关闭可双方 TIME-WAIT；合并 FIN+ACK 可跳过 FIN-WAIT-2。

详解

01 解析

若在 FIN-WAIT-1 收到同时确认本端 FIN 的有效 FIN+ACK，可直接进入 TIME-WAIT，跳过 FIN-WAIT-2。同时关闭时双方均可能进入 TIME-WAIT。同时打开时主动方也可经 SYN-RECEIVED；不能用“服务器专有状态”死记。

02 例子

双方几乎同时发 FIN，各自确认对方 FIN，再收到自己 FIN 的 ACK，双方均可 TIME-WAIT。

03 易错点

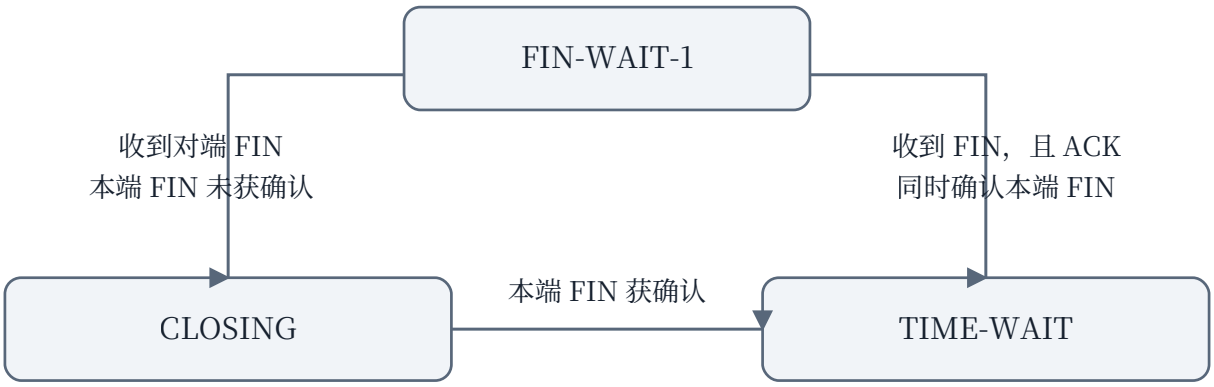
错误说法所有连接释放都必经过 FIN-WAIT-2；同时关闭只有一端 TIME-WAIT。

04 图表

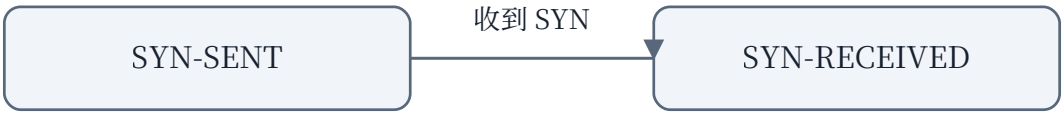
要点对照

情形	转换	关键条件
同时关闭的中间阶段	FIN-WAIT-1→CLOSING	收到 FIN，但本端 FIN 未获确认
随后确认到达	CLOSING→TIME-WAIT	本端 FIN 获确认
对端 FIN 与确认合并	FIN-WAIT-1→TIME-WAIT	收到 FIN 且本端 FIN 同时获确认
同时打开	SYN-SENT→SYN-RECEIVED	先收到对端不带 ACK 的 SYN

TCP 特殊分支：CLOSING 与同时打开



同时关闭：双方都可能经 CLOSING，最终都进入 TIME-WAIT。



同时打开：主动打开方也可进入 SYN-RECEIVED；该状态不专属于服务器。

FIN-WAIT-1 收到未确认本端 FIN 的对端 FIN，进入 CLOSING；随后本端 FIN 获确认进入 TIME-WAIT。若 FIN 与本端 FIN 的确认一起到达，可直接进入 TIME-WAIT。同时打开可从 SYN-SENT 到 SYN-RECEIVED。单独查看图片

参考资料

参考：RFC 9293 §3.3.2、§3.5：TCP 状态与关闭

自检：CLOSING 和 CLOSE-WAIT 的区别?

CLOSING 是自己已发 FIN、也收到对端 FIN，尚等自己 FIN 的确认；CLOSE-WAIT 是已收对端 FIN，仍等本地应用关闭。

拥塞控制名称：慢开始、拥塞避免、快重传、快恢复　快重传决定补发，快恢复决定窗口；不同 TCP 算法别混用。　　记忆概念　易混　图解　　要点

核心结论

慢开始、拥塞避免、快重传、快恢复是拥塞控制算法，不是连接状态。经典模型下，慢开始约每 RTT 翻倍，拥塞避免约每 RTT 加 1 MSS；三次重复 ACK 可触发快重传。

别混淆快重传决定补发，快恢复决定窗口；不同 TCP 算法别混用。

详解

01 解析

1. 读窗口曲线时先找指数增长段与线性增长段，分别对应经典模型的慢开始与拥塞避免。
2. 判断下降原因：超时和三次重复 ACK 是不同事件，门限与 cwnd 的重设规则须按题设版本。
3. Reno 三次重复 ACK 后可进入快恢复；不要把快重传当作连接状态，也不要将其窗口变化画成所有模型都相同。

补充原理与条件

快重传决定及时补发，快恢复决定丢包后的窗口调整。Reno、Tahoe 及教材简化模型不同：Reno 快恢复期间可能使用 ssthresh+3 MSS 的临时窗口，不能把所有实现写成“立即窗口减半”。按题设判断门限相等时采用的算法。

02 例子

曲线从指数增长转线性增长通常是慢开始转拥塞避免；超时后通常重新慢开始。

03 易错点

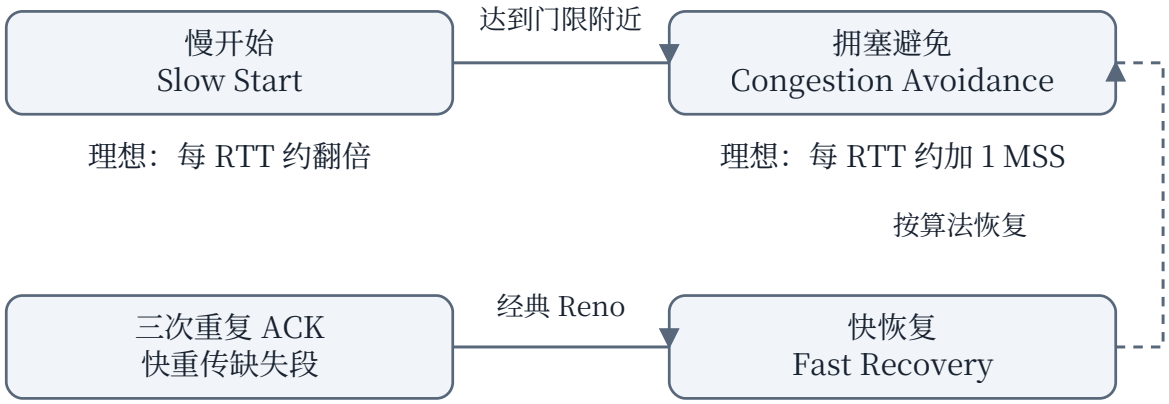
错误说法快恢复是 TCP 连接释放阶段；慢开始按每个 ACK 翻倍。

04 图表

要点对照

名称	英文	识别关键
慢开始	Slow Start	理想按 RTT 指数增长
拥塞避免	Congestion Avoidance	理想按 RTT 线性增长
快重传	Fast Retransmit	三次重复 ACK，尽早重传
快恢复	Fast Recovery	按指定算法调整窗口并恢复发送

拥塞控制：增长阶段与丢包处理



重传超时：经典模型重新慢开始；窗口数值与门限规则以题设为准。

慢开始通常呈每 RTT 指数增长，接近门限转拥塞避免的线性增长。三次重复 ACK 触发快重传，经典 Reno 配合快恢复后恢复拥塞避免；重传超时通常重新慢开始。单独查看图片

参考资料

参考：RFC 5681：TCP 拥塞控制

自检：三个重复 ACK 和超时后是否必采用相同过程?

不必；经典 Reno 的快恢复与超时后的慢开始不同。

传输层协议速查：TCP、UDP 与 QUIC　UDP 本身不保证可靠性，不代表使用 UDP 的应用不能实现可靠传输；QUIC 就基于 UDP 实现可靠流。　　记忆概念　易混　　要点

核心结论

TCP 是面向连接的可靠字节流；UDP 是无连接的尽最大努力报文服务。二者都用端口实现进程复用与分用。

别混淆UDP 本身不保证可靠性，不代表使用 UDP 的应用不能实现可靠传输；QUIC 就基于 UDP 实现可靠流。

详解

01 解析

TCP 可靠性不等于应用必定成功：连接失败仍需通知应用，也不能保证实时性。QUIC 是基于 UDP 的安全传输协议，提供可靠多流、拥塞控制等能力，HTTP/3 使用它；UDP 首部之后承载 QUIC 报文。UDP 校验和只能检错，不能保证恢复丢失数据。

02 例子

发送方连续写入两次 100 B，TCP 接收方可能一次读出 200 B，也可能分多次读取。

03 易错点

错误说法TCP 每次 send 都对应接收端一次 recv；UDP 不能承载可靠协议。

04 图表

要点对照

维度	TCP	UDP
连接与交付	建立连接；可靠、有序交付字节流	无连接；不保证到达、顺序或去重
数据边界	不保留应用写入边界，应用自行分帧	保留报文边界，一次提交对应一个 UDP 数据报
控制	确认重传、流量控制、拥塞控制	协议本身无确认重传、流量控制或拥塞控制
基本首部	最少 20 B，最大 60 B	固定 8 B
通信与典型用途	点对点连接；Web、文件、邮件等	支持单播、组播及 IPv4 广播等使用方式；DNS、DHCP、实时应用等
校验	首部、数据及伪首部参与校验	同样含伪首部；IPv4 可不使用校验和，IPv6 通常必须使用

参考资料

参考：RFC 9293；RFC 768；RFC 9000；RFC 9114

自检：HTTP/3 基于 UDP，是否说明 HTTP/3 不可靠?

否；QUIC 在 UDP 之上提供可靠流等传输能力。

应用层

DNS：递归/迭代与 UDP/TCP　缓存命中可省查询；DNS 既可 UDP，也可 TCP。　冷门易混　要点

核心结论

主机通常向递归解析器查询，解析器可迭代访问权威体系；DNS 既可用 UDP，也可用 TCP。

别混淆缓存命中可省查询；DNS 既可 UDP，也可 TCP。

详解

01 解析

1. 先查题目是否已有 DNS 缓存；命中就停止，不重复计算根、顶级域和权威查询。
2. 递归问的是“代查并给最终结果”，迭代问的是“给下一步该找谁”；这是查询方式，不是 UDP/TCP 的对应关系。
3. 传输方式另行判断：普通查询可用 UDP，必要时可用 TCP，区域传送通常使用 TCP。

补充原理与条件

缓存可缩短流程，不能每次都按“根→顶级域→权威”完整计算。区域传送等常用 TCP，响应截断后可重试 TCP。

02 例子

本地解析器命中缓存时，不必访问根服务器。

03 易错点

错误说法DNS 只使用 UDP；客户端自己一定迭代访问所有服务器。

自检：DNS 能否使用 TCP?

可以，不能绝对说只用 UDP。

HTTP、FTP 与邮件：连接和服务分工 FTP 分控制/数据连接；SMTP 发邮件，POP3/IMAP 访问邮箱。 冷门 易混 要点

核心结论

FTP 分控制与数据连接，主动/被动模式影响数据连接发起方；SMTP 用于邮件传输，POP3/IMAP 用于访问邮件。

别混淆FTP 分控制/数据连接；SMTP 发邮件，POP3/IMAP 访问邮箱。

详解

01 解析

HTTP 持久连接复用连接，但不等于 HTTP/1.1 普通串行请求自动并行。涉及持久连接或流水线请求时，须按题设版本与规则判断。

02 例子

FTP 控制连接常用端口 21，不能因此断言所有数据连接都固定由服务器端口 20 发起。

03 易错点

错误说法FTP 控制与数据同一连接；SMTP 用于收取邮箱邮件。

自检：IMAP 和 SMTP 是否承担相同任务?

不同，IMAP 访问邮箱，SMTP 传送邮件。

DNS 记录与缓存：A/AAAA/CNAME/MX　DNS TTL 是缓存时间，IP TTL 主要限制转发跳数。　记忆概念　易错　易混　要点

核心结论

A 映 IPv4，AAAA 映 IPv6，CNAME 表示别名，MX 指示邮件交换。DNS TTL 控制缓存有效期，和 IP TTL 不同。

别混淆DNS TTL 是缓存时间，IP TTL 主要限制转发跳数。

详解

01 解析

一个名字可有多条记录，别名解析可能需继续查询。缓存减少查询次数但可能短时保留旧数据；权威回答与递归解析器缓存回答要区分。

02 例子

MX 是邮件服务器相关记录，不等于浏览器直接访问的网页服务器 A 记录。

03 易错点

错误说法域名查询返回的永远只有一条 IPv4 地址，TTL 表示 TCP 时间等待。

自检：DNS TTL 用完是否意味着 IP 数据报此刻到达最后一跳?

不是，DNS 缓存 TTL 与 IP 首部 TTL 完全不同。

DHCP 与常见端口：分配配置，不是解析域名　DHCP 配地址等参数，DNS 解析名字；67 服务端、68 客户端。　记忆概念　易错　易混　要点

核心结论

DHCP 为主机提供租约式 IP 配置，常见流程 Discover→Offer→Request→Ack；常用 UDP 服务端 67、客户端 68。

别混淆DHCP 配地址等参数，DNS 解析名字；67 服务端、68 客户端。

详解

01 解析

初始客户端尚无可用 IP，常通过广播发现。租约续租不同于 TCP 保活；DNS、DHCP、HTTP、邮件协议分工不同。端口是常见默认约定，服务可另配置。

02 例子

DHCP 可以同时给网关和 DNS 服务器配置，后者由 DNS 服务执行名字解析。

03 易错点

错误说法DHCP 只分配 MAC 地址；DISCOVER 是 TCP 建立连接报文。

04 图表

要点对照

服务	常见默认端口	主要用途
HTTP / HTTPS	80 / 443	网页请求 / 加密访问
DNS	53（UDP/TCP）	域名记录查询
DHCP	67 / 68（UDP）	服务器 / 客户端配置
FTP 控制	21（TCP）	文件传输控制通道
SMTP	25（TCP）	邮件传输
POP3 / IMAP	110 / 143（TCP）	邮件访问

自检：常见 HTTP、HTTPS、DNS、FTP 控制、SMTP、POP3、IMAP 的默认端口?

分别常用 80、443、53、21、25、110、143；安全扩展或自定义服务可能另用端口。

HTTP 请求响应与持久连接：页面由多个对象构成　持久连接省重复握手，不等于自动并行返回所有对象。　　记忆概念　易错　易混　　要点

核心结论

HTTP 定义请求方法、资源和响应状态；持久连接复用已建立连接，减少重复握手，但是否流水线/并行需看题设。

别混淆持久连接省重复握手，不等于自动并行返回所有对象。

详解

01 解析

页面可能含 HTML 和多个嵌入对象，DNS、TCP、请求/响应与对象发送时间要分开。GET、POST、HEAD 等方法不同；HTTP 200/404 是应用响应，不是 TCP 连接状态。

02 例子

一个 HTML 引用 3 张图片，不能仅因只有一页就只算一次资源请求。

03 易错点

错误说法HTTP/1.1 持久连接自动意味着所有对象同时返回；HTTP 状态码与 TCP 状态相同。

自检：使用 HTTP 持久连接是否每个对象都需要新的 TCP 握手?

通常不需要，可在已有连接上复用；具体请求方式仍需看题设。

FTP 主动/被动：控制连接和数据连接　FTP 主动/被动改变数据连接发起方，端口 20 不通用。　　记忆概念　易错　易混　　要点

核心结论

FTP 控制连接常到服务器 TCP 21；主动模式典型由服务器发起数据连接，被动模式由客户端连接服务器指定数据端口。

别混淆FTP 主动/被动改变数据连接发起方，端口 20 不通用。

详解

01 解析

控制命令与文件内容分通道；不同文件或目录列表可使用单独数据连接。NAT/防火墙可影响主动模式可达，不能把端口 20 当所有模式的固定结论。

02 例子

被动模式中服务器返回可连接数据端口，客户端主动去连该端口。

03 易错点

错误说法所有 FTP 数据连接都由服务器端口 20 发起到客户端。

自检：FTP 传输目录列表是否也可能走数据连接?

可以，数据连接不只用于文件正文。

SMTP、POP3、IMAP 与 MIME：发、收、编码　SMTP 传邮件，POP3/IMAP 访邮箱，MIME 定内容格式。　　记忆概念　易错　易混　　要点

核心结论

SMTP 负责邮件传输；POP3/IMAP 访问邮箱；MIME 扩展邮件内容格式，使非 ASCII 内容和附件可表示。

别混淆SMTP 传邮件，POP3/IMAP 访邮箱，MIME 定内容格式。

详解

01 解析

邮件客户端到提交服务器、服务器之间传递、收件客户端取信是不同阶段。IMAP 支持服务器端邮箱组织与同步；默认端口和加密/提交端口需按题设区别。

02 例子

附件可通过 MIME 编码随邮件内容传递，但最终传输仍依赖相应邮件协议。

03 易错点

错误说法MIME 是传送邮件的可靠传输协议；SMTP 负责用户所有邮箱同步。

自检：MIME 是否替代 TCP 或 SMTP 完成网络传送?

不替代，它描述内容类型与编码格式。

DHCP 报文与租约状态：DORA、BOUND、RENEWING、REBINDING T1 通常单播续租，T2 通常广播重绑定。 记忆概念 易混 图解 要点

核心结论

初次典型分配为 DHCPDISCOVER→DHCPOFFER→DHCPREQUEST→DHCPACK；客户端状态与报文名称是两组概念。

别混淆T1 通常单播续租，T2 通常广播重绑定。

详解

01 解析

默认 T1 为租期的 50%，T2 为 87.5%，服务器可另给参数。T1 起通常向原服务器单播续租；T2 起广播重绑定。租约到期仍未续上需停止使用该地址并重新初始化。重启确认旧地址可走 INIT-REBOOT 等分支，不必完整发现。

02 例子

收到 Offer 尚不是最终获得租约；通常需完成 Request/ACK 及必要地址检查。

03 易错点

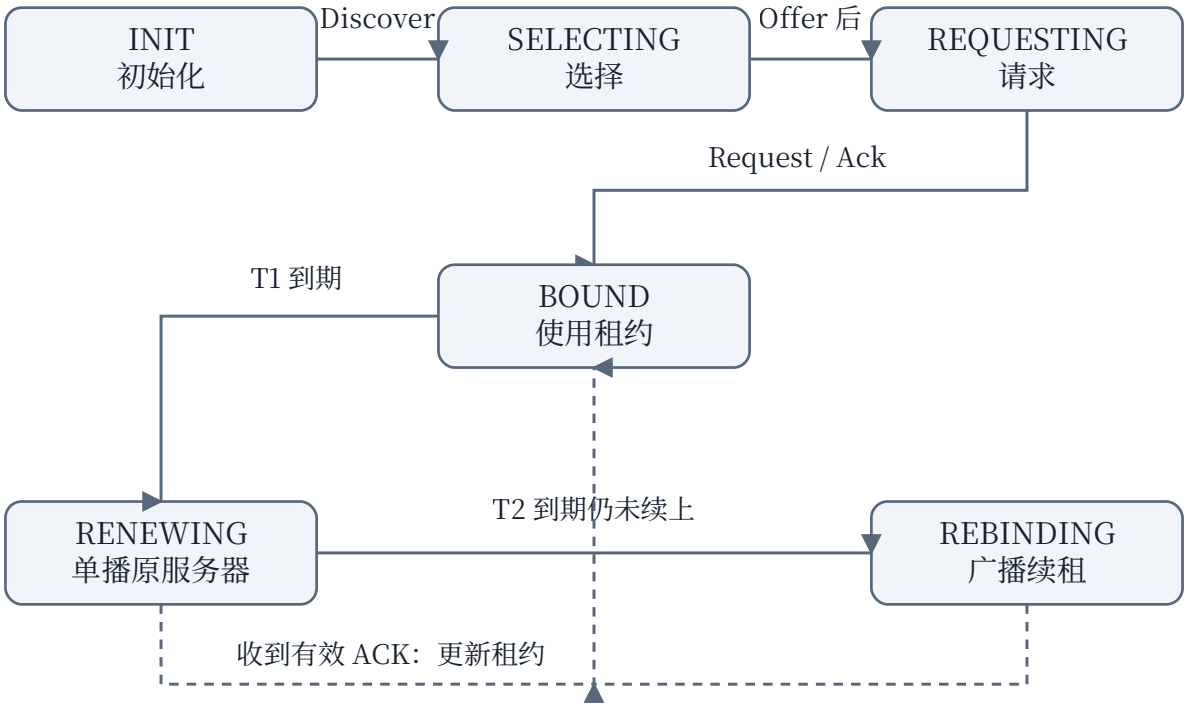
错误说法Discover、Offer 是 TCP 的连接状态；续租必须重新执行完整 DORA。

04 图表

要点对照

状态或报文组	名称/阶段	含义
DORA	Discover / Offer / Request / Ack	发现、提供、请求、确认
INIT→SELECTING→REQUESTING	初次分配主路径	寻找服务器并请求配置
BOUND	已绑定	已获得并使用租约
RENEWING	更新	T1 起通常单播续租
REBINDING	重绑定	T2 起通常广播续租

DHCP：报文流与租约状态分开记



默认 T1 = 50% 租期，T2 = 87.5%；到期未续上则停用地址、重新初始化。

主路径 INIT、SELECTING、REQUESTING、BOUND；报文为 Discover、Offer、Request、Ack。租期达到 T1 通常单播续租，达到 T2 通常广播重绑定。成功续租回到 BOUND。单独查看图片

参考资料

参考：RFC 2131：DHCP

自检：RENEWING 和 REBINDING 的主要区别?

前者通常向原服务器单播续租，后者广播寻求可续租的服务器。

应用层协议速查：Web、域名、文件、邮件与配置　DNS 不只用 UDP；HTTP 不总用 TCP；SMTP 负责发送与中继，POP3/IMAP 负责访问邮箱。　　记忆概念　易混　　要点

核心结论

应用层定义应用报文格式与交互规则；应用层协议使用 TCP 或 UDP，不会因此变成传输层协议。

别混淆DNS 不只用 UDP；HTTP 不总用 TCP；SMTP 负责发送与中继，POP3/IMAP 负责访问邮箱。

详解

01 解析

端口是默认或常用服务端口，并非协议不可改变的身份。HTTP 无状态并不等于非持久连接：HTTP/1.1 默认持久连接。DNS 的查询方式与传输协议是两个维度；不能把递归等同于 TCP、迭代等同于 UDP，也不能把是否超过 512 B 当成现代 DNS 唯一判断条件。DHCP 初次获取常用广播，但续租等阶段可单播；跨子网可使用中继。

02 例子

浏览器访问 HTTPS 页面前可能先用 DNS 查地址，再通过 TCP+TLS 或 QUIC 建立通信。

03 易错点

错误说法FTP 数据连接总是用 20；MIME 负责发送邮件；所有 DHCP 报文都是广播。

04 图表

要点对照

协议	用途与特点	常见承载 / 端口
HTTP/1.1、HTTP/2	请求—响应；HTTP 本身无状态，可借助 Cookie 等保持会话信息	通常 TCP；HTTP 80、HTTPS 443；HTTP/2 常用于 TLS 上
HTTP/3	HTTP 语义映射到 QUIC	QUIC / UDP，HTTPS 常用 443
DNS	分布式层次命名；支持递归/迭代、缓存和多种资源记录	传统 DNS 用 UDP 或 TCP 53；区域传送用 TCP；截断后可改用 TCP
FTP	分离控制连接与数据连接；主动 / 被动两种数据建连方式	TCP 21 控制；主动数据连接服务器通常用 20，被动数据端口由协商确定
SMTP	发送邮件及邮件服务器之间中继；推送	TCP 25 常用于服务器中继；587 常用于邮件提交
POP3	从服务器获取邮件；支持下载后保留或删除	TCP 110；隐式 TLS 常用 995
IMAP	服务器端管理邮箱、同步文件夹和邮件状态	TCP 143；隐式 TLS 常用 993
MIME	扩展邮件内容类型与编码，支持附件和多媒体	不是独立传输协议，没有独立端口
DHCPv4	动态分配地址及掩码、网关、DNS 等参数；租约机制	UDP：服务器 67、客户端 68；典型初次获取 DORA
SNMP	网络管理；管理站查询/设置，代理发送通知	通常 UDP 161 请求响应、162 通知
TFTP	简单文件传输，功能较少，基于块确认重传	UDP；初始请求到服务器 69，后续使用协商的传输标识端口

协议	用途与特点	常见承载 / 端口
SSH / Telnet	远程登录；SSH 加密，Telnet 本身明文	TCP 22 / TCP 23
NTP	网络时间同步	通常 UDP 123

参考资料

参考：RFC 9110；RFC 9112；RFC 9113；RFC 9114；RFC 1035；RFC 7766；RFC 959；RFC 5321；RFC 1939；RFC 9051；RFC 2131；RFC 1350

自检：网页邮箱收发邮件时，浏览器与网站之间一定使用 SMTP 和 POP3 吗?

不一定；浏览器通常通过 HTTPS 访问网站，邮件后端再使用相关邮件协议。

阅读与复习

本页保留个人错题、易错辨析、推理步骤、例子与自检。先确认正确结论，再展开原理和条件；不要脱离题设直接套用公式。

系统知识手册按四科知识脉络展开；本页用于针对性纠错，计算专项单独练习分步计算。三个入口独立，复习时可以交叉对照。

- 蓝色提示结论与原理，红色突出关键条件，橙色标记易错关系，紫色用于例子与计算，绿色用于自检和记忆。
- “展开完整解析”会展开当前筛选范围的原理、图表与答案；打印也只包含当前结果。
- 复习标记保存在当前浏览器。原有知识点编号和标记保持连续，系统手册使用独立记录。